

王道考研——数据结构

WWW.CSKAOYAN.COM

第五章 树与二叉树

1

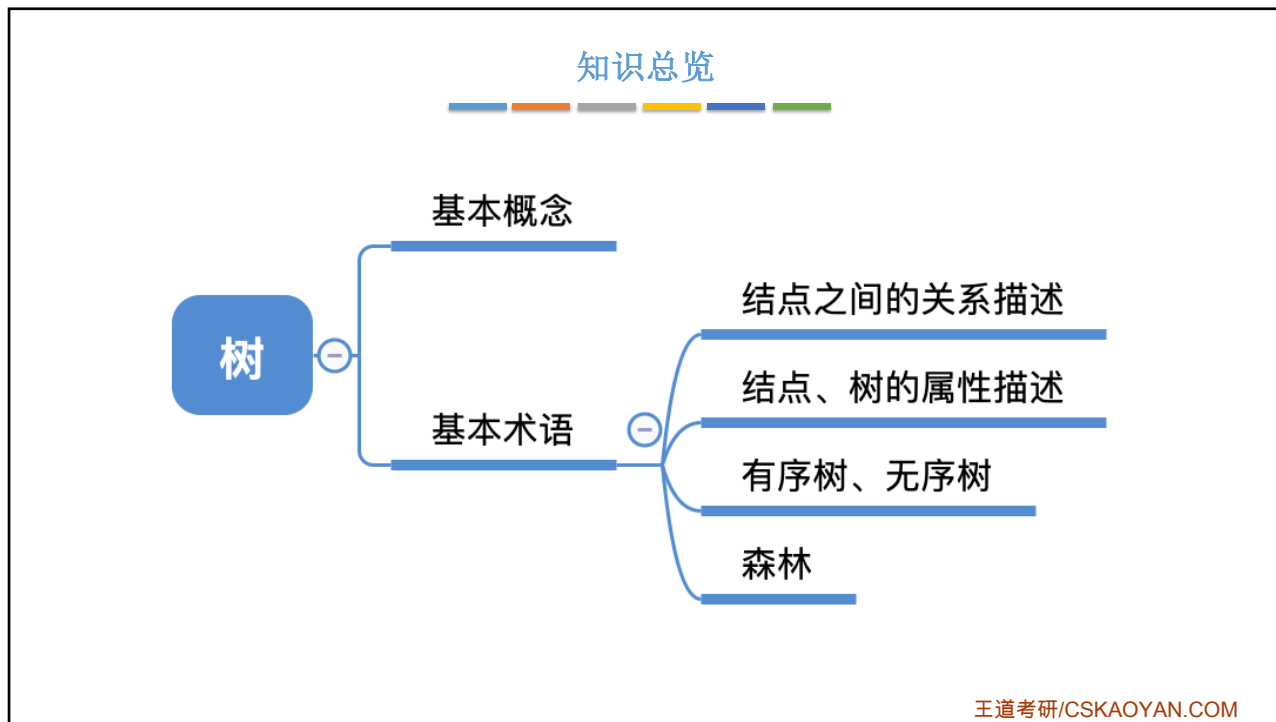
本节内容

树

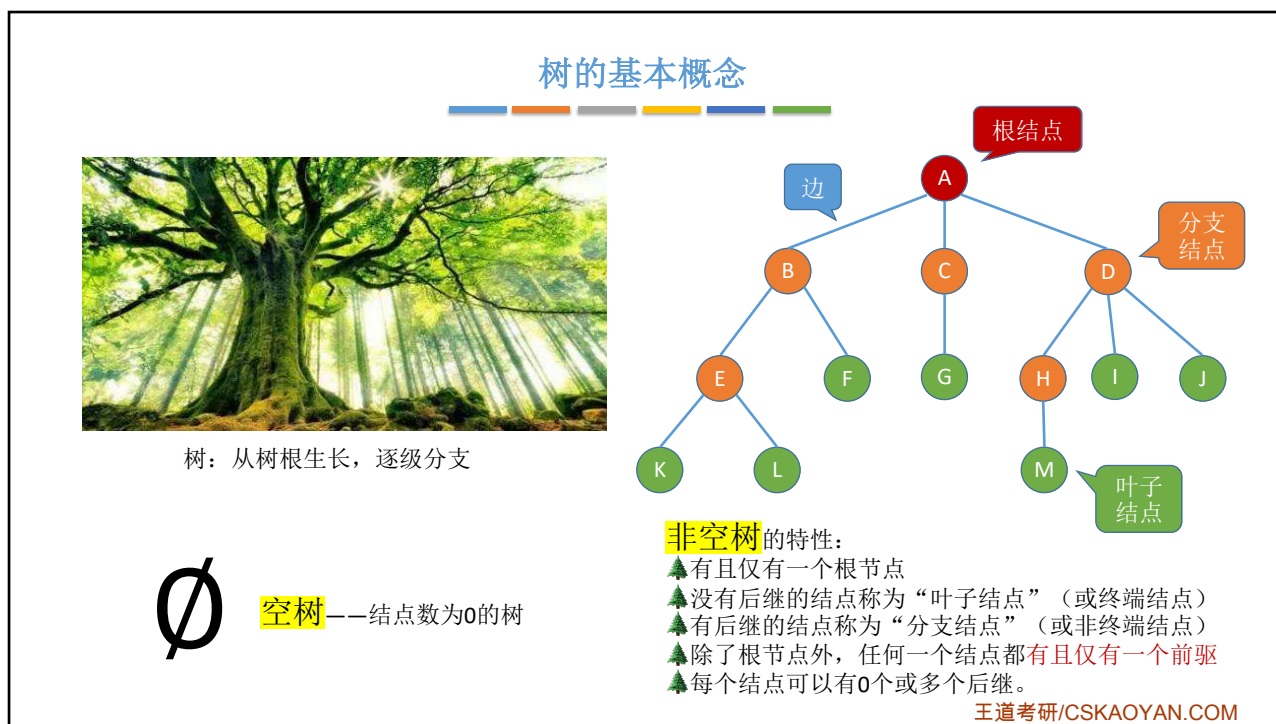
定义
基本术语

王道考研/CSKAOYAN.COM

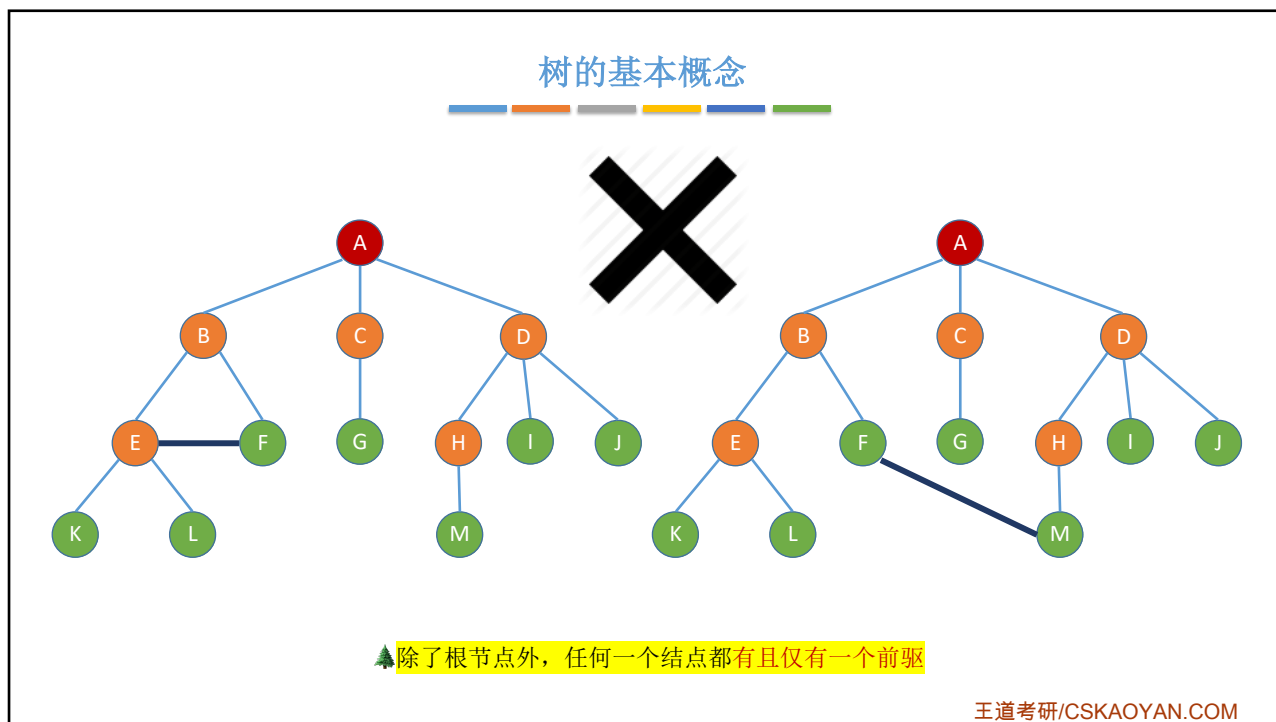
2



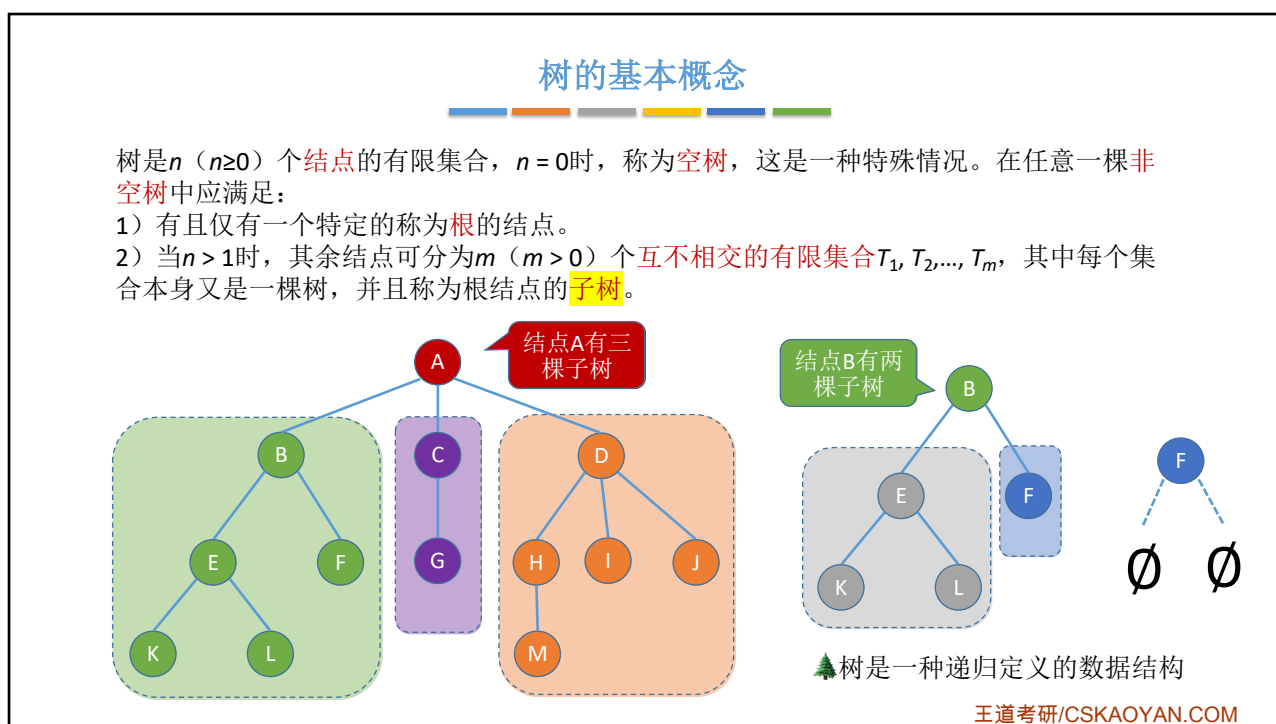
3



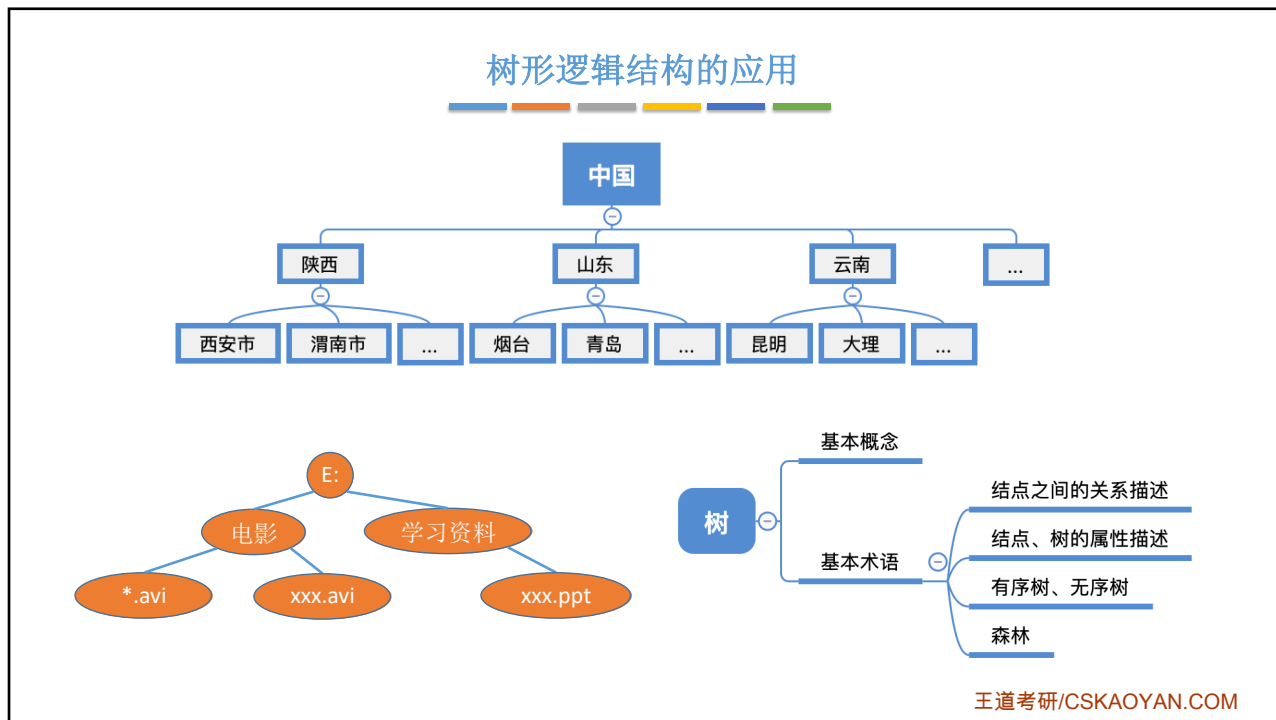
4



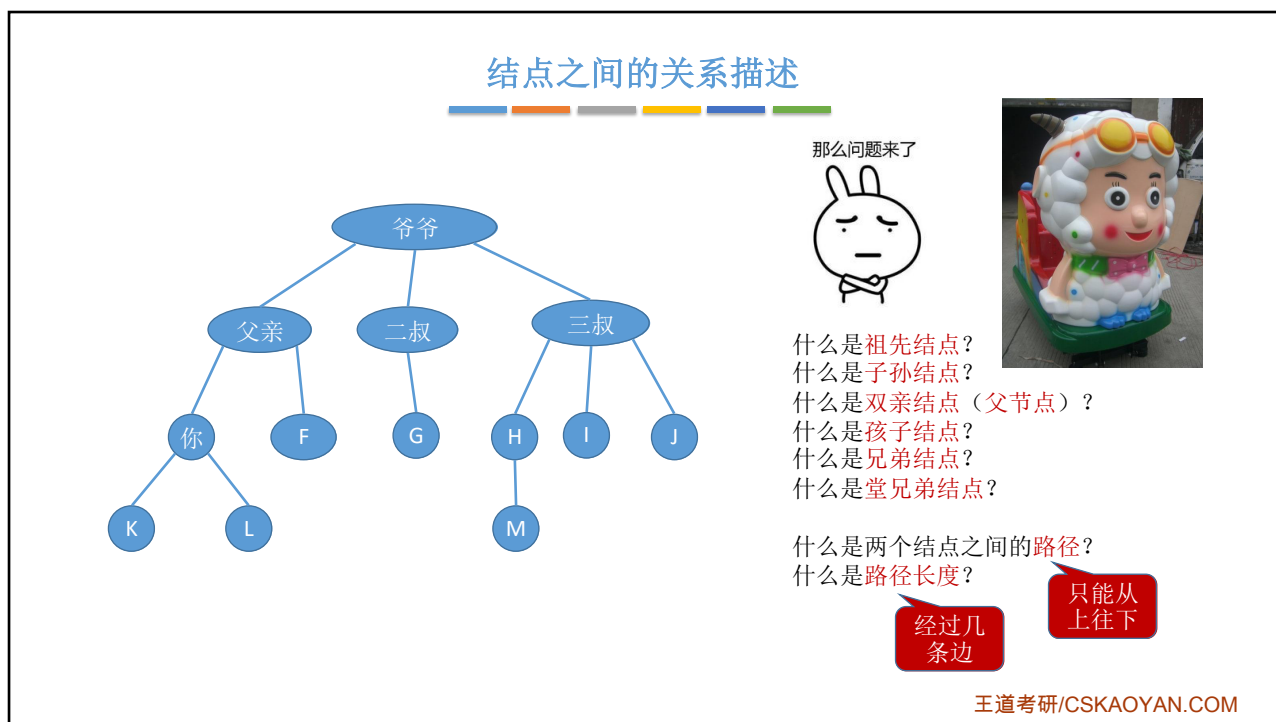
5



6

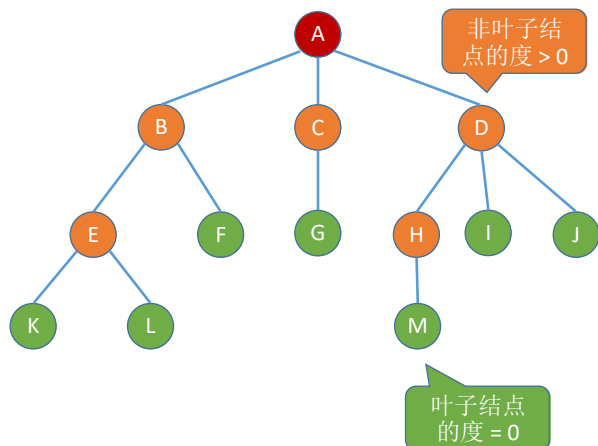


7



8

结点、树的属性描述



默认从1开始

属性:
结点的层次(深度)——从上往下数

结点的高度——从下往上数

树的高度(深度)——总共多少层

结点的度——有几个孩子(分支)

树的度——各结点的度的最大值

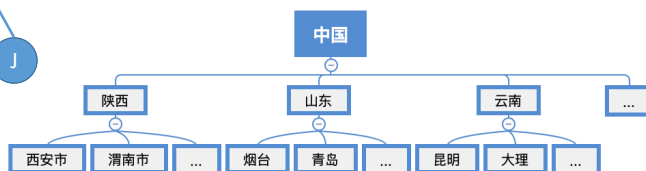
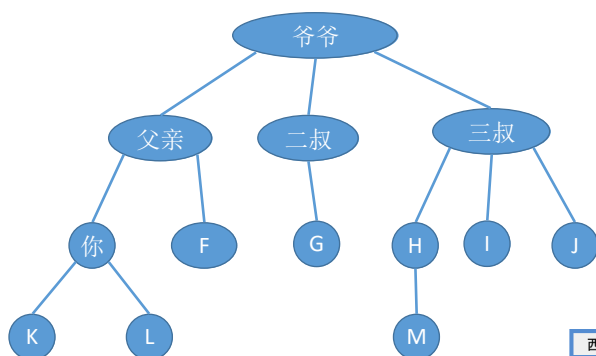
王道考研/CSKAOYAN.COM

9

有序树 v.s 无序树

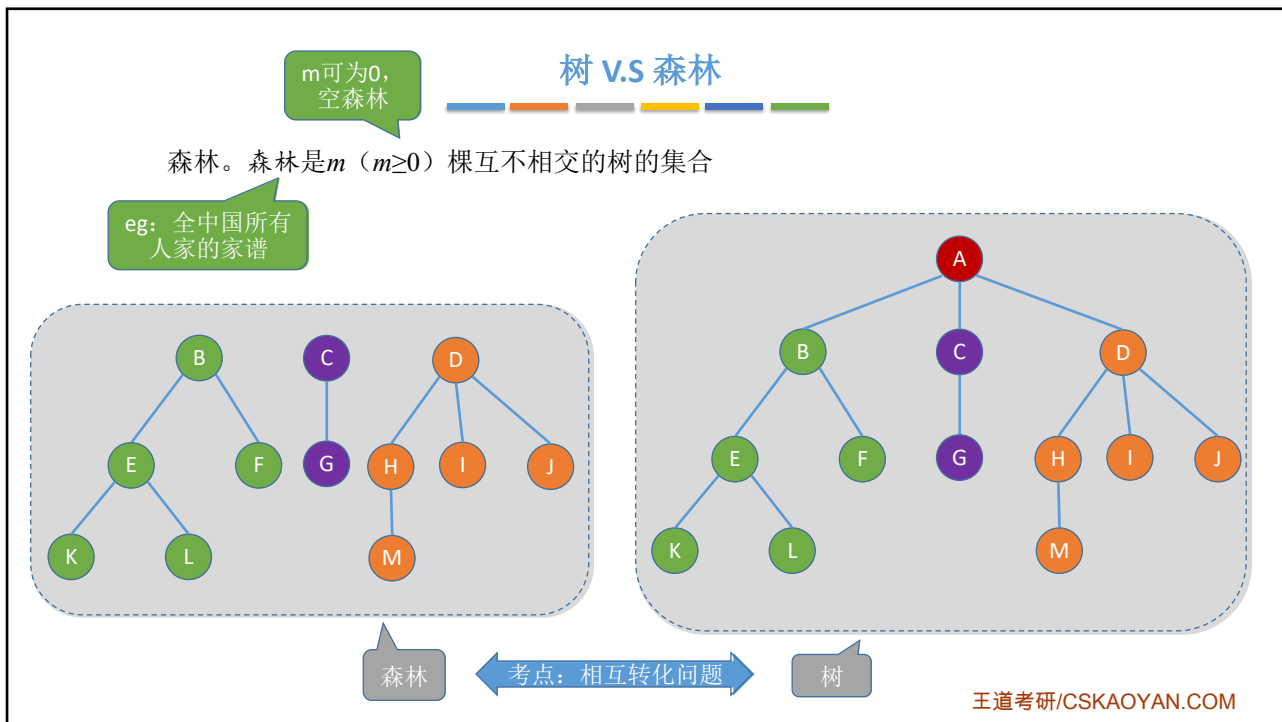
有序树——逻辑上看, 树中结点的各子树从左至右是**有次序的**, 不能互换
无序树——逻辑上看, 树中结点的各子树从左至右是**无次序的**, 可以互换

具体看你用树存什么, 是否需要用结点的左右位置反映某些逻辑关系

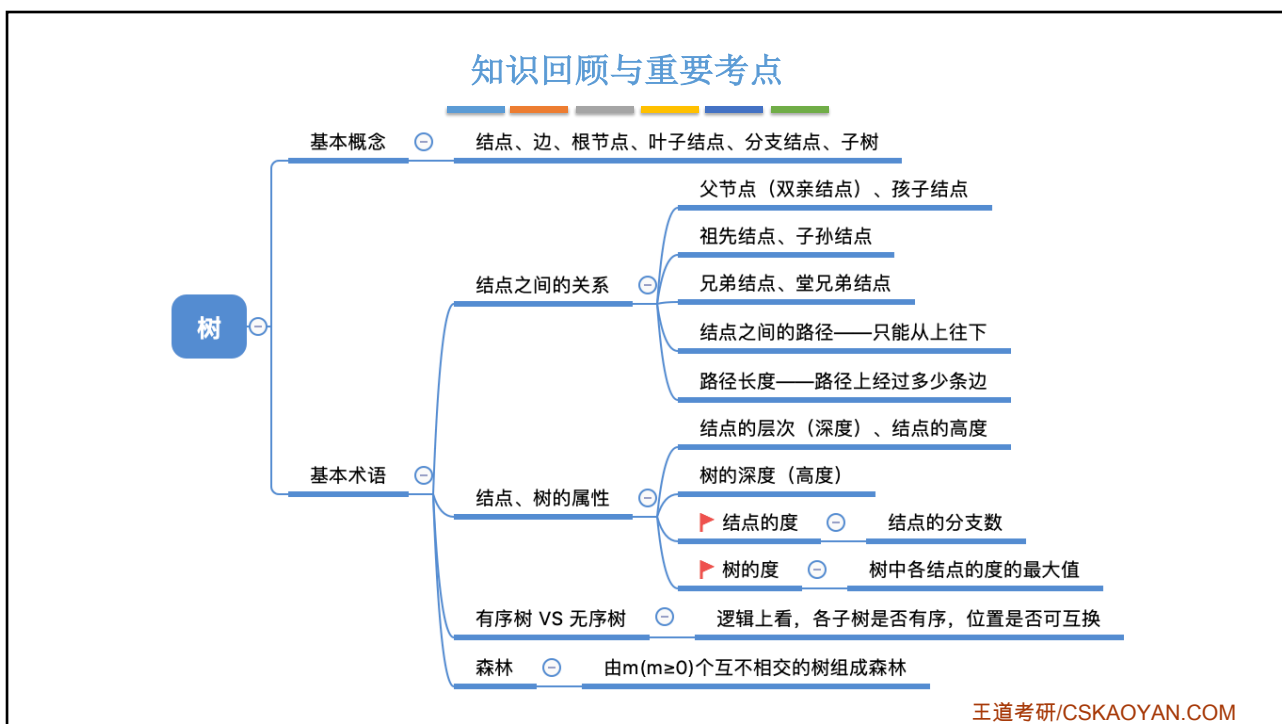


王道考研/CSKAOYAN.COM

10



11



12

本节内容

树

常考性质

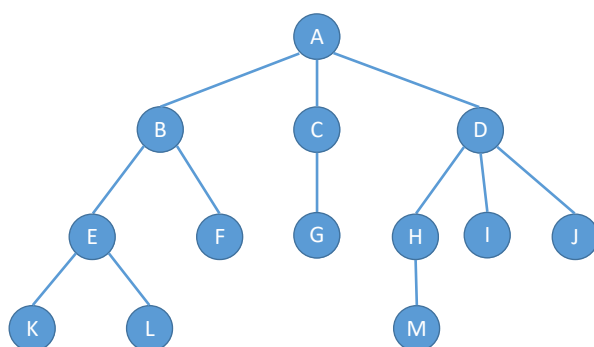
王道考研/CSKAOYAN.COM

1

树的常考性质

常见考点1: 结点数=总度数+1

结点的度——结点有几个孩子(分支)



王道考研/CSKAOYAN.COM

2

树的常考性质

树的度——各结点的度的最大值

m叉树——每个结点最多只能有m个孩子的树

度为m的树	m叉树
任意结点的度 $\leq m$ (最多m个孩子)	任意结点的度 $\leq m$ (最多m个孩子)
至少有一个结点度 = m (有m个孩子)	允许所有结点的度都 $< m$
一定是非空树，至少有m+1个结点	可以是空树

度为3的树

3叉树

3叉空树

常见考点2：度为m的树、m叉树 的区别

王道考研/CSKAOYAN.COM

3

树的常考性质

常见考点3：度为m的树第i层至多有 m^{i-1} 个结点 ($i \geq 1$)

m叉树第i层至多有 m^{i-1} 个结点 ($i \geq 1$)

第1层: m^0
第2层: m^1
第3层: m^2
第4层: m^3

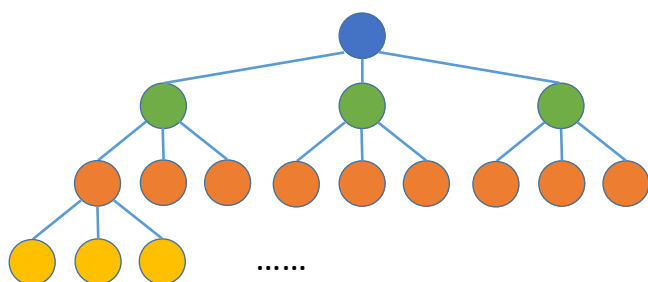
王道考研/CSKAOYAN.COM

4

树的常考性质

常见考点4: 高度为 h 的 m 叉树至多有 $\frac{m^h-1}{m-1}$ 个结点。

等比数列求和公式: $a + aq + aq^2 + \dots + aq^{n-1} = \frac{a(1-q^n)}{1-q}$



至少有多少个?



第1层: m^0

第2层: m^1

第3层: m^2

第4层: m^3

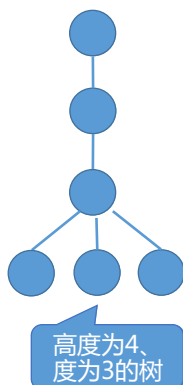
王道考研/CSKAOYAN.COM

5

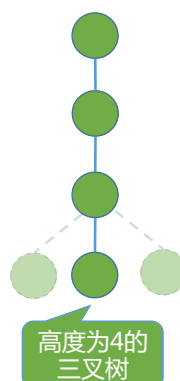
树的常考性质

常见考点5: 高度为 h 的 m 叉树至少有 h 个结点。

高度为 h 、度为 m 的树至少有 $h+m-1$ 个结点。



不会犯错



王道考研/CSKAOYAN.COM

6

树的常考性质

常见考点6: 具有n个结点的m叉树的最小高度为 $\lceil \log_m(n(m-1)+1) \rceil$

高度最小的情况——所有结点都有m个孩子

前h-1层最多有几个结点

$$\frac{m^{h-1}-1}{m-1} < n \leq \frac{m^h-1}{m-1}$$

前h层最多有几个结点

$$m^{h-1} < n(m-1) + 1 \leq m^h$$

$$h-1 < \log_m(n(m-1)+1) \leq h$$

$$h_{\min} = \lceil \log_m(n(m-1)+1) \rceil$$

王道考研/CSKAOYAN.COM

7

知识回顾与重要考点

树的常考性质

考点1 ⊖ 结点数=总度数+1

考点2 ⊖

度为m的树

至少有一个结点度 = m

一定是非空树

m叉树

允许所有结点的度都 < m

可以是空树

考点3 ⊖

度为m的树第i层至多有几个结点?

考点4 ⊖

高度为h的m叉树至多有几个结点?

考点5 ⊖

高度为h的m叉树至少有多少个结点?

高度为h、度为m的树至少有多少个结点?

考点6 ⊖

具有n个结点的m叉树的最小高度为?

王道考研/CSKAOYAN.COM

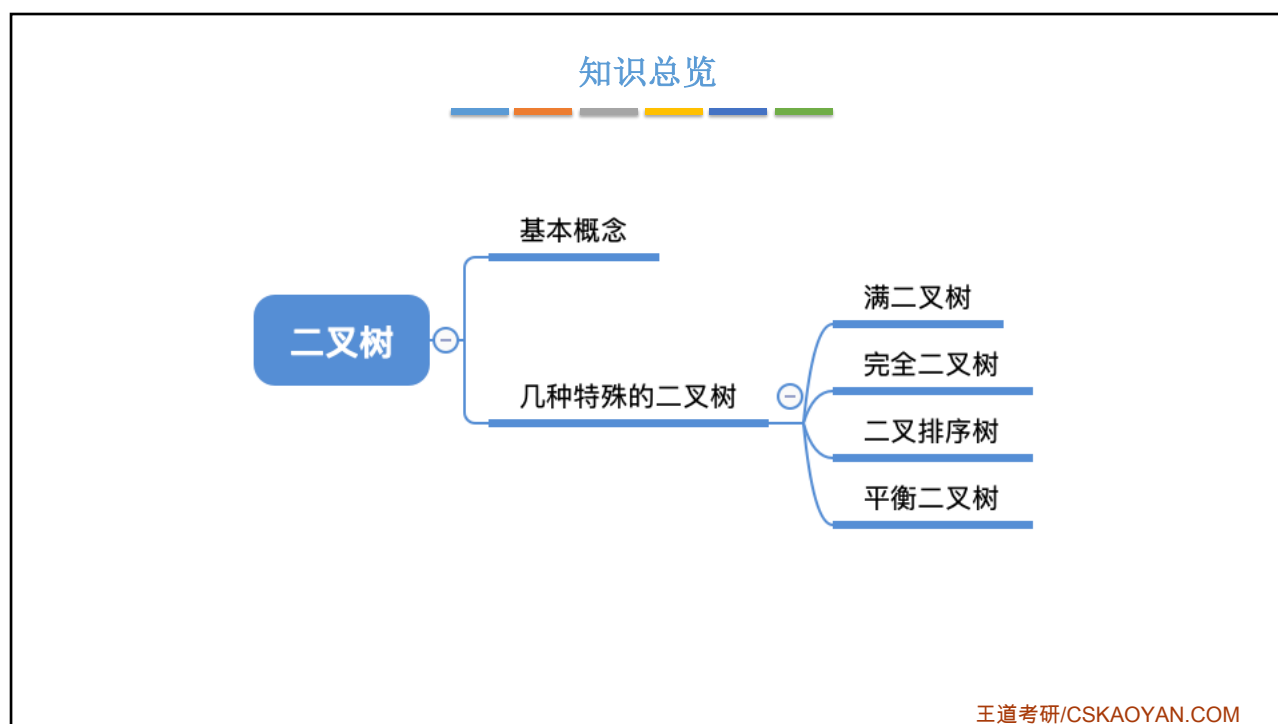
8

本节内容

二叉树
定义
基本术语

王道考研/CSKAOYAN.COM

1



2

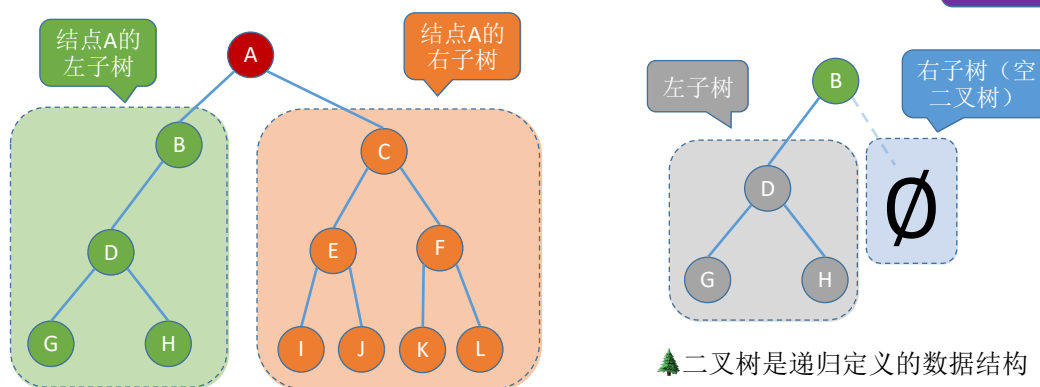
二叉树的基本概念

二叉树是 n ($n \geq 0$) 个结点的有限集合:

- ① 或者为**空二叉树**, 即 $n = 0$ 。
- ② 或者由一个**根结点**和两个互不相交的被称为根的**左子树**和**右子树**组成。左子树和右子树又分别是一棵二叉树。

特点: ① 每个结点至多只有两棵子树 ② 左右子树不能颠倒 (二叉树是**有序树**)

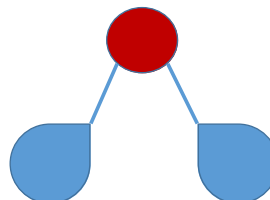
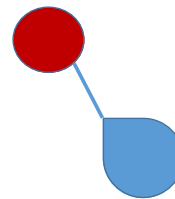
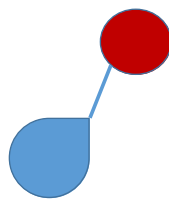
注意区别: 度为2的有序树



王道考研/CSKAOYAN.COM

3

二叉树的五种状态

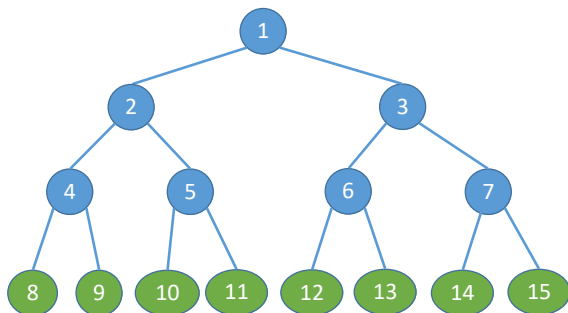


王道考研/CSKAOYAN.COM

4

几个特殊的二叉树

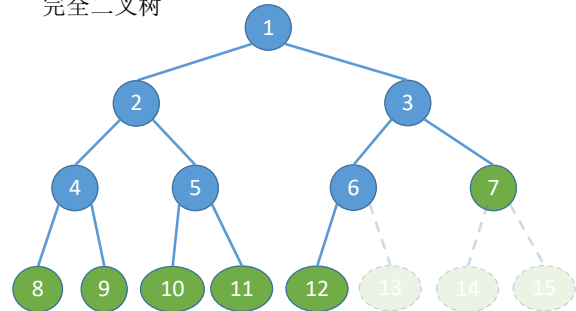
满二叉树。一棵高度为 h ，且含有 $2^h - 1$ 个结点的二叉树



特点:

- ① 只有最后一层有叶子结点
- ② 不存在度为1的结点
- ③ 按层序从1开始编号，结点 i 的左孩子为 $2i$ ，右孩子为 $2i+1$ ；结点 i 的父结点为 $\lfloor i/2 \rfloor$ （如果有的话）

完全二叉树。当且仅当其每个结点都与高度为 h 的满二叉树中编号为 $1 \sim n$ 的结点一一对应时，称为完全二叉树



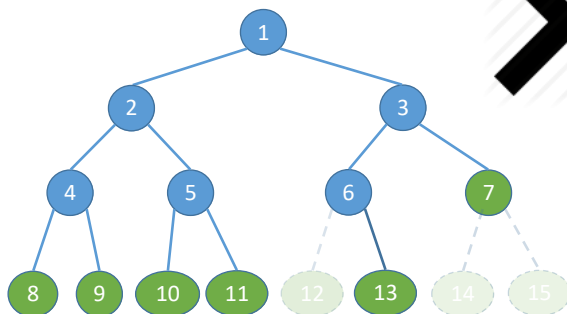
特点:

- ① 只有最后两层可能有叶子结点
- ② 最多只有一个度为1的结点
- ③ 同左③
- ④ $i \leq \lfloor n/2 \rfloor$ 为分支结点， $i > \lfloor n/2 \rfloor$ 为叶子结点

王道考研/CSKAOYAN.COM

5

几个特殊的二叉树



不是“完全二叉树”

如果某结点只有一个孩子，那么一定是左孩子

王道考研/CSKAOYAN.COM

6

几个特殊的二叉树

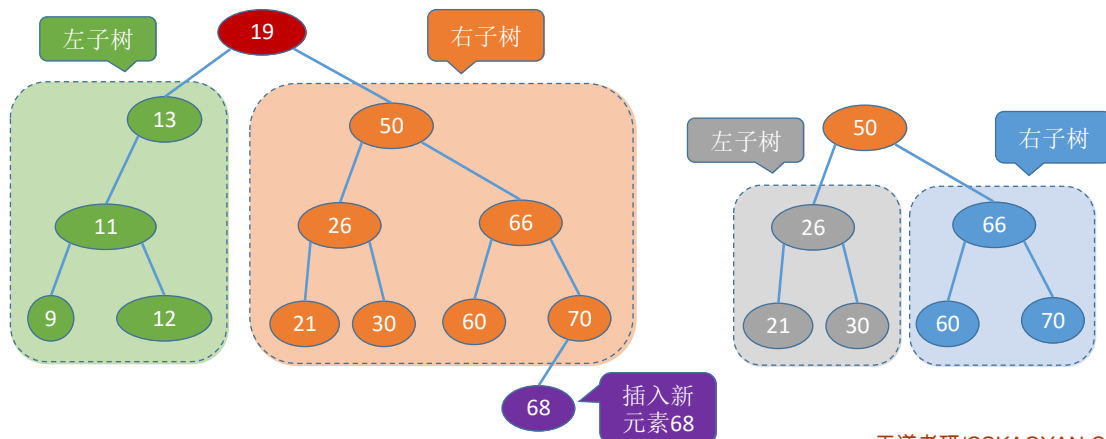
二叉排序树。一棵二叉树或者是空二叉树，或者是具有如下性质的二叉树：

左子树上所有结点的关键字均小于根结点的关键字；

右子树上所有结点的关键字均大于根结点的关键字。

左子树和右子树又各是一棵二叉排序树。

二叉排序树可用于元素的排序、搜索



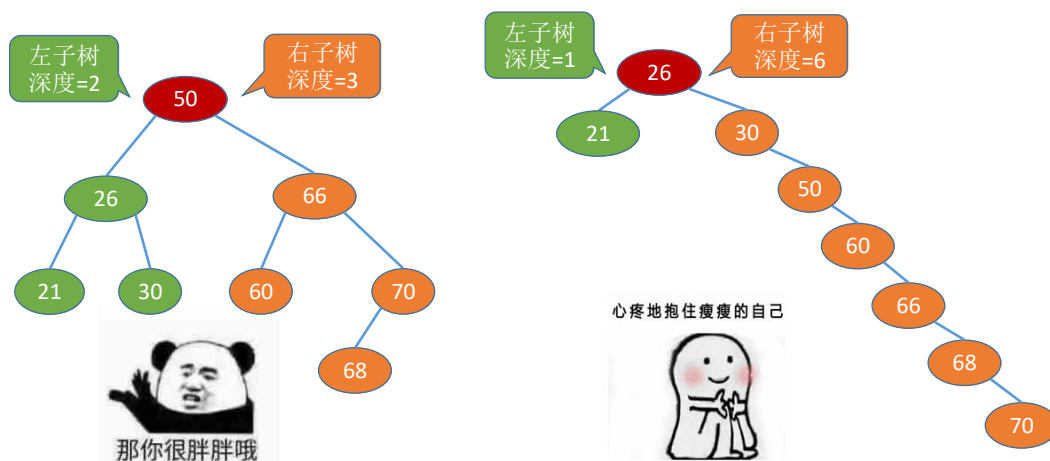
7

几个特殊的二叉树

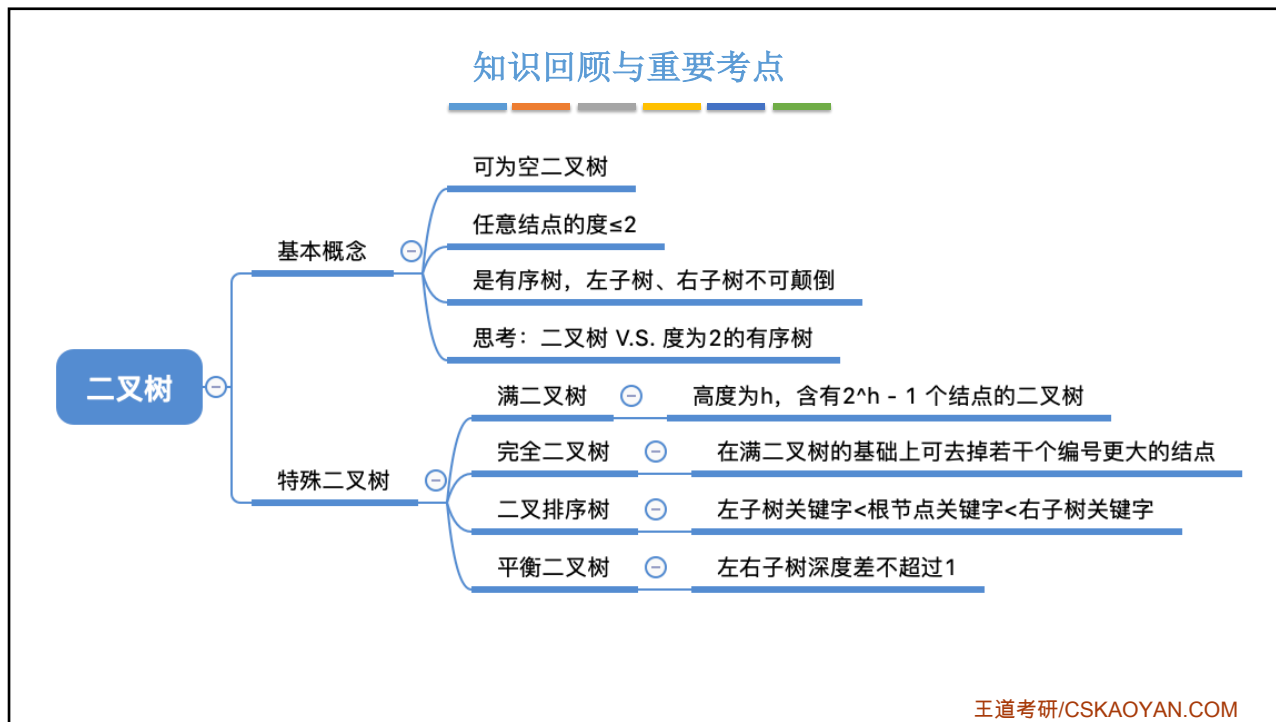
胖胖的、丰满的树

平衡二叉树。树上任一结点的左子树和右子树的深度之差不超过1。

平衡二叉树能有更高的搜索效率



8



本节内容

二叉树

常考性质

王道考研/CSKAOYAN.COM

1

二叉树的常考性质

常见考点1: 设非空二叉树中度为0、1和2的结点个数分别为 n_0 、 n_1 和 n_2 , 则 **$n_0 = n_2 + 1$**
(叶子结点比二分支结点多一个)

假设树中结点总数为 n , 则

- ① $n = n_0 + n_1 + n_2$
- ② $n = n_1 + 2n_2 + 1$

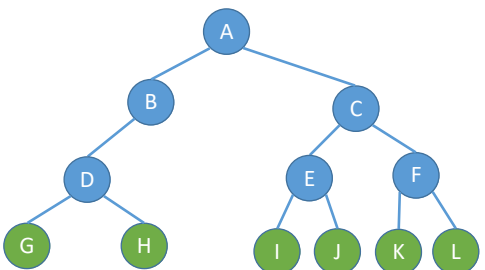
② - ①

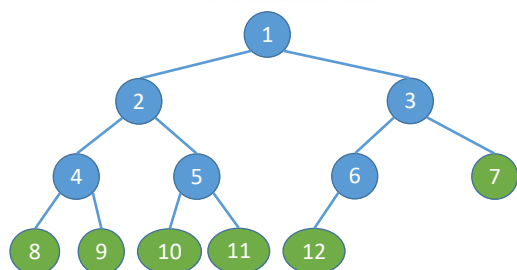
$n_0 = n_2 + 1$



上面可都是重点

树的结点数=总度数+1





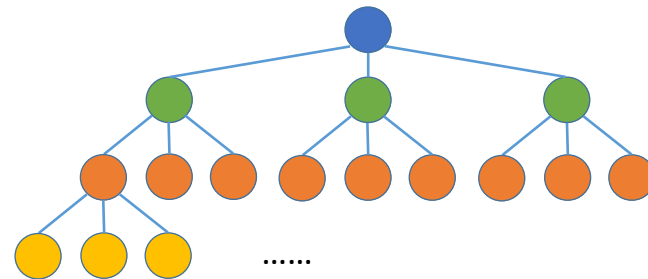
王道考研/CSKAOYAN.COM

2

二叉树的常考性质

常见考点2: 二叉树第 i 层至多有 2^{i-1} 个结点 ($i \geq 1$)

m 叉树第 i 层至多有 m^{i-1} 个结点 ($i \geq 1$)



第 1 层: m^0

第 2 层: m^1

第 3 层: m^2

第 4 层: m^3

王道考研/CSKAOYAN.COM

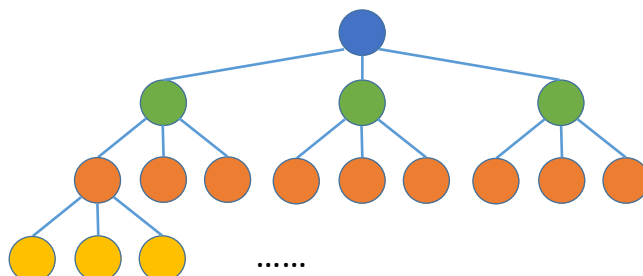
3

二叉树的常考性质

常见考点3: 高度为 h 的二叉树至多有 $2^h - 1$ 个结点 (满二叉树)

高度为 h 的 m 叉树至多有 $\frac{m^h - 1}{m - 1}$ 个结点

等比数列求和公式: $a + aq + aq^2 + \dots + aq^{n-1} = \frac{a(1 - q^n)}{1 - q}$



第 1 层: m^0

第 2 层: m^1

第 3 层: m^2

第 4 层: m^3

王道考研/CSKAOYAN.COM

4

完全二叉树的常考性质

常见考点1: 具有 n 个($n > 0$)结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

高为 h 的满二叉树共有 $2^h - 1$ 个结点
高为 $h-1$ 的满二叉树共有 $2^{h-1} - 1$ 个结点

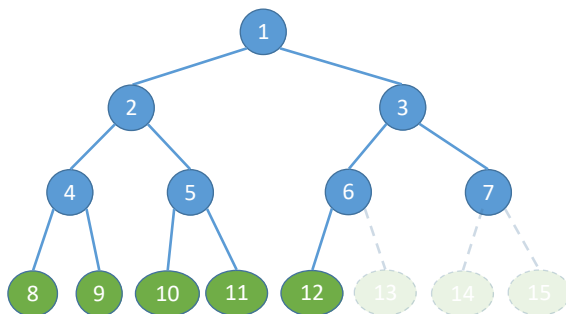


$$2^{h-1} - 1 < n \leq 2^h - 1$$

$$2^{h-1} < n + 1 \leq 2^h$$

$$h - 1 < \log_2(n + 1) \leq h$$

$$h = \lceil \log_2(n + 1) \rceil$$



王道考研/CSKAOYAN.COM

5

完全二叉树的常考性质

常见考点1: 具有 n 个($n > 0$)结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

高为 $h-1$ 的满二叉树共有 $2^{h-1} - 1$ 个结点
高为 h 的完全二叉树至少 2^{h-1} 个结点
至多 $2^h - 1$ 个结点



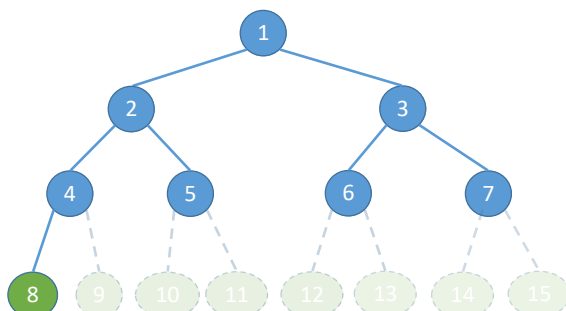
$$2^{h-1} \leq n < 2^h$$

$$h - 1 \leq \log_2 n < h$$

$$h = \lfloor \log_2 n \rfloor + 1$$



第 i 个结点所在层次为 $\lceil \log_2(n + 1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$



王道考研/CSKAOYAN.COM

6

完全二叉树的常考性质

常见考点2: 对于完全二叉树, 可以由的结点数 n 推出度为0、1和2的结点个数为 n_0 、 n_1 和 n_2

完全二叉树最多只有一个度为1的结点, 即

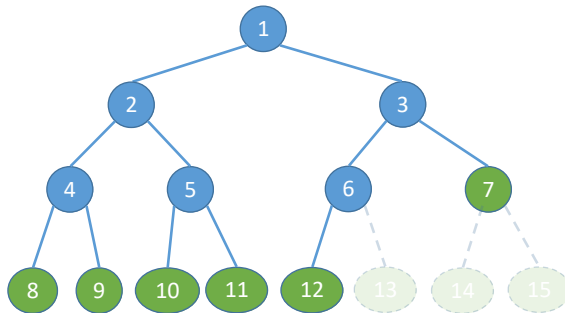
$n_1=0$ 或1

$n_0 = n_2 + 1 \rightarrow n_0 + n_2$ 一定是奇数



若完全二叉树有 $2k$ 个(偶数)个结点, 则必有 $n_1=1$, $n_0=k$, $n_2=k-1$

若完全二叉树有 $2k-1$ 个(奇数)个结点, 则必有 $n_1=0$, $n_0=k$, $n_2=k-1$



王道考研/CSKAOYAN.COM

7

知识回顾与重要考点

二叉树:

- $n_0 = n_2 + 1$
- 第 i 层至多有 2^{i-1} 个结点 ($i \geq 1$)
- 高度为 h 的二叉树至多有 $2^h - 1$ 个结点

完全二叉树:

- 具有 n 个 ($n > 0$) 结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$
- 对于完全二叉树, 可以由的结点数 n 推出为0、1和2的结点个数为 n_0 、 n_1 和 n_2 (突破点: 完全二叉树最多只会一个度为1的结点)

王道考研/CSKAOYAN.COM

8

2020/3/7

本节内容

二叉树
存储结构

王道考研/CSKAOYAN.COM

2

知识总览

二叉树的存储结构

顺序存储

链式存储

王道考研/CSKAOYAN.COM

3

二叉树的顺序存储

```

#define MaxSize 100
struct TreeNode {
    ElemType value; // 结点中的数据元素
    bool isEmpty;   // 结点是否为空
};

TreeNode t[MaxSize];

// 定义一个长度为 MaxSize 的数组 t，按照
// 从上至下、从左至右的顺序依次存储完
// 全二叉树中的各个结点

for (int i=0; i<MaxSize; i++){
    t[i].isEmpty=true;
}
        
```

初始化时所有结点标记为空

可以让第一个位置空缺，保证数组下标和结点编号一致

王道考研/CSKAOYAN.COM

4

二叉树的顺序存储

几个重要常考的基本操作:

- i 的左孩子 $\rightarrow 2i$
- i 的右孩子 $\rightarrow 2i+1$
- i 的父节点 $\rightarrow \lfloor i/2 \rfloor$
- i 所在的层次 $\rightarrow \lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

若完全二叉树中共有 n 个结点，则

- 判断 i 是否有左孩子? $\rightarrow 2i \leq n$?
- 判断 i 是否有右孩子? $\rightarrow 2i+1 \leq n$?
- 判断 i 是否是叶子/分支结点? $\rightarrow i > \lfloor n/2 \rfloor$?

初始化时所有结点标记为空

可以让第一个位置空缺，保证数组下标和结点编号一致

王道考研/CSKAOYAN.COM

5

二叉树的顺序存储

^ 1 2 3 4 5 6 7 8 ^ ^ ^ ^ ^ ^ ^ ^

t[0] t[1] t[2]

如果不是完全二叉树, 依然按层序将各节点顺序存储, 那么...

无法从结点编号反映出结点间的逻辑关系

• i 的左孩子 $2i$

• i 的右孩子 $2i+1$

• i 的父节点 $\lfloor i/2 \rfloor$

X

王道考研/CSKAOYAN.COM

6

二叉树的顺序存储

^ 1 2 3 ^ 5 6 7 ^ ^ ^ 11 12 ^ ^ ^ ^

t[0] t[1] t[2]

★ 二叉树的顺序存储中, 一定要把二叉树的结点编号与完全二叉树对应起来

• i 的左孩子 $2i$

• i 的右孩子 $2i+1$

• i 的父节点 $\lfloor i/2 \rfloor$

若非完全二叉树中共有n个结点, 则

• 判断i是否有左孩子? $2i \leq n?$

• 判断i是否有右孩子? $2i+1 \leq n?$

• 判断i是否是叶子/分支结点? $i > \lfloor n/2 \rfloor?$

X

王道考研/CSKAOYAN.COM

7

二叉树的顺序存储

★ 二叉树的顺序存储中, 一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$

最坏情况: 高度为 h 且只有 h 个结点的单支树(所有结点只有右孩子), 也至少需要 2^h-1 个存储单元

结论: 二叉树的顺序存储结构, 只适合存储完全二叉树

^	1	^	3	^	^	^	7	^	^	^	^	^	^	15	^
---	---	---	---	---	---	---	---	---	---	---	---	---	---	----	---

t[0] t[1] t[2]

王道考研/CSKAOYAN.COM

8

二叉树的链式存储

```
//二叉树的结点(链式存储)
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

//数据域
//左、右孩子指针

★ n 个结点的二叉链表共有 $n+1$ 个空链域

可以用于构造线索二叉树

王道考研/CSKAOYAN.COM

9

二叉树的链式存储

```

struct ElemType{
    int value;
};

typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;

//定义一棵空树
BiTree root = NULL;

//插入根节点
root = (BiTree) malloc(sizeof(BiTNode));
root->data = {1};
root->lchild = NULL;
root->rchild = NULL;

//插入新结点
BiTNode * p = (BiTNode *) malloc(sizeof(BiTNode));
p->data = {2};
p->lchild = NULL;
p->rchild = NULL;
root->lchild = p;    //作为根节点的左孩子
        
```

王道考研/CSKAOYAN.COM

10

二叉树的链式存储

找到指定结点 p 的左/右孩子——超简单

如何找到指定结点 p 的父结点?

只能从根开始遍历寻找

Tips: 根据实际需求决定要不要加父结点指针

```

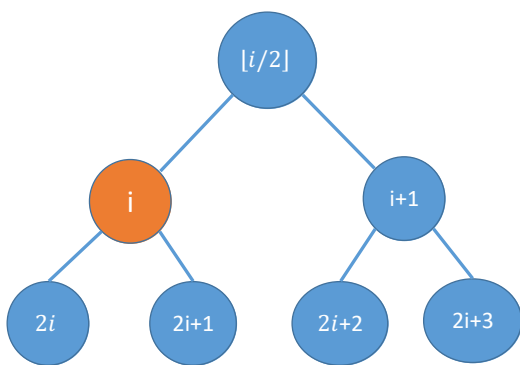
//二叉树的结点(链式存储)
typedef struct BiTNode{
    ElemType data;                //数据域
    struct BiTNode *lchild,*rchild; //左、右孩子指针
    struct BiTNode *parent;        //父结点指针
}BiTNode,*BiTree;
        
```

三叉链表——方便找父结点

王道考研/CSKAOYAN.COM

11

知识回顾与重要考点



思考: 如果结点从0开始编号, 该怎么算

★ 二叉树的顺序存储中, 一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $[i/2]$

最坏情况: 高度为 h 且只有 h 个结点的单支树(所有结点只有右孩子), 也至少需要 2^h-1 个存储单元

结论: 二叉树的顺序存储结构, 只适合存储完全二叉树

王道考研/CSKAOYAN.COM

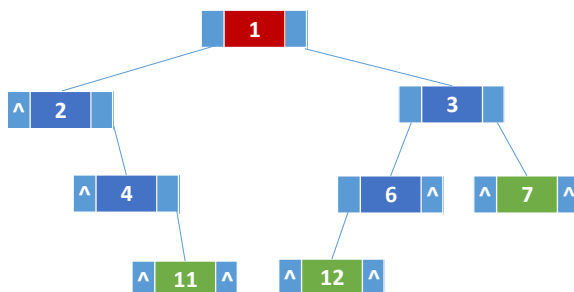
12

知识回顾与重要考点

//二叉树的结点(链式存储)

```
typedef struct BiTNode{
    ElemType data;           //数据域
    struct BiTNode *lchild, *rchild; //左、右孩子指针
}BiTNode, *BiTree;
```

★ n 个结点的二叉链表共有 $n+1$ 个空链域



王道考研/CSKAOYAN.COM

13

本节内容

二叉树
先/中/后序
遍历

王道考研/CSKAOYAN.COM

1

知识总览

二叉树的遍历

先序遍历

中序遍历

后序遍历

遍历算法的应用举例

王道考研/CSKAOYAN.COM

2

什么是遍历

遍历: 按照某种次序把所有结点都访问一遍



线性结构

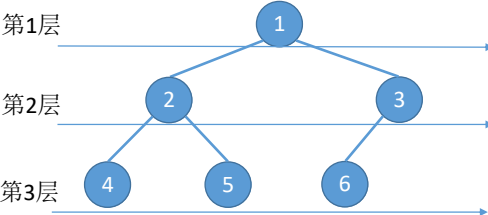


简单来说

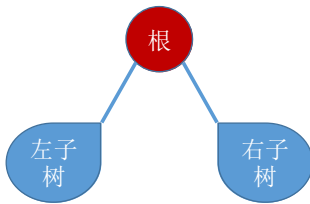
第1层

第2层

第3层



层次遍历: 基于树的层次特性确定的次序规则



先/中/后序遍历: 基于树的递归特性确定的次序规则

王道考研/CSKAOYAN.COM

3

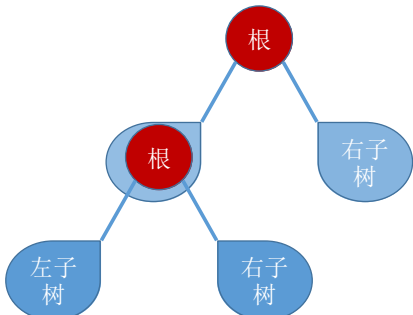
二叉树的遍历

二叉树的递归特性:

- ① 要么是个空二叉树
- ② 要么就是由“根节点+左子树+右子树”组成的二叉树



空二叉树



非空二叉树

先序遍历: 根左右 (NLR)

中序遍历: 左根右 (LNR)

后序遍历: 左右根 (LRN)

王道考研/CSKAOYAN.COM

4

二叉树的遍历

先序遍历: **A**BC
中序遍历: B**A**C
后序遍历: B**C**A

先序遍历: **A**B
中序遍历: B**A**
后序遍历: B**A**

右子树为空树

先序遍历: **A**C
中序遍历: **A**C
后序遍历: **C**A

左子树为空树

先序遍历: **A** B D E C F G
中序遍历: D B E **A** F C G
后序遍历: D E B F G C **A**

先序遍历: 根左右 (NLR)
中序遍历: 左根右 (LNR)
后序遍历: 左右根 (LRN)

王道考研/CSKAOYAN.COM

5

二叉树的遍历 (手算练习)

先序遍历: 根 左 右
根 (根 左 右) (根 左)
根 (根 (根 右) 右) (根 左)

中序遍历: (左 根 右) 根 (左 根)
((根右) 根 右) 根 (左 根)

后序遍历: (左 右 根) (左 根) 根
((右 根) 右 根) (左 根) 根

分支结点逐层展开法...

先序遍历: A B C
A B D E C F
A B D G E C F

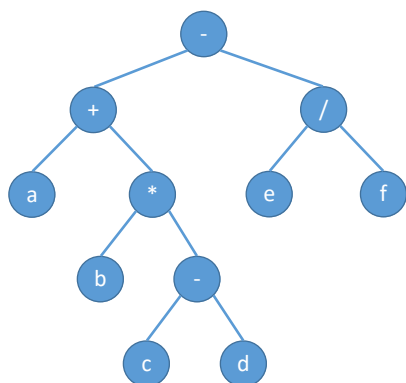
中序遍历: D B E A F C
D G B E A F C

后序遍历: D E B F C A
G D E B F C A

王道考研/CSKAOYAN.COM

6

二叉树的遍历(手算练习)



先序遍历: $-+a*b-cd/ef$

中序遍历: $a+b*c-d-e/f$

后序遍历: $abcd-*+ef/-$

先序遍历 $\rightarrow$ 前缀表达式

中序遍历 $\rightarrow$ 中缀表达式(需要加界限符)

后序遍历 $\rightarrow$ 后缀表达式

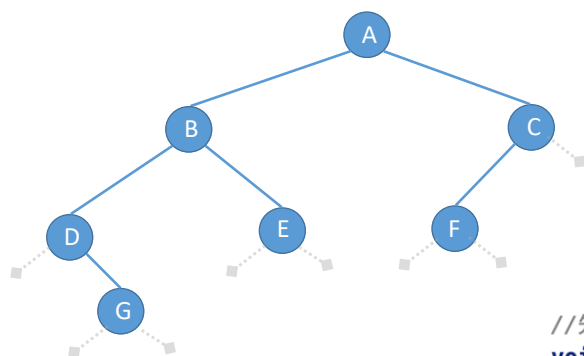
算数表达式的“分析树”

$a + b * (c - d) - e / f$

王道考研/CSKAOYAN.COM

7

先序遍历(代码)



先序遍历(PreOrder)的操作过程如下:

1. 若二叉树为空, 则什么也不做;

2. 若二叉树非空:

①访问根结点;

②先序遍历左子树;

③先序遍历右子树。

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

//先序遍历

```
void PreOrder(BiTree T){
```

```
    if(T!=NULL){
```

```
        visit(T);
```

//访问根结点

```
        PreOrder(T->lchild);
```

//递归遍历左子树

```
        PreOrder(T->rchild);
```

//递归遍历右子树

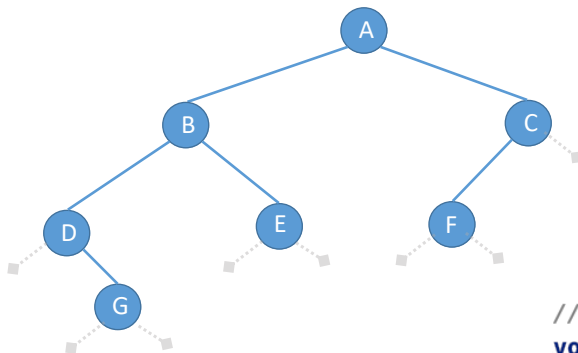
```
    }
```

```
}
```

王道考研/CSKAOYAN.COM

8

中序遍历(代码)



中序遍历(InOrder)的操作过程如下:

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①先序遍历左子树;
 - ②访问根结点;
 - ③先序遍历右子树。

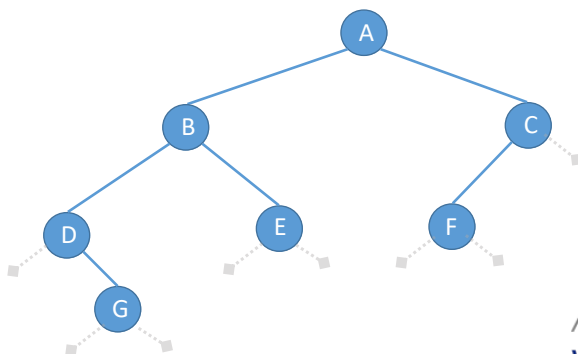
```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

```
//中序遍历
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild);    //递归遍历左子树
        visit(T);              //访问根结点
        InOrder(T->rchild);    //递归遍历右子树
    }
}
```

王道考研/CSKAOYAN.COM

9

后序遍历(代码)



后序遍历(InOrder)的操作过程如下:

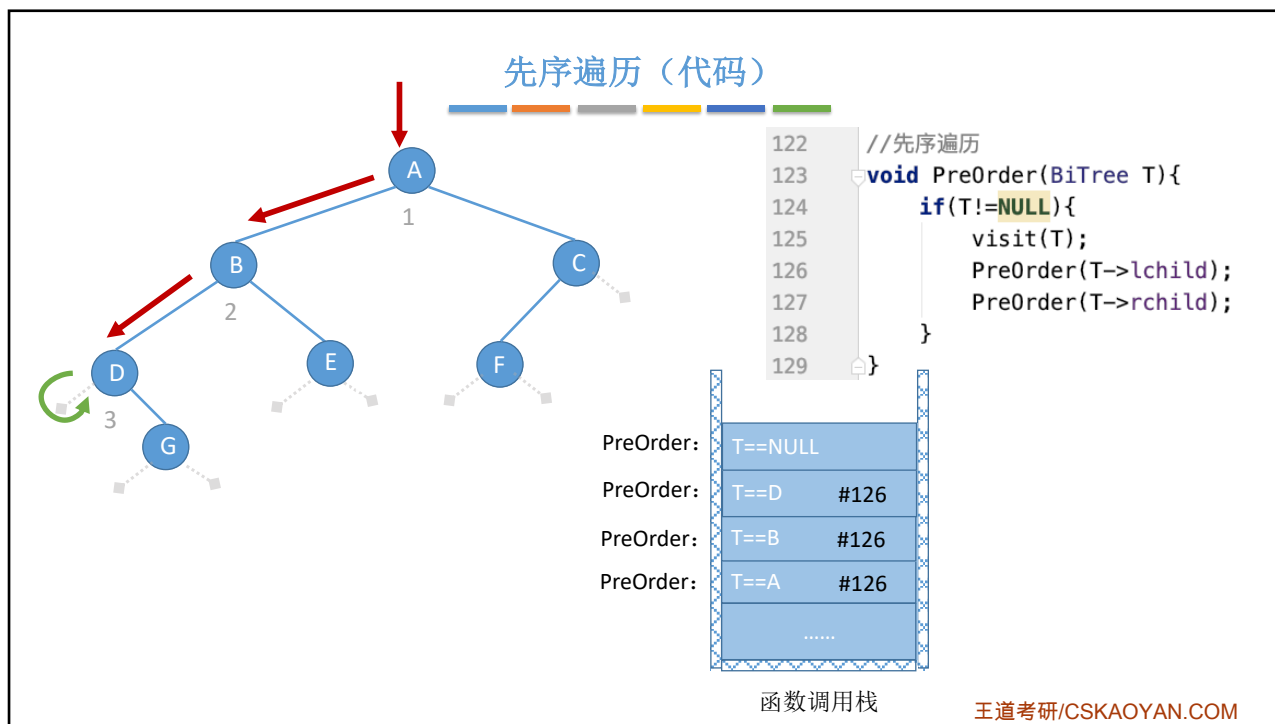
1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①先序遍历左子树;
 - ②先序遍历右子树;
 - ③访问根结点。

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

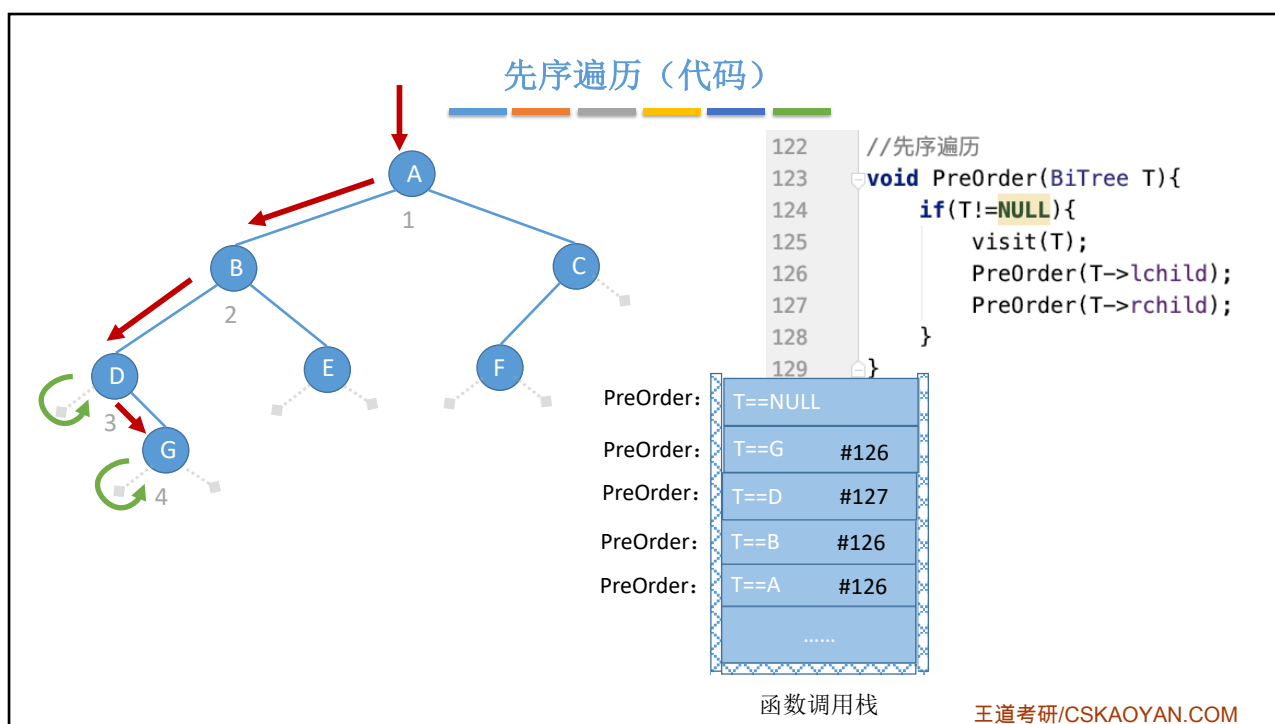
```
//后序遍历
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild);    //递归遍历左子树
        PostOrder(T->rchild);    //递归遍历右子树
        visit(T);              //访问根结点
    }
}
```

王道考研/CSKAOYAN.COM

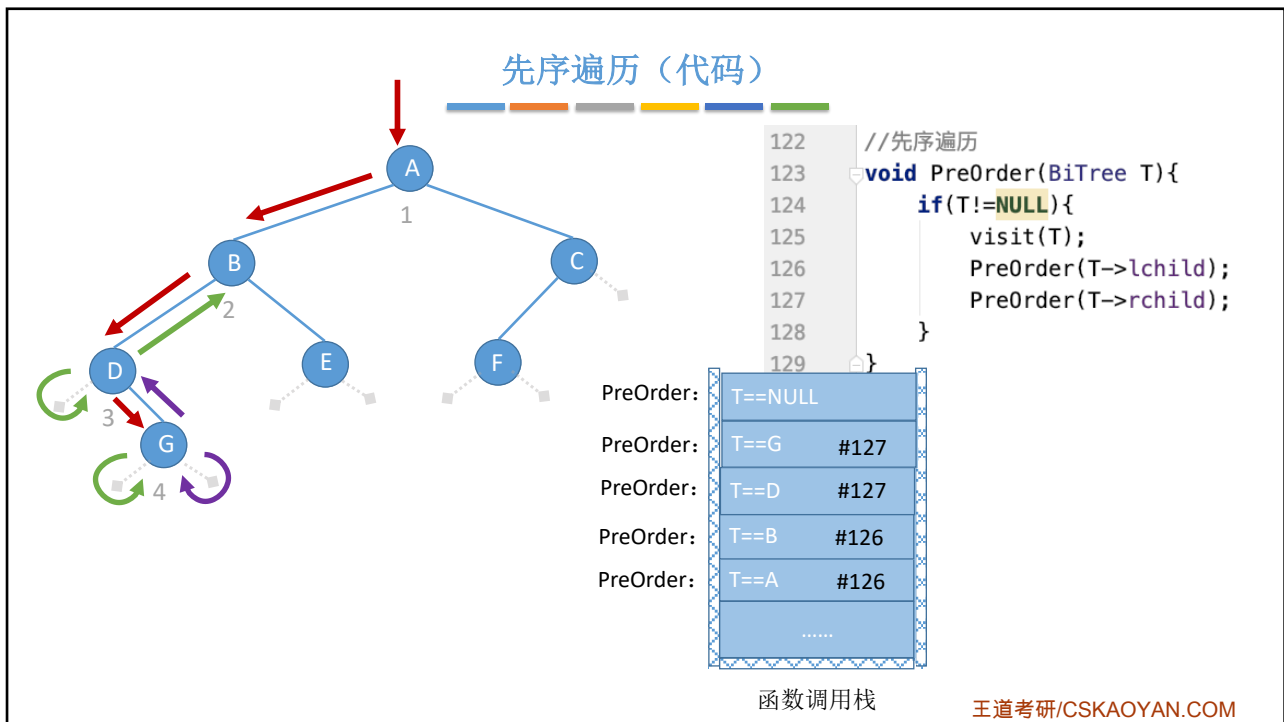
10



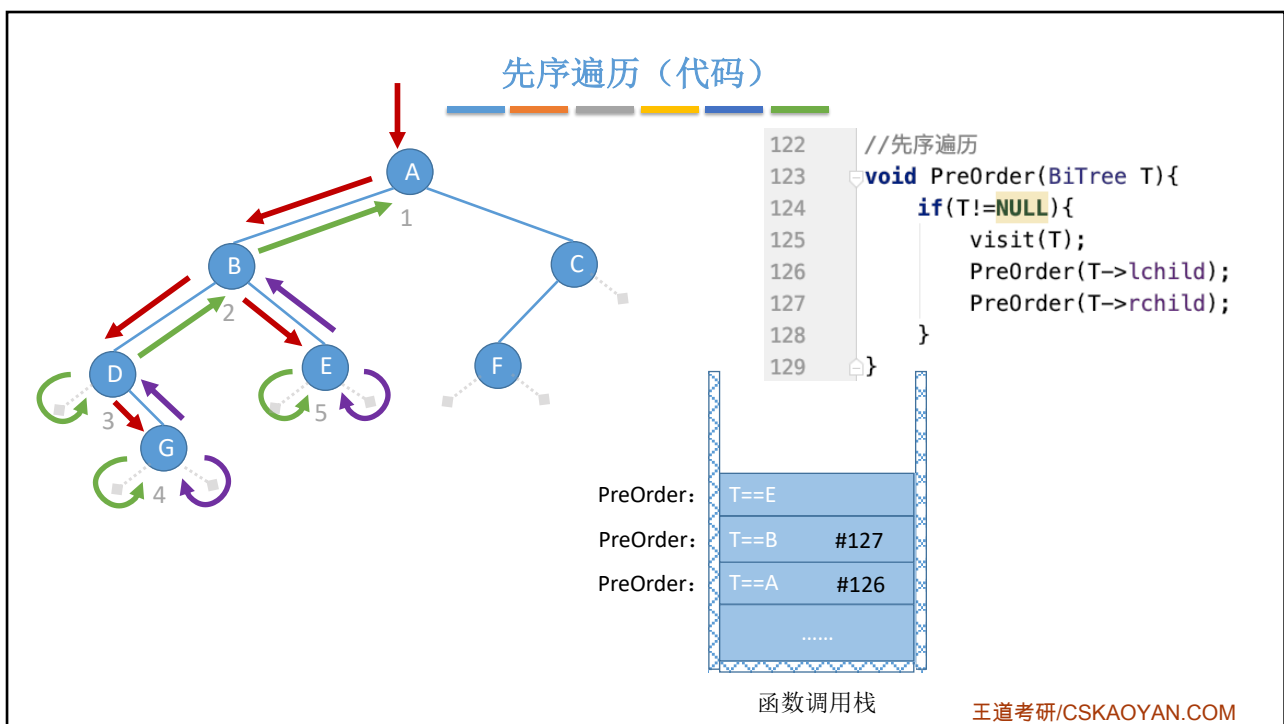
11



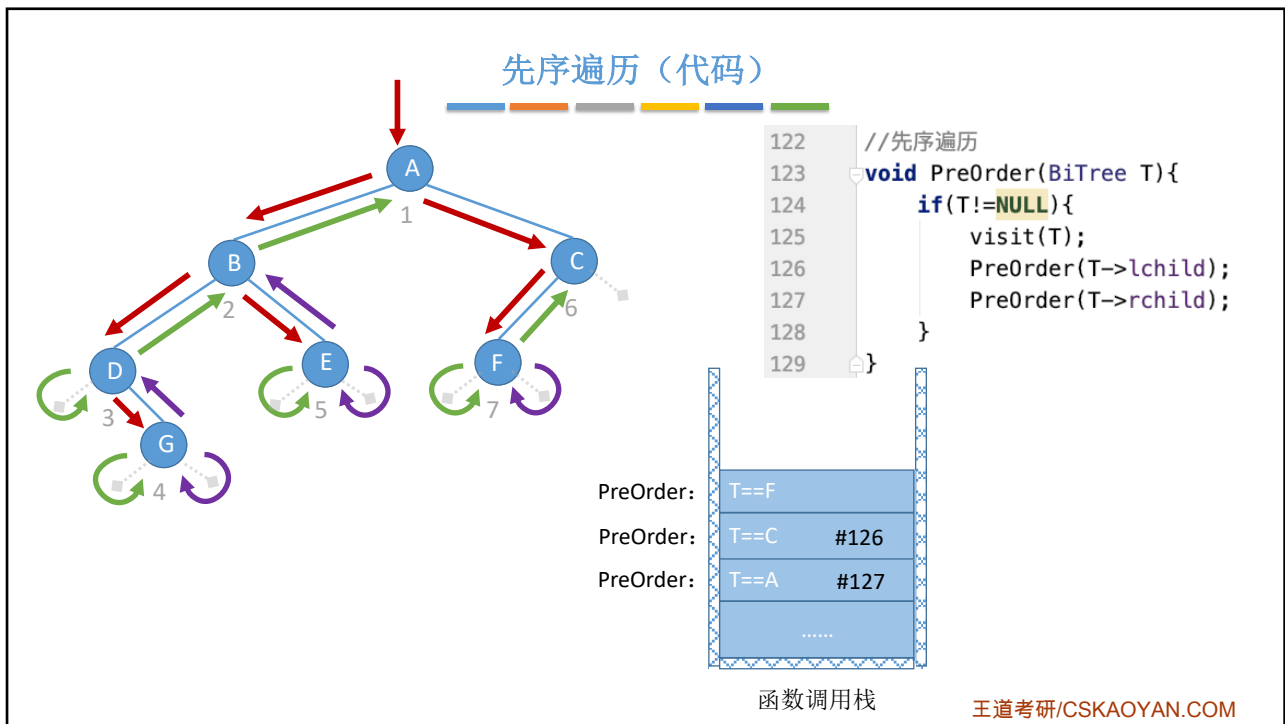
12



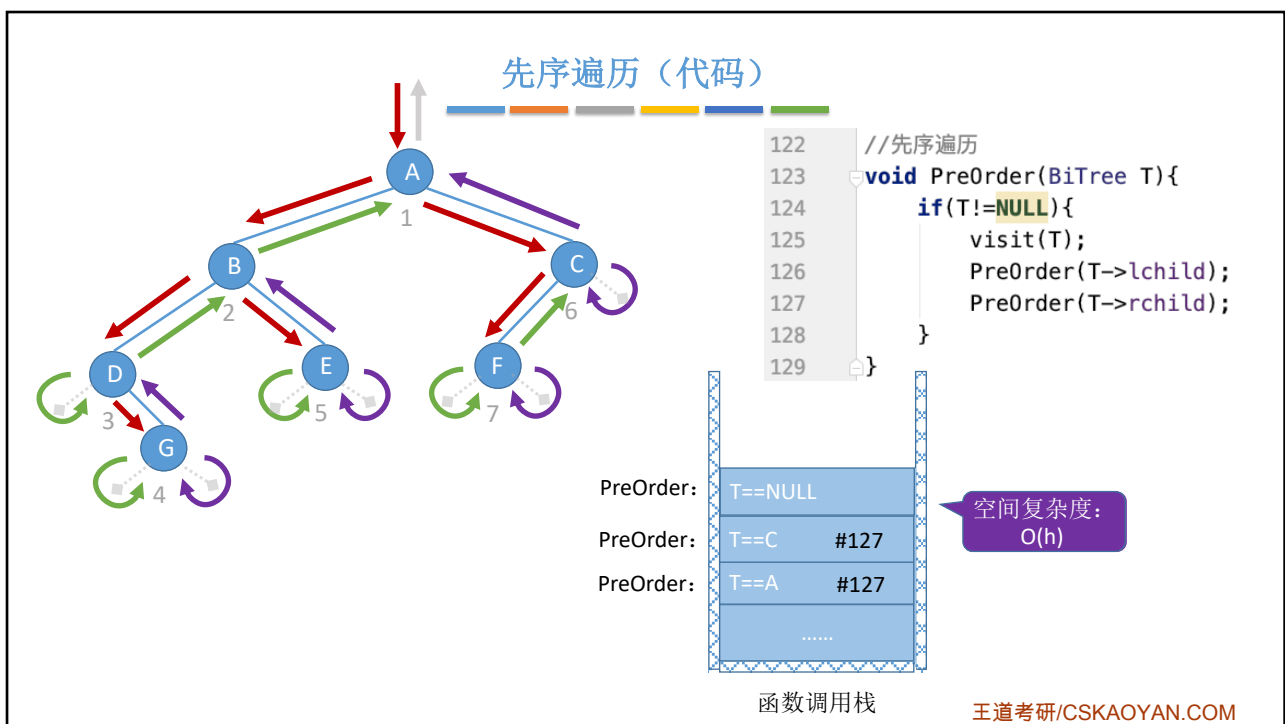
13



14



15



16

求先序遍历序列

先序遍历 (PreOrder) 的操作过程如下:

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①访问根结点;
 - ②先序遍历左子树;
 - ③先序遍历右子树。

脑补空结点, 从根节点出发, 画一条路:
 如果左边还有没走的路, 优先往左边走
 走到路的尽头(空结点)就往回走
 如果左边没路了, 就往右边走
 如果左、右都没路了, 则往上面走
先序遍历——第一次路过时访问结点

图示说明:
 第一次路过 →
 第二次路过 →
 第三次路过 →

每个结点都会被路过3次

王道考研/CSKAOYAN.COM

17

求中序遍历序列

中序遍历 (InOrder) 的操作过程如下:

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①先序遍历左子树;
 - ②访问根结点;
 - ③先序遍历右子树。

脑补空结点, 从根节点出发, 画一条路:
 如果左边还有没走的路, 优先往左边走
 走到路的尽头(空结点)就往回走
 如果左边没路了, 就往右边走
 如果左、右都没路了, 则往上面走
中序遍历——第二次路过时访问结点

图示说明:
 第一次路过 →
 第二次路过 →
 第三次路过 →

每个结点都会被路过3次

王道考研/CSKAOYAN.COM

18

从你的全世界路过法...

图示说明:
第一次路过 → (red)
第二次路过 → (green)
第三次路过 → (purple)

每个结点都会被路过3次

求后序遍历序列

后序遍历 (InOrder) 的操作过程如下:

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ① 先序遍历左子树;
 - ② 先序遍历右子树;
 - ③ 访问根结点。

脑补空结点, 从根节点出发, 画一条路:
如果左边还有没走的路, 优先往左边走
走到路的尽头(空结点)就往回走
如果左边没路了, 就往右边走
如果左、右都没路了, 则往上面走
后序遍历——第三次路过时访问结点

王道考研/CSKAOYAN.COM

19

二叉树的遍历 (手算练习)

算数表达式的“分析树”

$a + b * (c - d) - e / f$

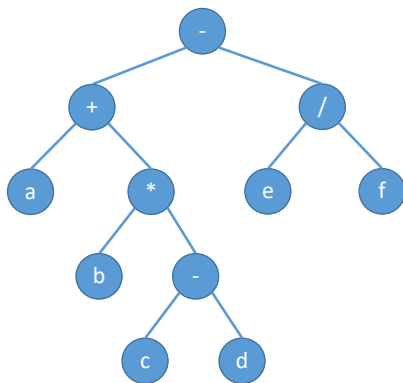
先序遍历: $- + a * b - c d / e f$
 中序遍历: $a + b * c - d - e / f$
 后序遍历: $abcd - * + ef / -$

先序遍历 → 前缀表达式
 中序遍历 → 中缀表达式 (需要加界限符)
 后序遍历 → 后缀表达式

王道考研/CSKAOYAN.COM

20

例: 求树的深度 (应用)



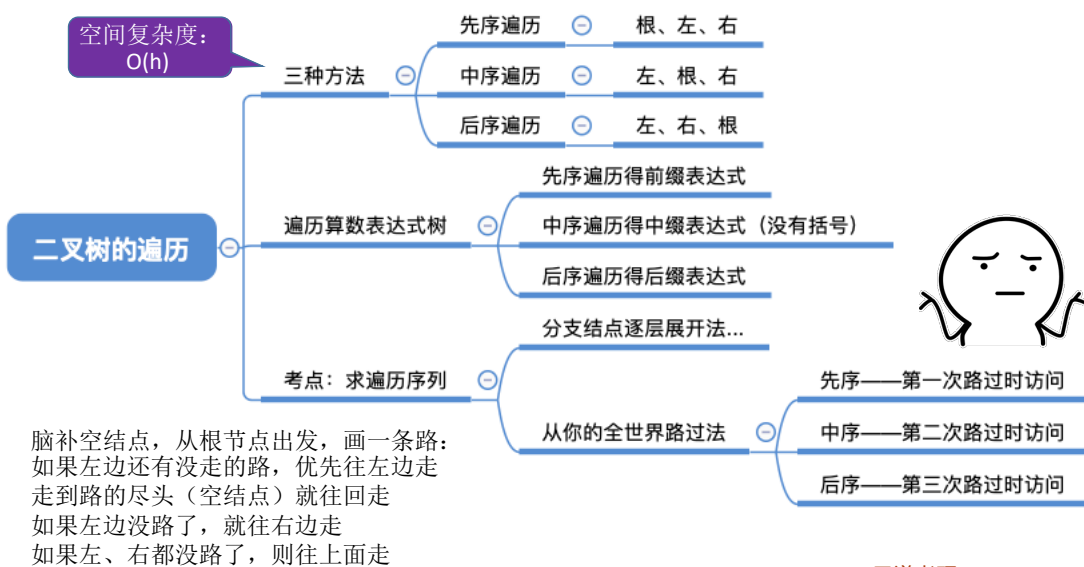
```

int treeDepth(BiTree T){
    if (T == NULL) {
        return 0;
    }
    else {
        int l = treeDepth(T->lchild);
        int r = treeDepth(T->rchild);
        //树的深度=Max(左子树深度, 右子树深度)+1
        return l>r ? l+1 : r+1;
    }
}
    
```

王道考研/CSKAOYAN.COM

21

知识回顾与重要考点



王道考研/CSKAOYAN.COM

22

本节内容

二叉树
层序遍历

王道考研/CSKAOYAN.COM

1

二叉树的层序遍历

第1层

第2层

第3层

第4层

出队

入队

算法思想：
①初始化一个辅助队列
②根结点入队
③若队列非空，则队头结点出队，访问该结点，并将其左、右孩子插入队尾（如果有的话）
④重复③直至队列为空

王道考研/CSKAOYAN.COM

2

代码实现

算法思想:

- ①初始化一个辅助队列
- ②根结点入队
- ③若队列非空, 则队头结点出队, 访问该结点, 并将其左、右孩子插入队尾(如果有的话)
- ④重复③直至队列为空

```
//层序遍历
void LevelOrder(BiTree T){
    LinkQueue Q;
    InitQueue(Q);           //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);           //将根结点入队
    while(!IsEmpty(Q)){    //队列非空则循环
        DeQueue(Q, p);      //队头结点出队
        visit(p);           //访问出队结点
        if(p->lchild!=NULL)  //左孩子入队
            EnQueue(Q,p->lchild);
        if(p->rchild!=NULL)  //右孩子入队
            EnQueue(Q,p->rchild);
    }
}
```

```
//二叉树的结点(链式存储)
typedef struct BiTNode{
    char data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;

//链式队列结点
typedef struct LinkNode{
    BiTNode *data;
    struct LinkNode *next;
}LinkNode;

typedef struct{
    LinkNode *front,*rear; //队头队尾
}LinkQueue;
```

存指针而不是结点

王道考研/CSKAOYAN.COM

3

知识回顾与重要考点

树的层次遍历算法思想:

- ①初始化一个辅助队列
- ②根结点入队
- ③若队列非空, 则队头结点出队, 访问该结点, 并将其左、右孩子插入队尾(如果有的话)
- ④重复③直至队列为空

王道考研/CSKAOYAN.COM

4

本节内容

由遍历序列构造二叉树

王道考研/CSKAOYAN.COM

1

不同二叉树的中序遍历序列

中序遍历：中序遍历左子树、根结点、中序遍历右子树

中序遍历序列：BDCAE

中序遍历序列：BDCAE

中序遍历序列：BDCAE

结论：一个中序遍历序列可能对应多种二叉树形态

王道考研/CSKAOYAN.COM

2

不同二叉树的前序遍历序列

前序遍历：根结点、前序遍历左子树、前序遍历右子树

前序遍历序列：BDCAE

前序遍历序列：BDCAE

前序遍历序列：BDCAE

结论：一个前序遍历序列可能对应多种二叉树形态

王道考研/CSKAOYAN.COM

3

不同二叉树的后序遍历序列

后序遍历：前序遍历左子树、前序遍历右子树、根结点

后序遍历序列：BDCAE

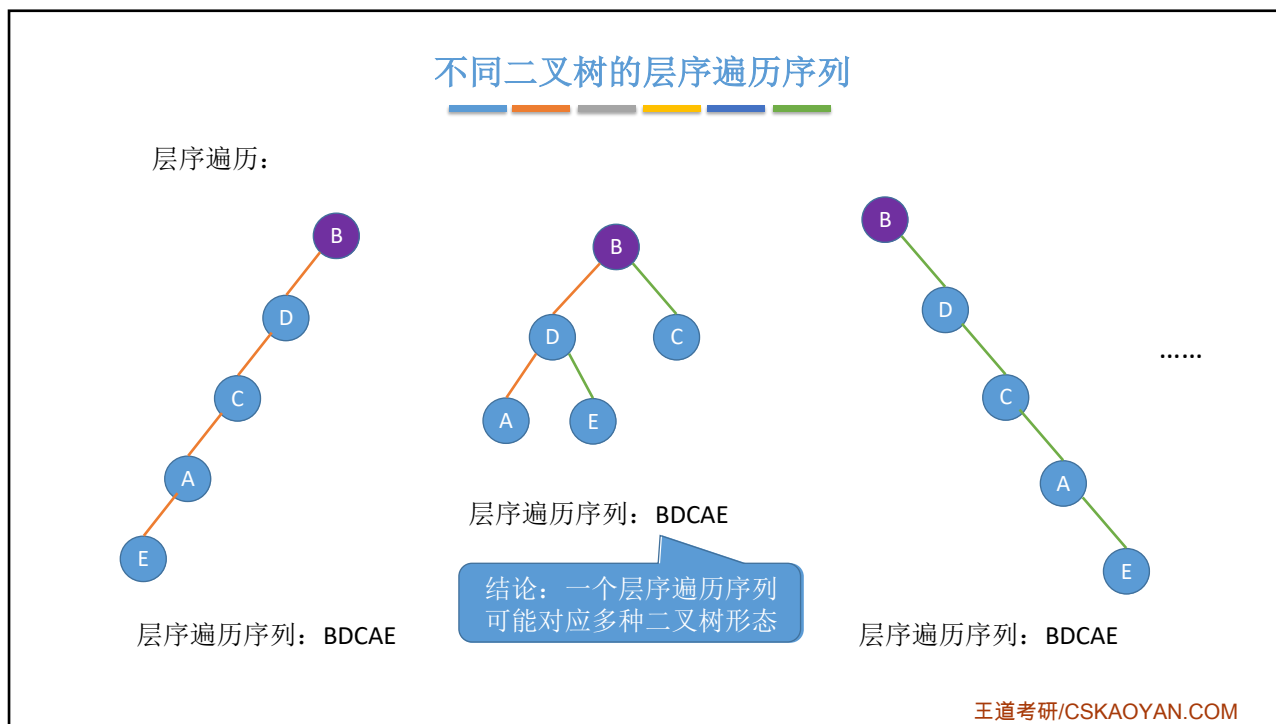
后序遍历序列：BDCAE

后序遍历序列：BDCAE

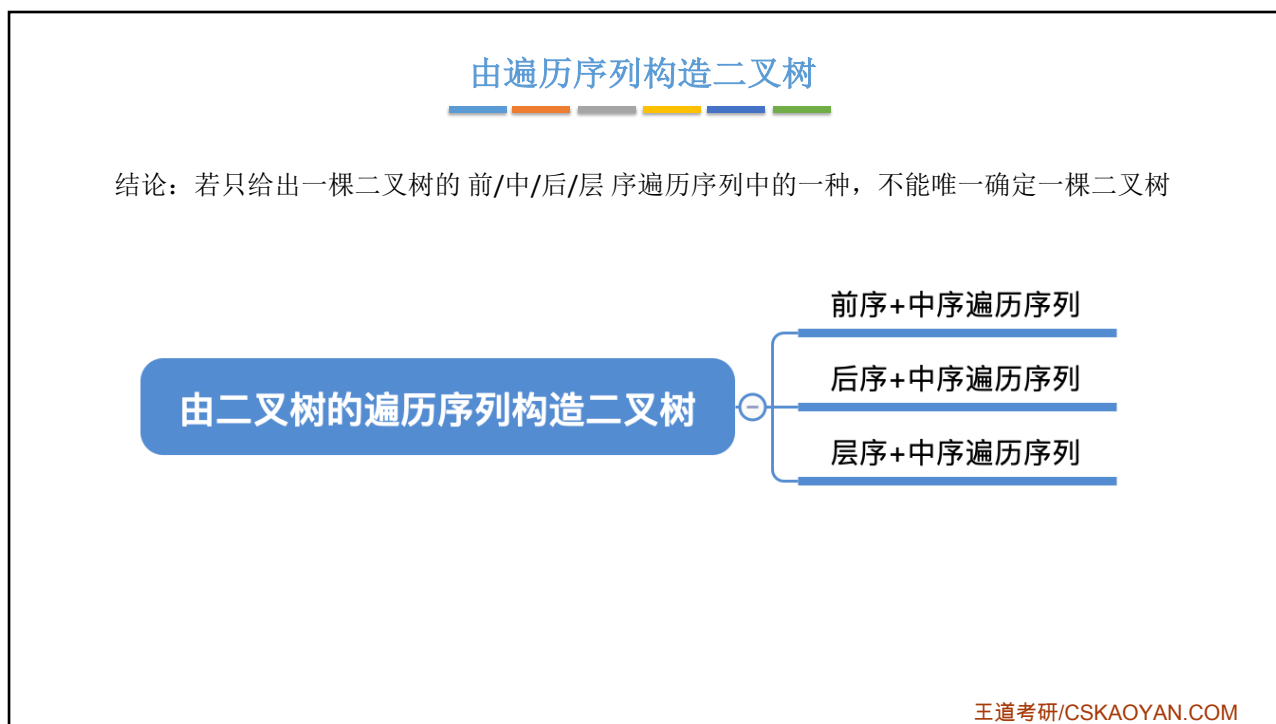
结论：一个后序遍历序列可能对应多种二叉树形态

王道考研/CSKAOYAN.COM

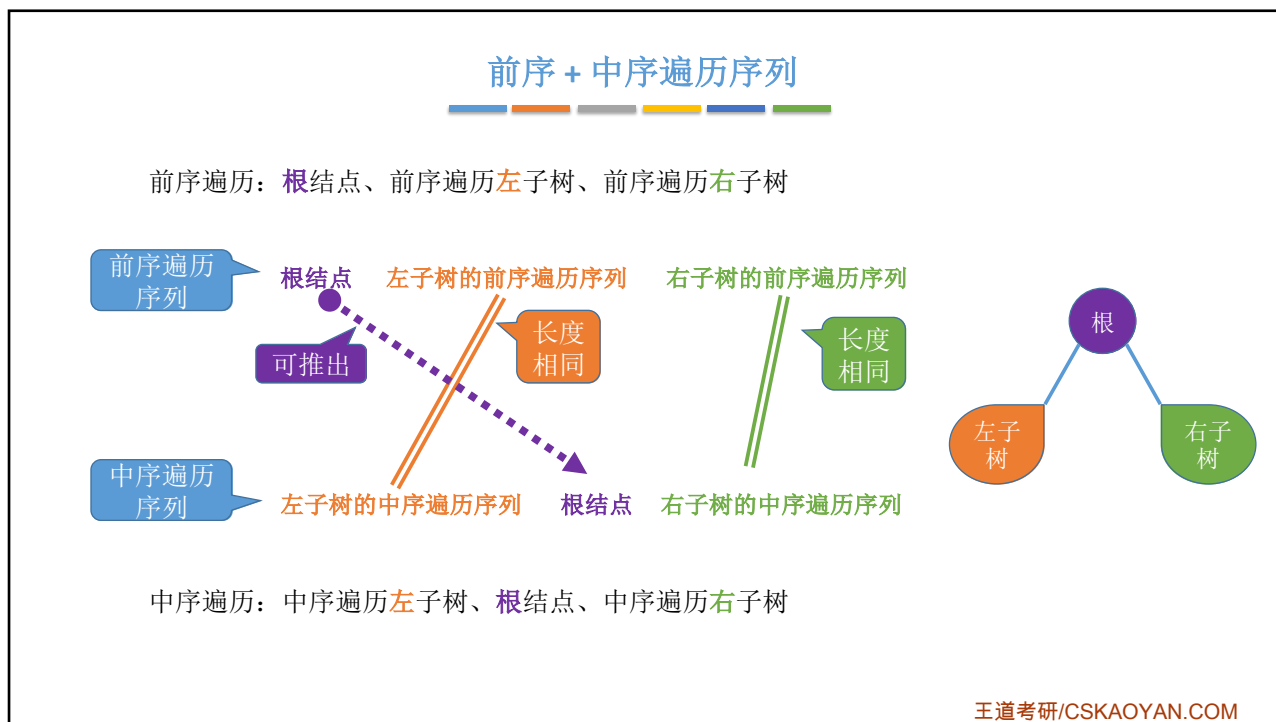
4



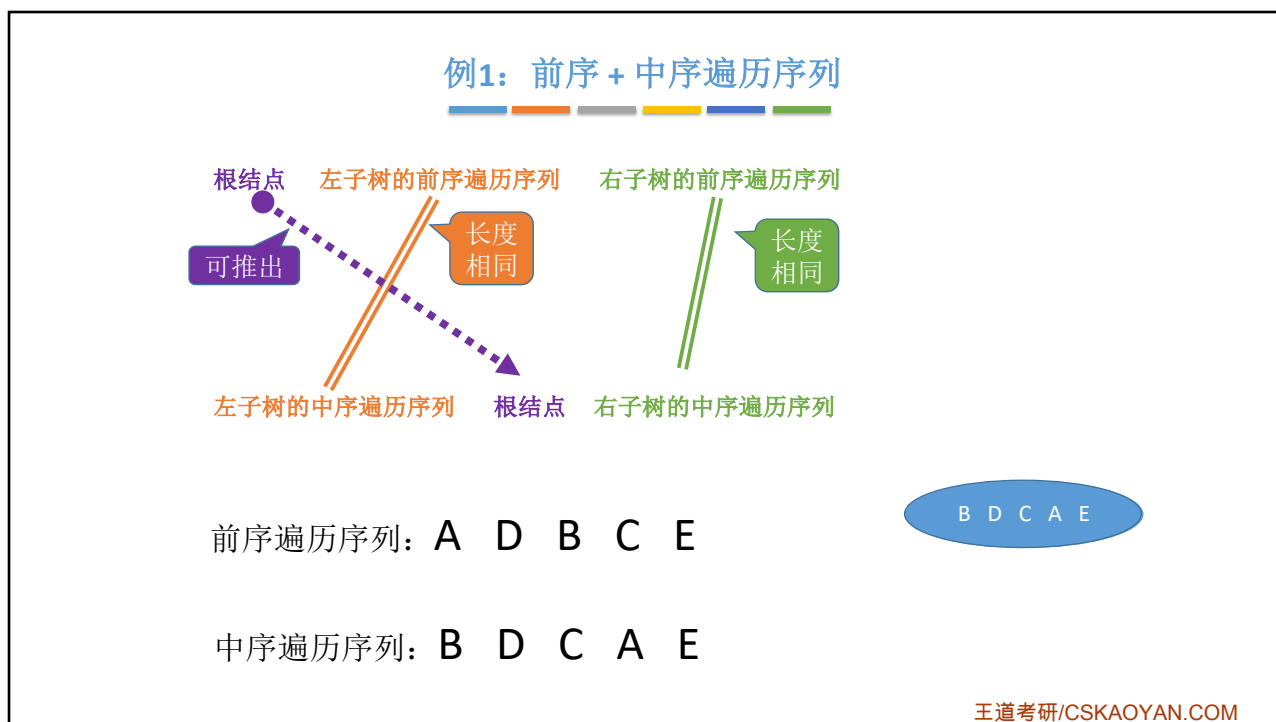
5



6



7



8

例1: 前序 + 中序遍历序列

根结点

左子树的前序遍历序列

右子树的前序遍历序列

可推出

长度相同

长度相同

左子树的中序遍历序列

根结点

右子树的中序遍历序列

前序遍历序列: A D B C E

中序遍历序列: B D C A E

A

BDC

E

王道考研/CSKAOYAN.COM

9

例1: 前序 + 中序遍历序列

根结点

左子树的前序遍历序列

右子树的前序遍历序列

可推出

长度相同

长度相同

左子树的中序遍历序列

根结点

右子树的中序遍历序列

前序遍历序列: A D B C E

中序遍历序列: B D C A E

A

BDC

E

王道考研/CSKAOYAN.COM

10

王道考研/CSKAOYAN.COM

5

例1: 前序 + 中序遍历序列

根结点
可推出

左子树的前序遍历序列

右子树的前序遍历序列

左子树的中序遍历序列

根结点

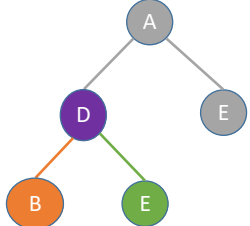
右子树的中序遍历序列

长度相同

长度相同

前序遍历序列: A D B C E

中序遍历序列: B D C A E



王道考研/CSKAOYAN.COM

11

例1: 前序 + 中序遍历序列

根结点
可推出

左子树的前序遍历序列

右子树的前序遍历序列

左子树的中序遍历序列

根结点

右子树的中序遍历序列

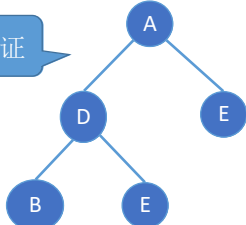
长度相同

长度相同

前序遍历序列: A D B C E

中序遍历序列: B D C A E

注意验证



王道考研/CSKAOYAN.COM

12

例2: 前序 + 中序遍历序列

根结点

可推出

左子树的前序遍历序列

长度相同

右子树的前序遍历序列

长度相同

左子树的中序遍历序列

根结点

右子树的中序遍历序列

EA F D H C B G I

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

13

例2: 前序 + 中序遍历序列

根结点

可推出

左子树的前序遍历序列

长度相同

右子树的前序遍历序列

长度相同

左子树的中序遍历序列

根结点

右子树的中序遍历序列

D

EA F

H C B G I

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

14

例2: 前序 + 中序遍历序列

根结点 左子树的前序遍历序列 右子树的前序遍历序列

可推出 长度相同 长度相同

左子树的中序遍历序列 根结点 右子树的中序遍历序列

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

15

例2: 前序 + 中序遍历序列

根结点 左子树的前序遍历序列 右子树的前序遍历序列

可推出 长度相同 长度相同

左子树的中序遍历序列 根结点 右子树的中序遍历序列

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

16

例2: 前序 + 中序遍历序列

根结点
可推出

左子树的前序遍历序列

右子树的前序遍历序列

左子树的中序遍历序列

根结点

右子树的中序遍历序列

长度相同

长度相同

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

17

例2: 前序 + 中序遍历序列

根结点
可推出

左子树的前序遍历序列

右子树的前序遍历序列

左子树的中序遍历序列

根结点

右子树的中序遍历序列

长度相同

长度相同

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

18

例2: 前序 + 中序遍历序列

根结点

左子树的前序遍历序列

右子树的前序遍历序列

可推出

长度相同

长度相同

左子树的中序遍历序列

根结点

右子树的中序遍历序列

注意验证

前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

王道考研/CSKAOYAN.COM

19

后序 + 中序遍历序列

后序遍历: 前序遍历左子树、前序遍历右子树、根结点

后序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

可推出

长度相同

长度相同

左子树的中序遍历序列

根结点

右子树的中序遍历序列

中序遍历: 中序遍历左子树、根结点、中序遍历右子树

王道考研/CSKAOYAN.COM

20

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

根结点

可推出

右子树的中序遍历序列

后序遍历序列: E F A H C I G B D

中序遍历序列: E A F D H C B G I

E A F D H C B G I

王道考研/CSKAOYAN.COM

21

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

根结点

可推出

右子树的中序遍历序列

后序遍历序列: E F A H C I G B D

中序遍历序列: E A F D H C B G I

```

graph TD
    D((D)) --- EAF(EAF)
    D --- HCBGI(HCBGI)
        
```

王道考研/CSKAOYAN.COM

22

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

右子树的中序遍历序列

根结点

可推出

后序遍历序列: **E F A** H C I G B D

中序遍历序列: **E A F** D H C B G I

王道考研/CSKAOYAN.COM

23

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

右子树的中序遍历序列

根结点

可推出

后序遍历序列: **E F A** H C I G B D

中序遍历序列: **E A F** D H C B G I

王道考研/CSKAOYAN.COM

24

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

右子树的中序遍历序列

根结点

可推出

后序遍历序列: E F A H C I G B D

中序遍历序列: E A F D H C B G I

```

graph TD
    D((D)) --- A((A))
    D --- HCBGI((HCBGI))
    A --- E((E))
    A --- F((F))
    
```

王道考研/CSKAOYAN.COM

25

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

右子树的中序遍历序列

根结点

可推出

后序遍历序列: E F A H C I G B D

中序遍历序列: E A F D H C B G I

```

graph TD
    D((D)) --- A((A))
    D --- B((B))
    A --- E((E))
    A --- F((F))
    B --- HC((HC))
    B --- GI((GI))
    
```

王道考研/CSKAOYAN.COM

26

例3: 后序 + 中序遍历序列

左子树的后序遍历序列

长度相同

左子树的中序遍历序列

右子树的后序遍历序列

长度相同

右子树的中序遍历序列

根结点

可推出

后序遍历序列: E F A H C I G B D

中序遍历序列: E A F D H C B G I

注意验证

王道考研/CSKAOYAN.COM

27

层序 + 中序遍历序列

层序遍历序列

根结点

可推出

左子树的根

进一步确认左子树的根

右子树的根

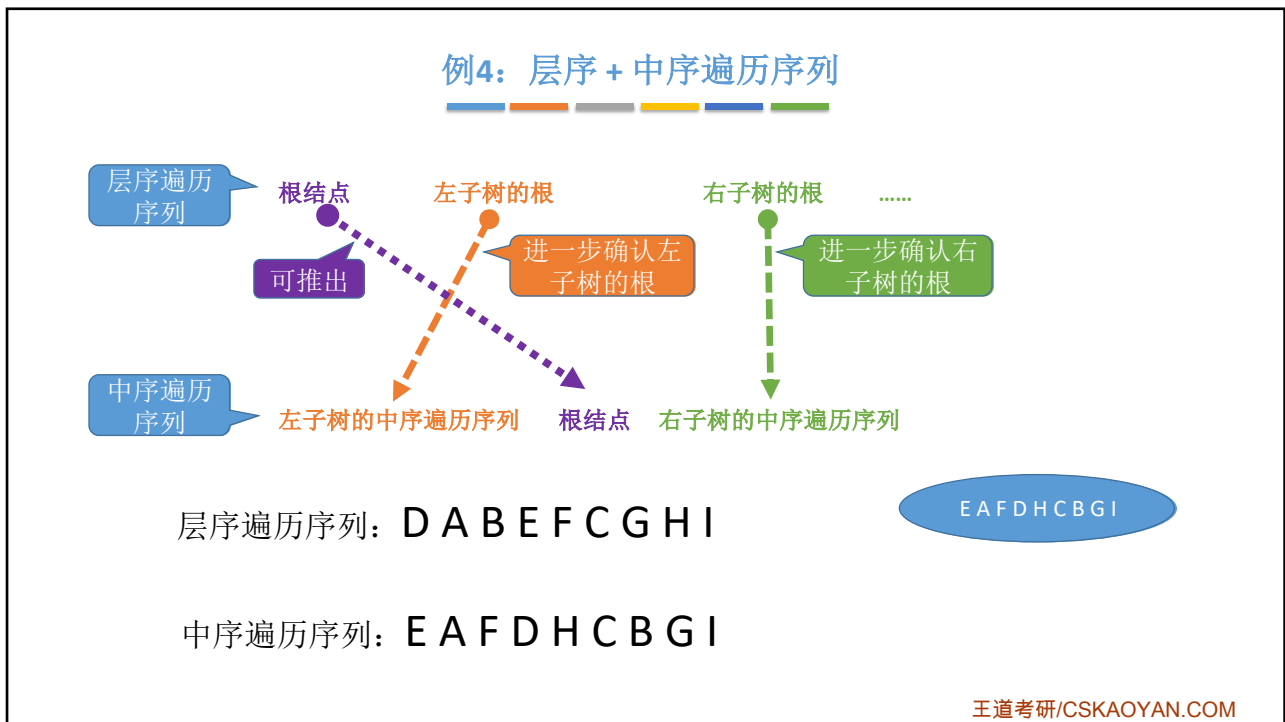
进一步确认右子树的根

中序遍历序列

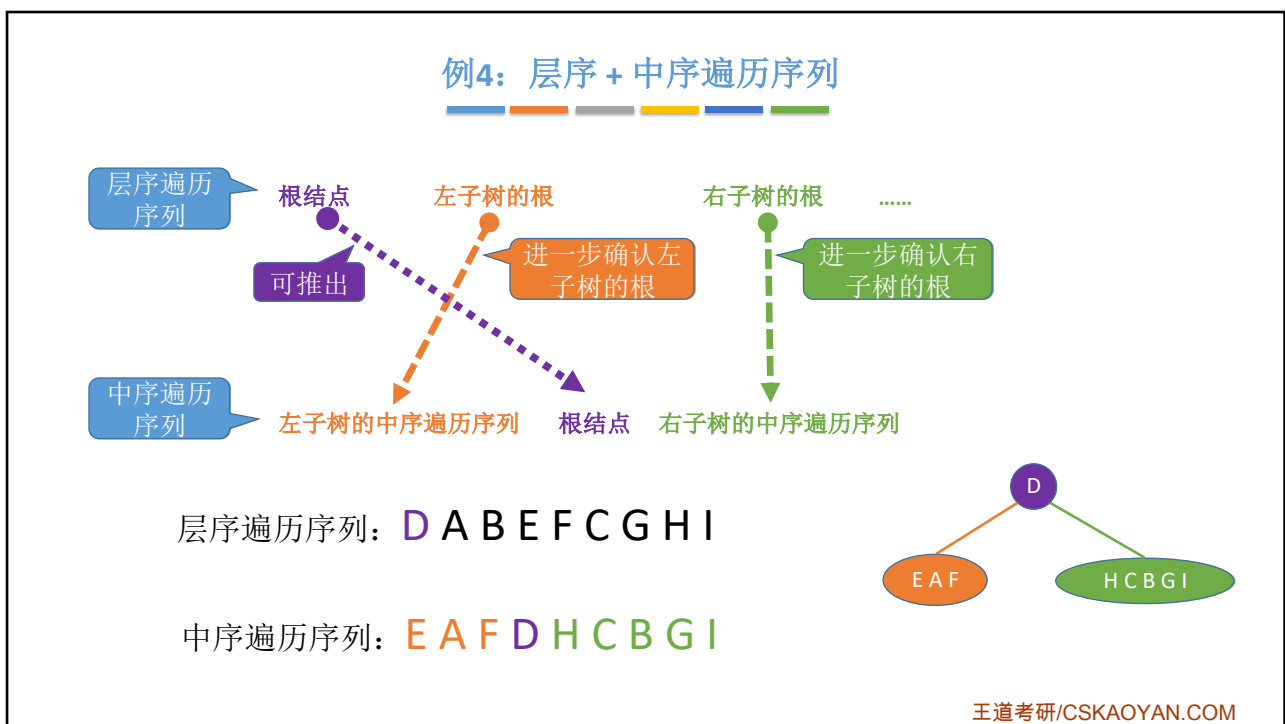
左子树的中序遍历序列 根结点 右子树的中序遍历序列

王道考研/CSKAOYAN.COM

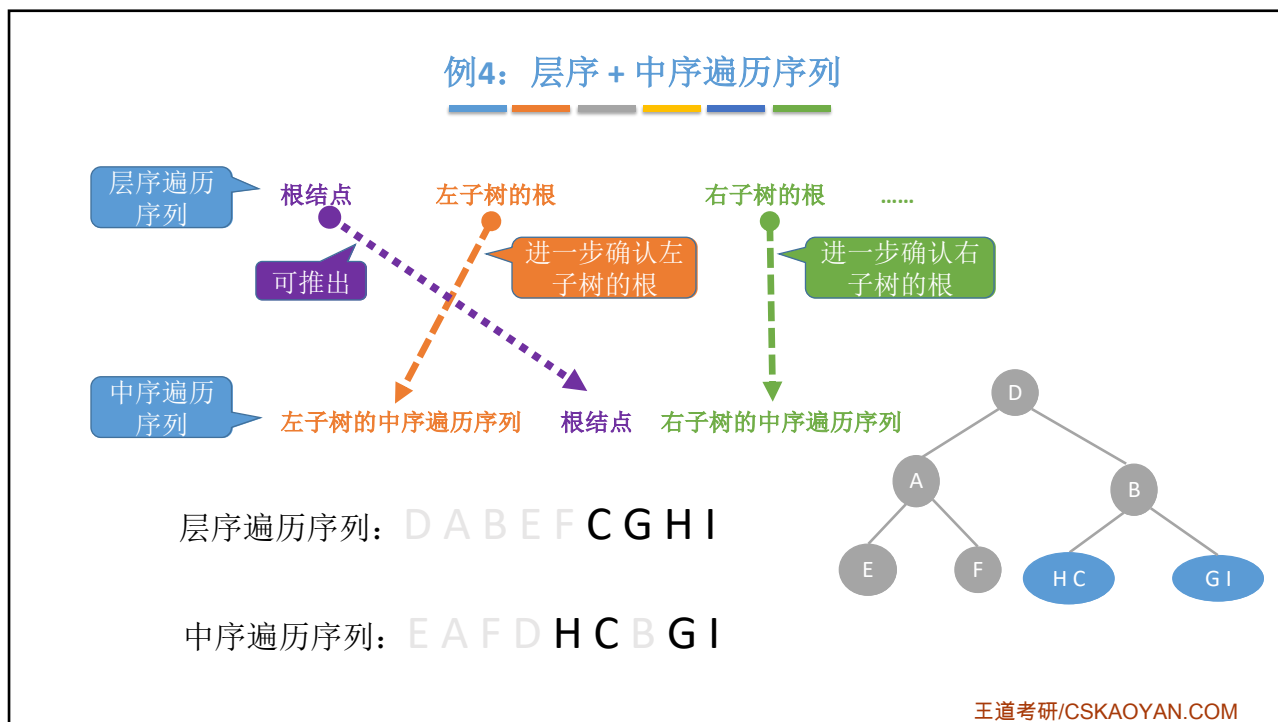
28



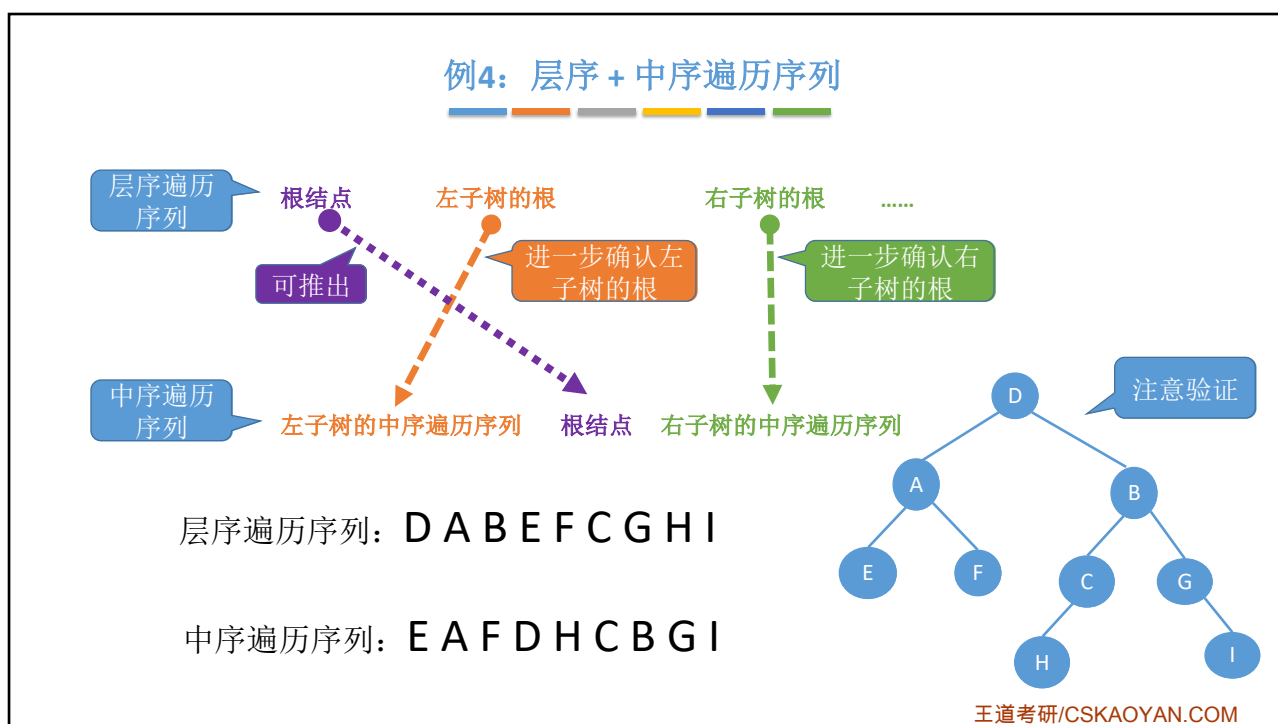
29



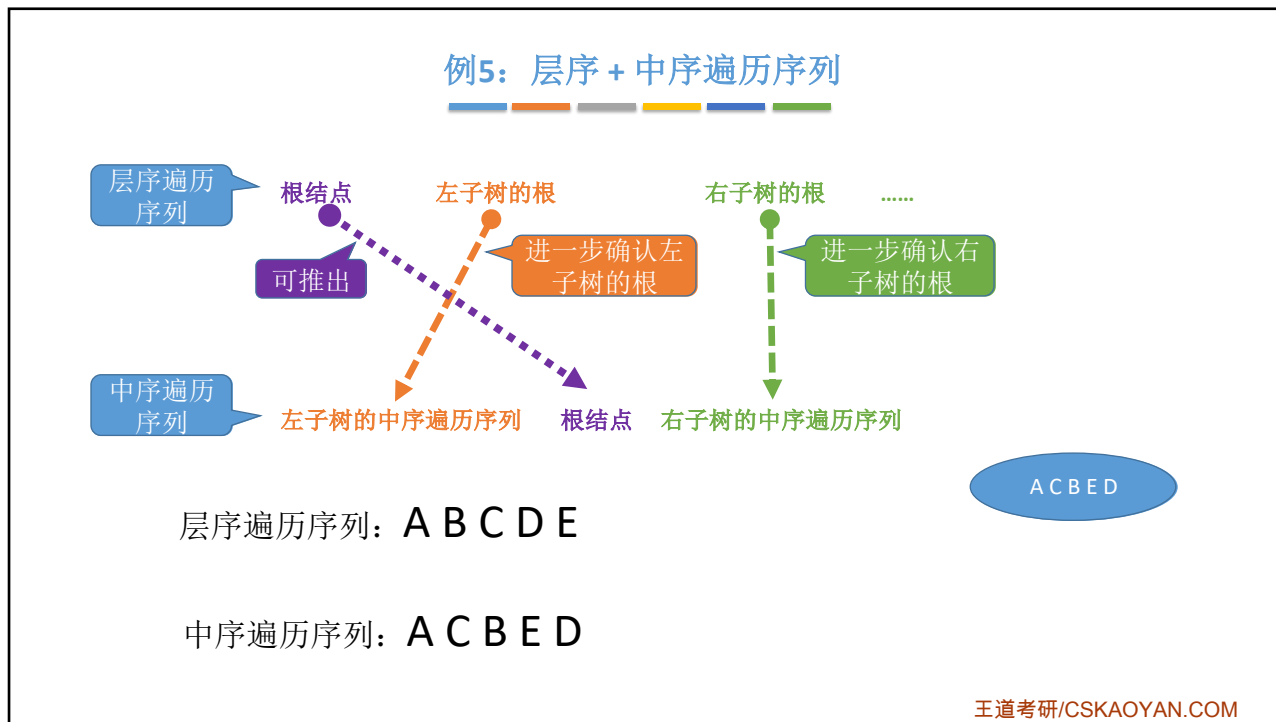
30



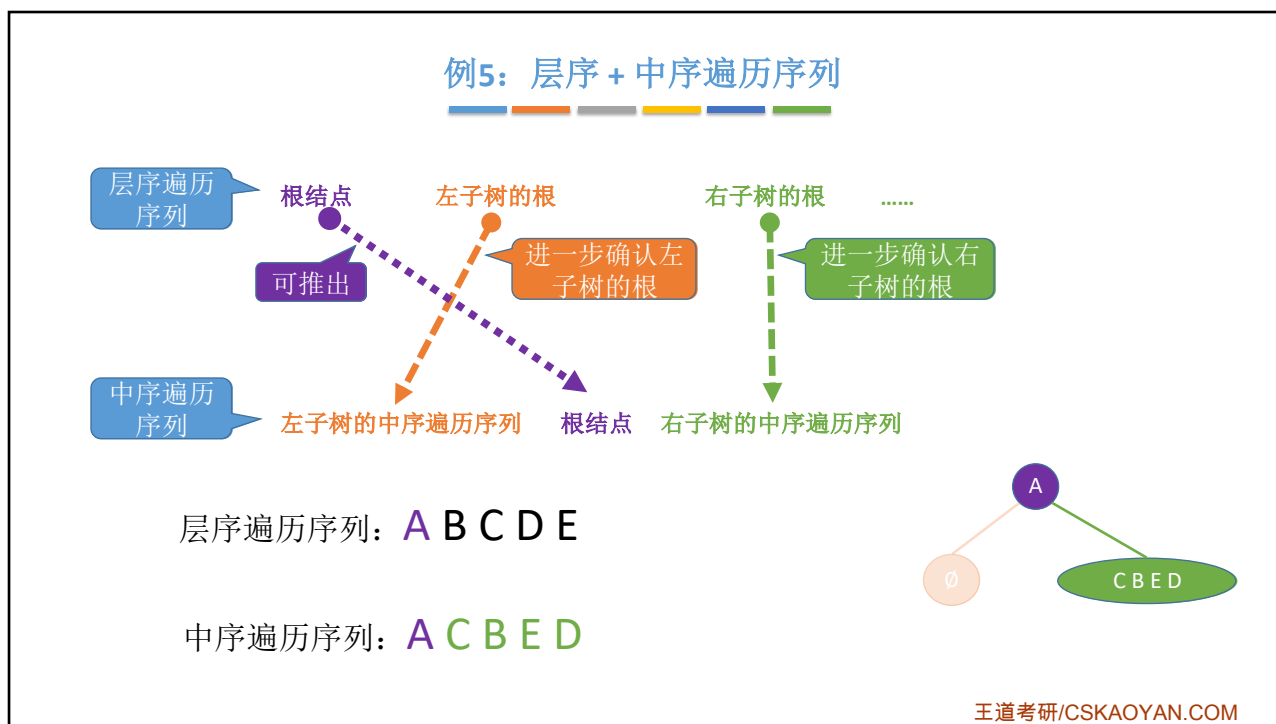
31



32



33



34

例5: 层序 + 中序遍历序列

层序遍历序列

根结点
可推出

中序遍历序列

左子树的中序遍历序列 根结点 右子树的中序遍历序列

左子树的根 右子树的根

进一步确认左子树的根 进一步确认右子树的根

层序遍历序列: A B C D E

中序遍历序列: A C B E D

王道考研/CSKAOYAN.COM

35

例5: 层序 + 中序遍历序列

层序遍历序列

根结点
可推出

中序遍历序列

左子树的中序遍历序列 根结点 右子树的中序遍历序列

左子树的根 右子树的根

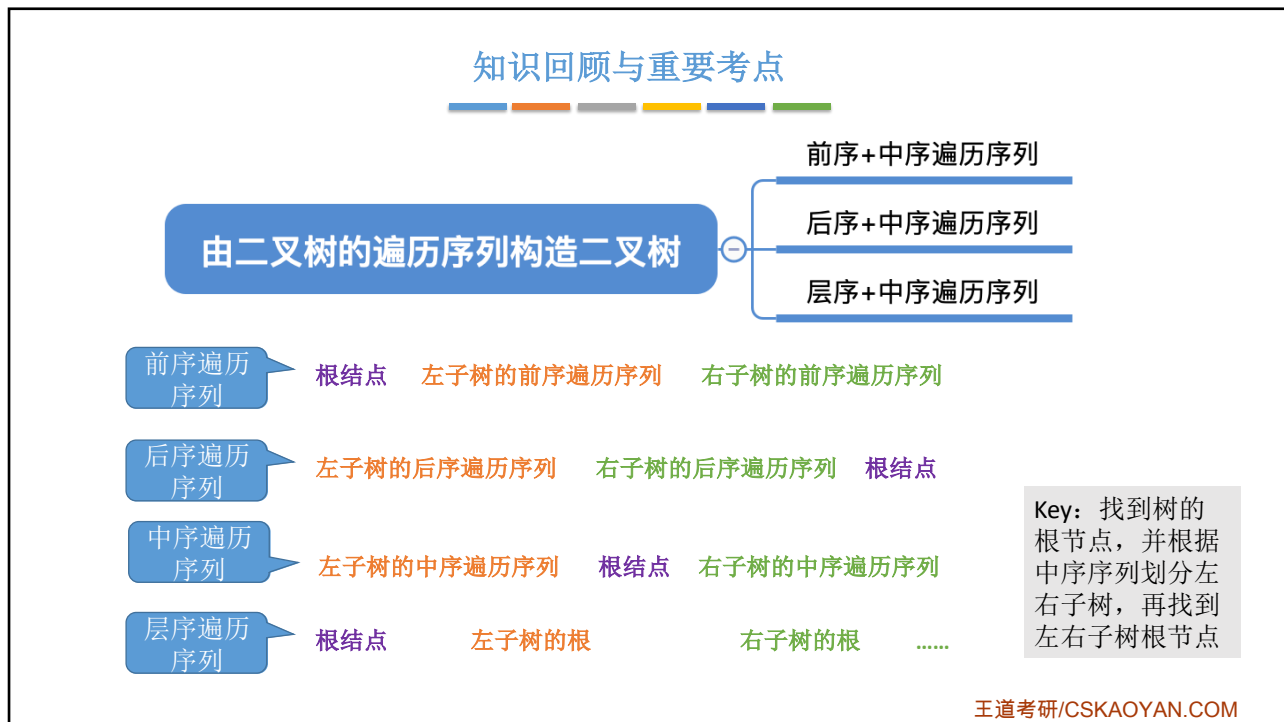
进一步确认左子树的根 进一步确认右子树的根

层序遍历序列: A B C D E

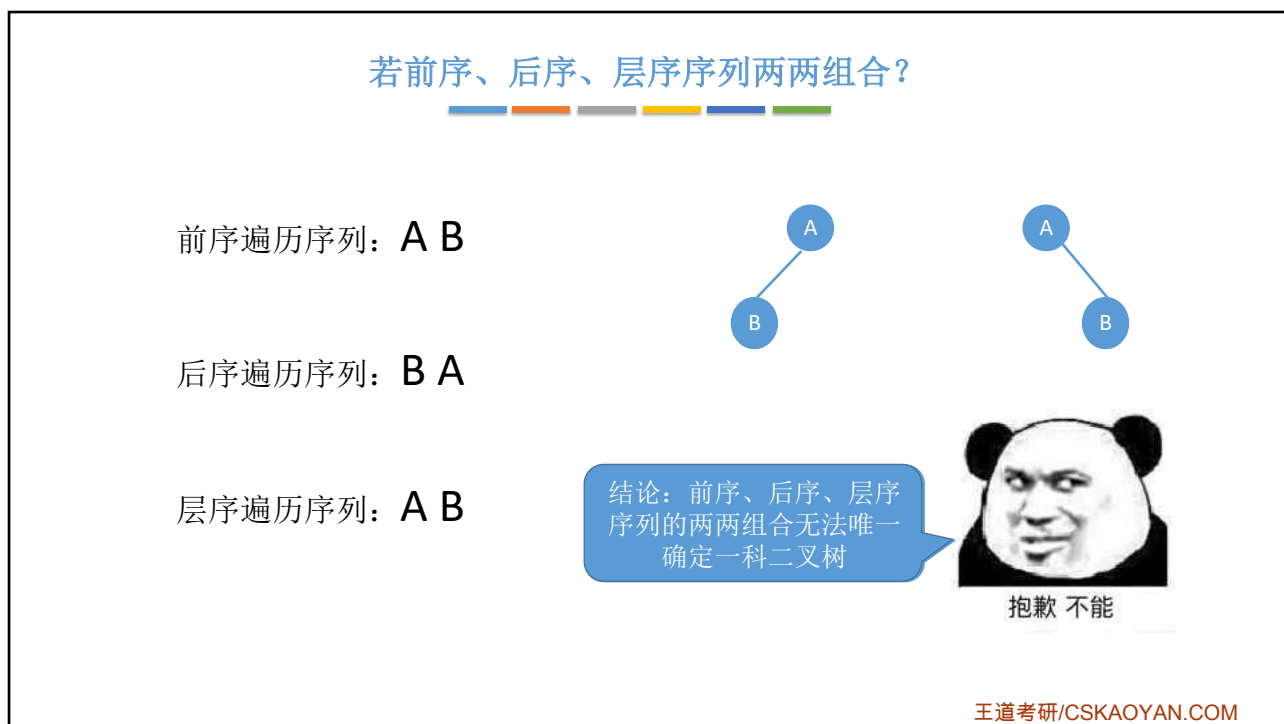
中序遍历序列: A C B E D

王道考研/CSKAOYAN.COM

36



37



38

本节内容

线索二叉树
概念

王道考研/CSKAOYAN.COM

1

知识总览

线索二叉树

线索二叉树的作用

线索二叉树的存储结构

三种线索二叉树

王道考研/CSKAOYAN.COM

2

二叉树的中序遍历序列

中序遍历序列: D G B E A F C

能否从一个指定结点开始中序遍历?

```
//中序遍历
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild); //递归遍历左子树
        visit(T);           //访问根结点
        InOrder(T->rchild); //递归遍历右子树
    }
}
```

①如何找到指定结点p在中序遍历序列中的前驱?

②如何找到p的中序后继?

思路:
从根节点出发,重新进行一次中序遍历,指针q记录当前访问的结点,指针pre记录上一个被访问的结点
①当q==p时,pre为前驱
②当pre==p时,q为后继

缺点:找前驱、后继很不方便;遍历操作必须从根开始

王道考研/CSKAOYAN.COM

3

中序线索二叉树

中序遍历序列: D G B E A F C

n个结点的二叉树,有n+1个空链域!可用来记录前驱、后继的信息

线索化

问题:如何找到G的后继?

图示说明
前驱线索(由左孩子指针充当):
后继线索(由右孩子指针充当):

指向前驱、后继的指针称为“线索”

王道考研/CSKAOYAN.COM

4

线索二叉树的存储结构

图示说明
前驱线索（由左孩子指针充当）：
后继线索（由右孩子指针充当）：

```
//二叉树的结点（链式存储）
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;

//线索二叉树结点
typedef struct ThreadNode{
    ElemType data;
    struct ThreadNode *lchild,*rchild;
    int ltag,rtag; //左、右线索标志
}ThreadNode,*ThreadTree;
```

术语：二叉链表

术语：线索链表

王道考研/CSKAOYAN.COM

5

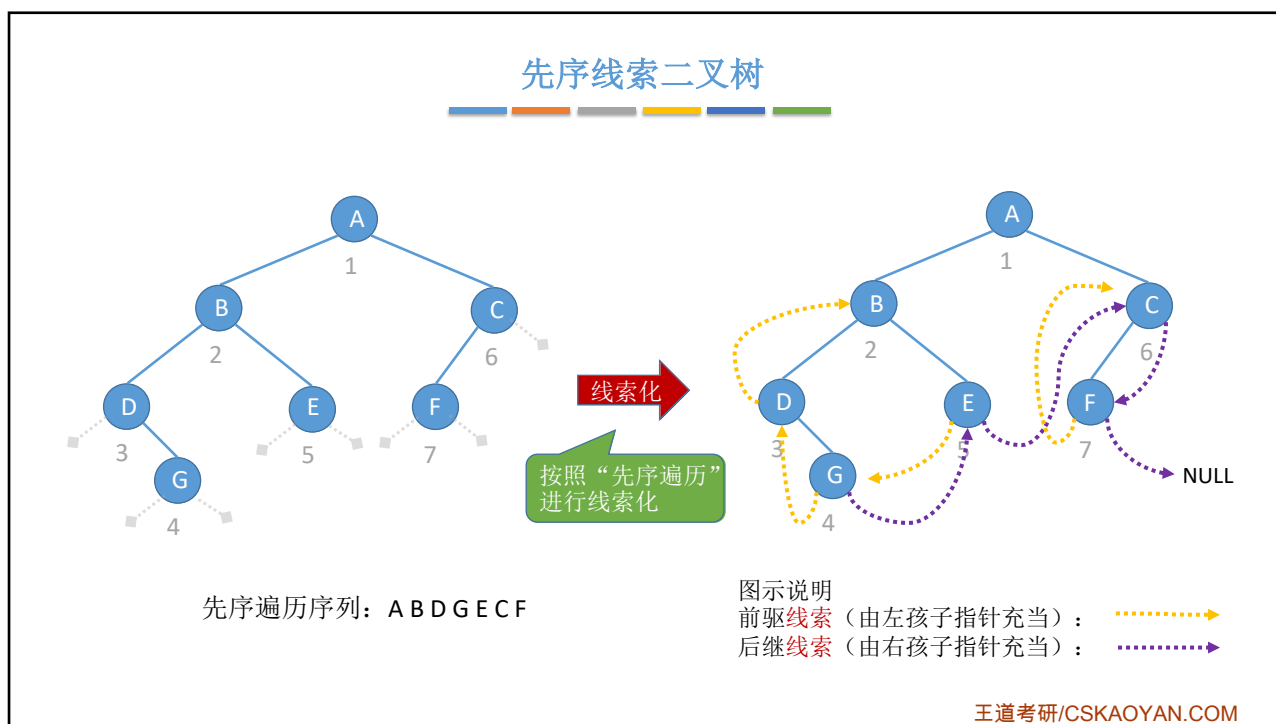
中序线索二叉树的存储

对应tag位为0时，表示指针指向其孩子

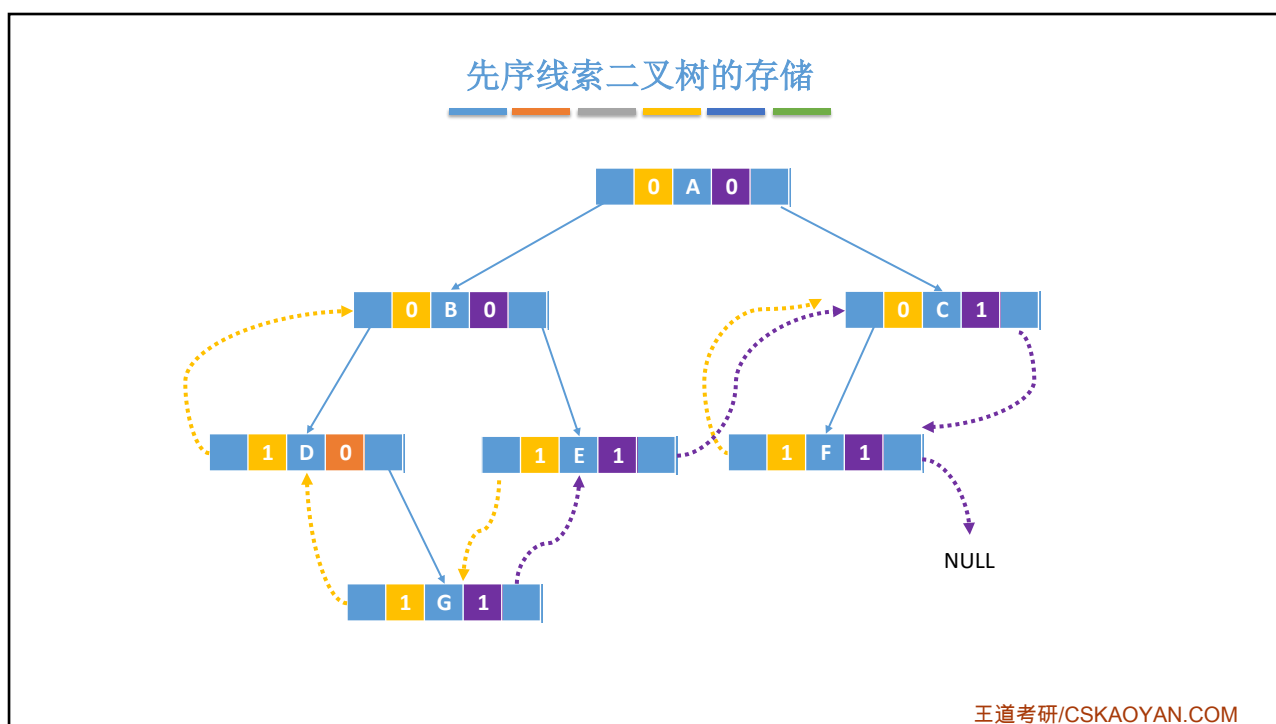
对应tag位为1时，表示指针是“线索”

王道考研/CSKAOYAN.COM

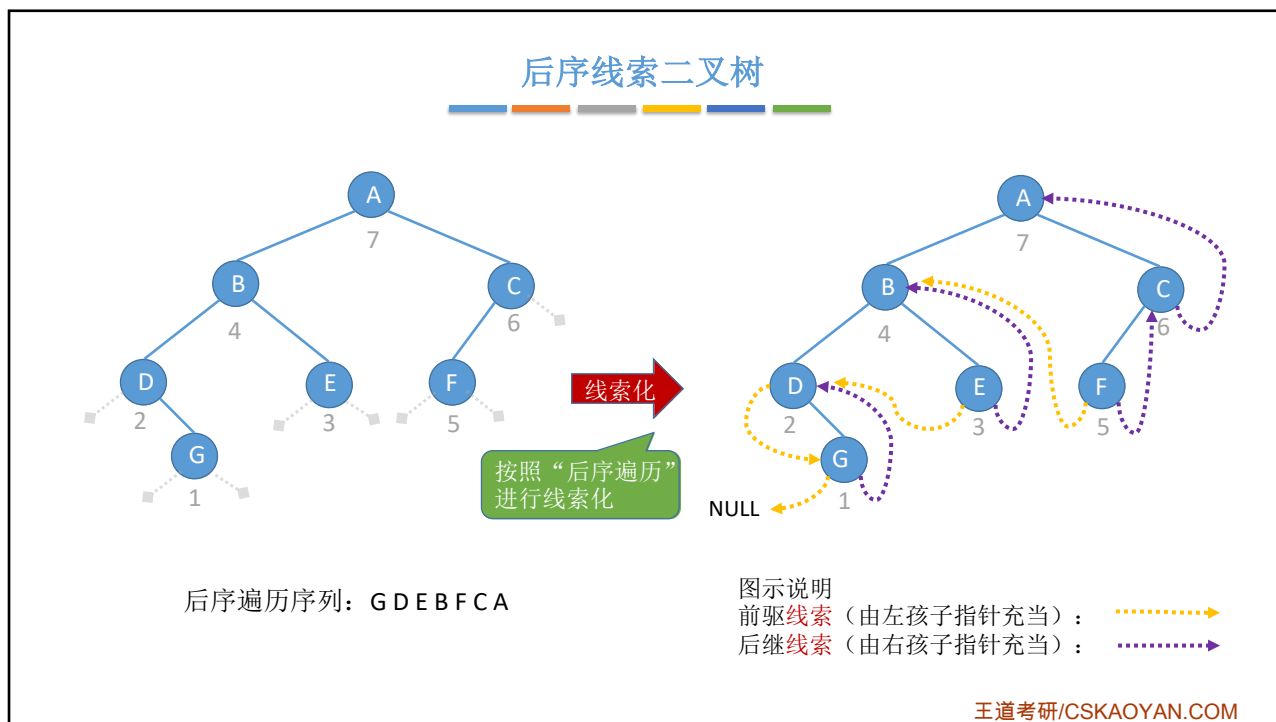
6



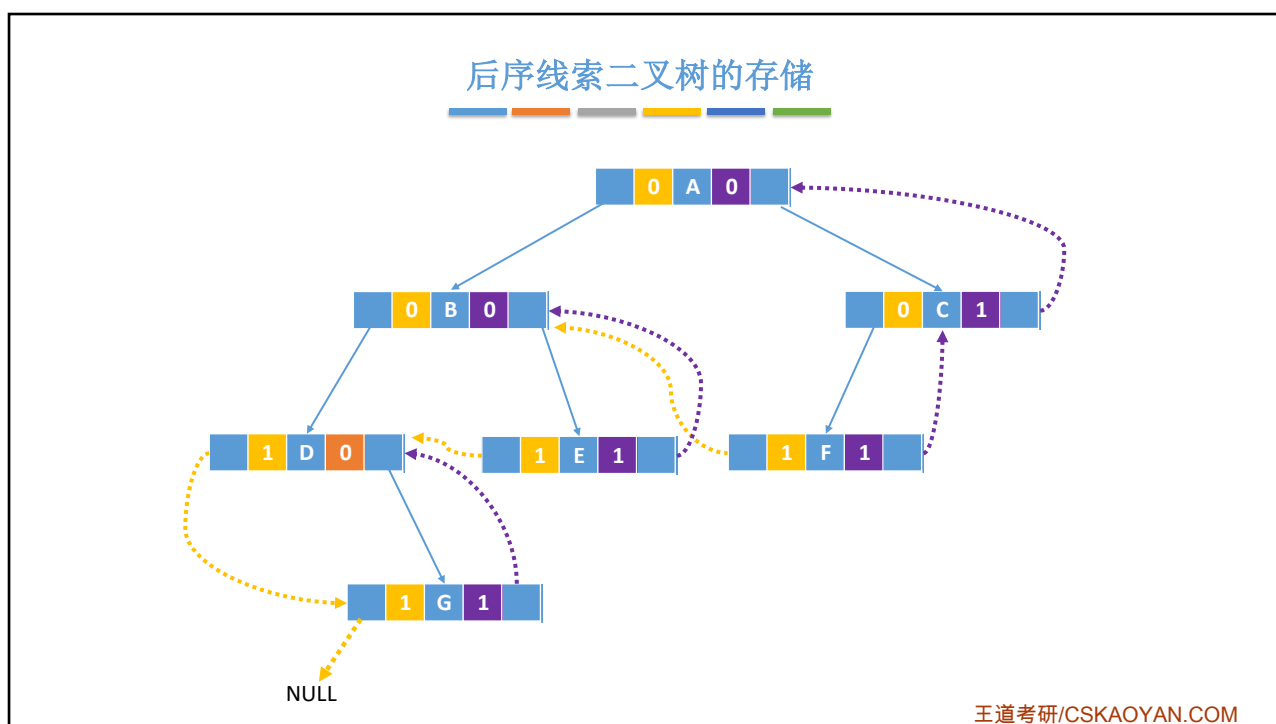
7



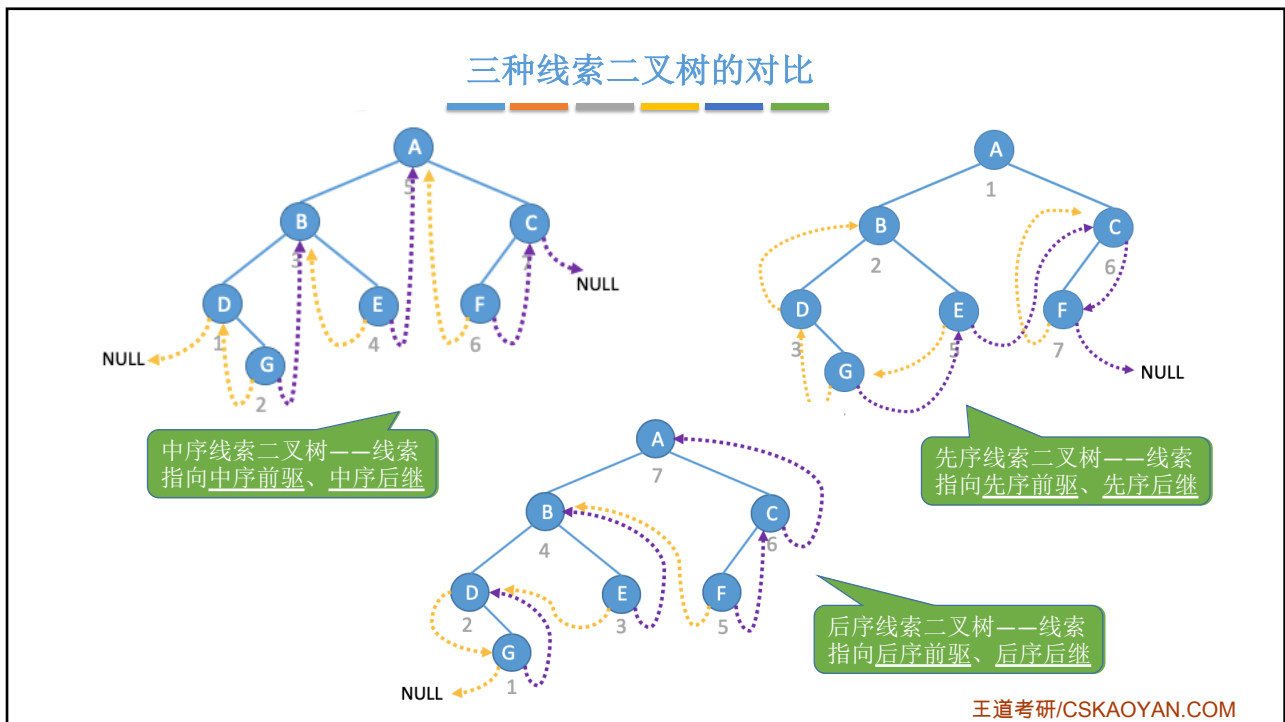
8



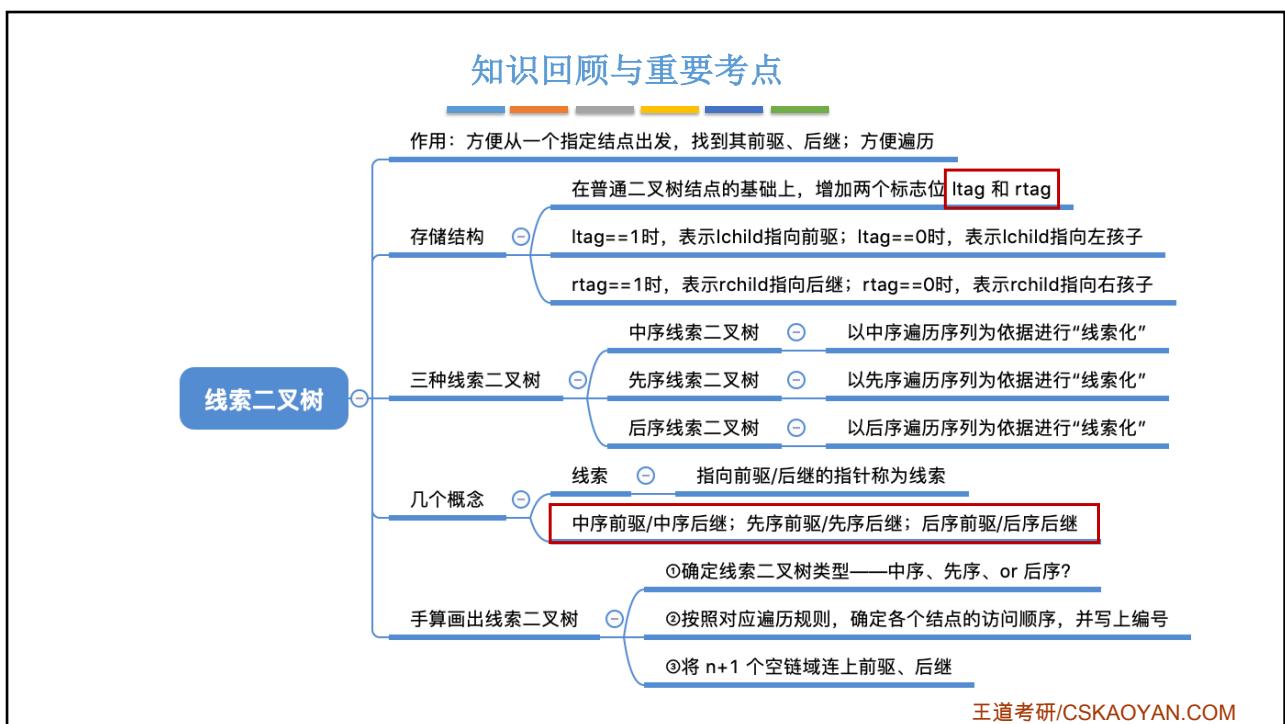
9



10



11

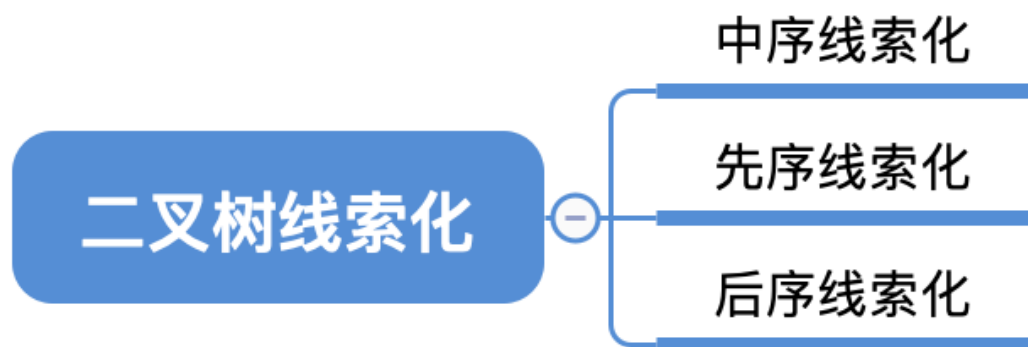


12

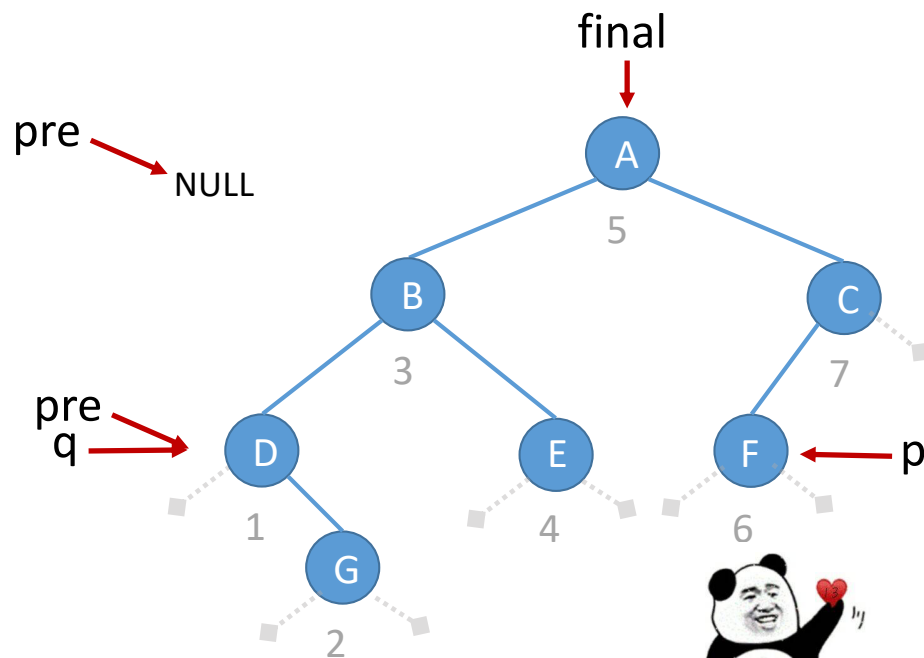
本节内容

二叉树的 线索化

知识总览



用土办法找到中序前驱



啊找到了

中序遍历序列: D G B E A F C

最好改一个函数名，如 findPre

```
//中序遍历
```

```
void InOrder(BiTree T){
```

```
if (T != NULL) {
```

```
InOrder(T->lchild);
```

```
//递归遍历左子树
```

```
visit(T);
```

```
//访问根结点
```

```
InOrder(T->rchild);
```

```
//递归遍历右子树
```

}

}

```
//访问结点q
```

```
void visit(BiTNode * q){
```

```
if (q==p)
```

```
//当前访问结点刚好是结点p
```

```
final = pre;
```

```
//找到p的前驱
```

```
else
```

```
pre = q;
```

//pre指向当前访问的结点

3

//辅助全局变量, 用于查找结点p的前驱

BiTNode *p;

//p指向目标结点

```
BiTNode * pre=NULL;
```

//指向当前访问结点的前驱

```
BiTNode * final=NULL;
```

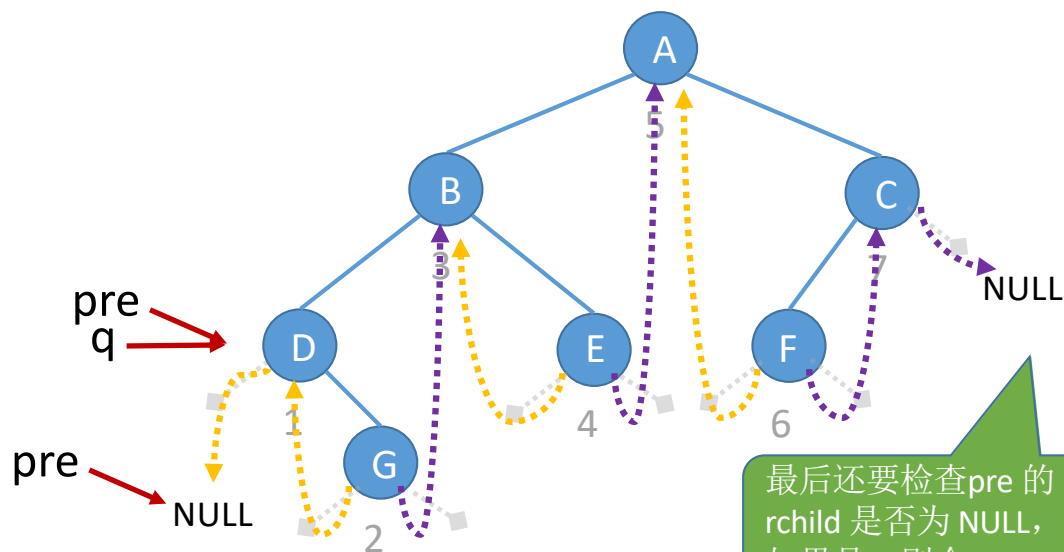
```
//用于记录最终结果
```

王道考研/CSKAOYAN.COM

初步建成的树,
ltag、rtag=0

中序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------



//线索二叉树结点

```
typedef struct ThreadNode{
    ElemType data;
    struct ThreadNode *lchild,*rchild;
    int ltag,rtag;    //左、右线索标志
}ThreadNode,* ThreadTree;
```

//中序遍历二叉树, 一边遍历一边线索化

```
void InThread(ThreadTree T){
    if(T!=NULL){
        InThread(T->lchild);    //中序遍历左子树
        visit(T);                //访问根节点
        InThread(T->rchild);    //中序遍历右子树
    }
}

void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空, 建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q;    //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}

//全局变量 pre, 指向当前访问结点的前驱
ThreadNode *pre=NULL;
```

王道考研/CSKAOYAN.COM

初步建成的树，
ltag、rtag=0

中序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

```
//全局变量 pre, 指向当前访问结点的前驱
ThreadNode *pre=NULL;

//中序线索化二叉树T
void CreateInThread(ThreadTree T){
    pre=NULL;                //pre初始为NULL
    if(T!=NULL){              //非空二叉树才能线索化
        InThread(T);           //中序线索化二叉树
        if (pre->rchild==NULL)
            pre->rtag=1;        //处理遍历的最后一个结点
    }
}

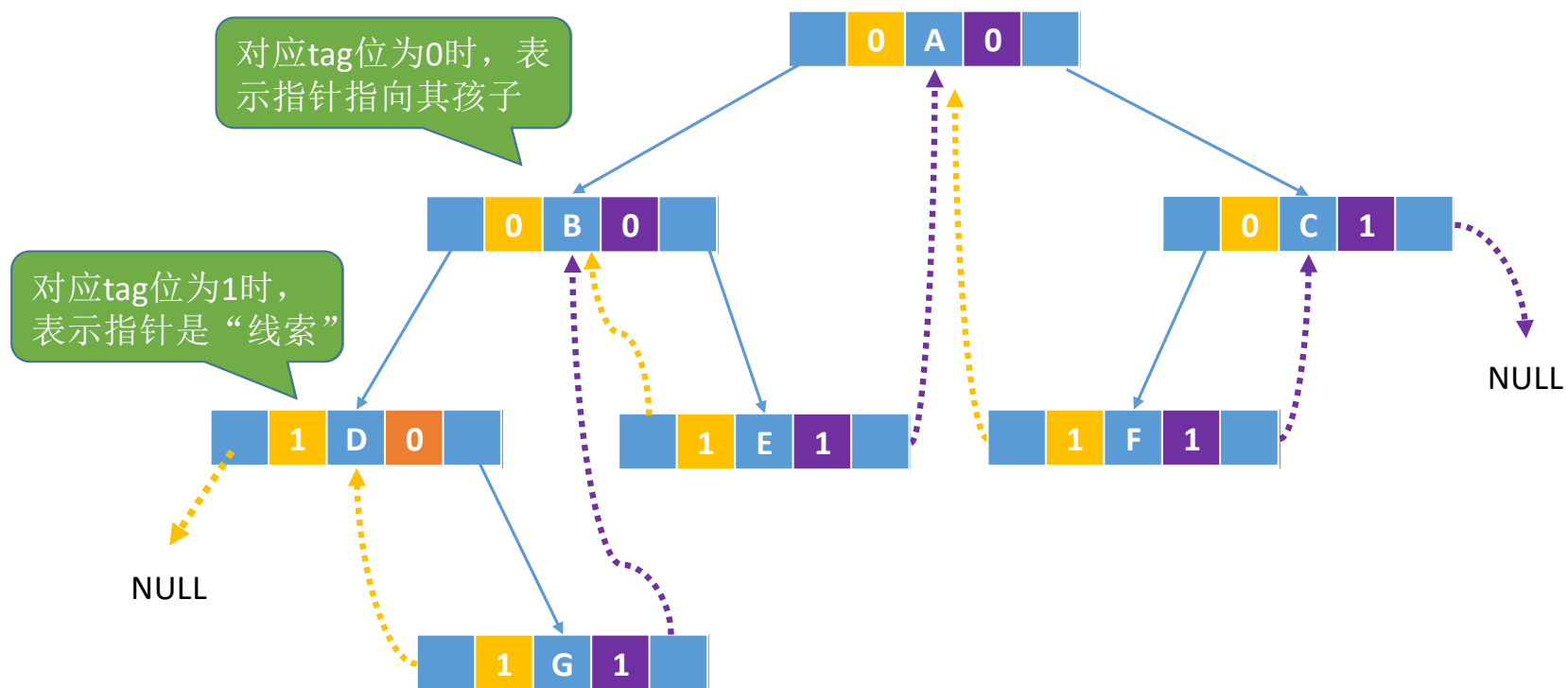
//线索二叉树结点
typedef struct ThreadNode{
    ElemType data;
    struct ThreadNode *lchild,*rchild;
    int ltag,rtag;           //左、右线索标志
}ThreadNode,* ThreadTree;
```

//中序遍历二叉树，一边遍历一边线索化

```
void InThread(ThreadTree T){
    if(T!=NULL){
        InThread(T->lchild);    //中序遍历左子树
        visit(T);               //访问根节点
        InThread(T->rchild);    //中序遍历右子树
    }
}

void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q; //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```

中序线索二叉树



中序线索化（王道教材版）

```
//中序线索化
void InThread(ThreadTree p, ThreadTree &pre){
    if(p != NULL){
        InThread(p->lchild, pre);    //递归，线索化左子树
        if(p->lchild == NULL){      //左子树为空，建立前驱线索
            p->lchild = pre;
            p->ltag = 1;
        }
        if(pre != NULL && pre->rchild == NULL){
            pre->rchild = p;        //建立前驱结点的后继线索
            pre->rtag = 1;
        }
        pre = p;
        InThread(p->rchild, pre);
    } //if(p != NULL)
}
```

思考：为什么 pre 参数是引用类型？

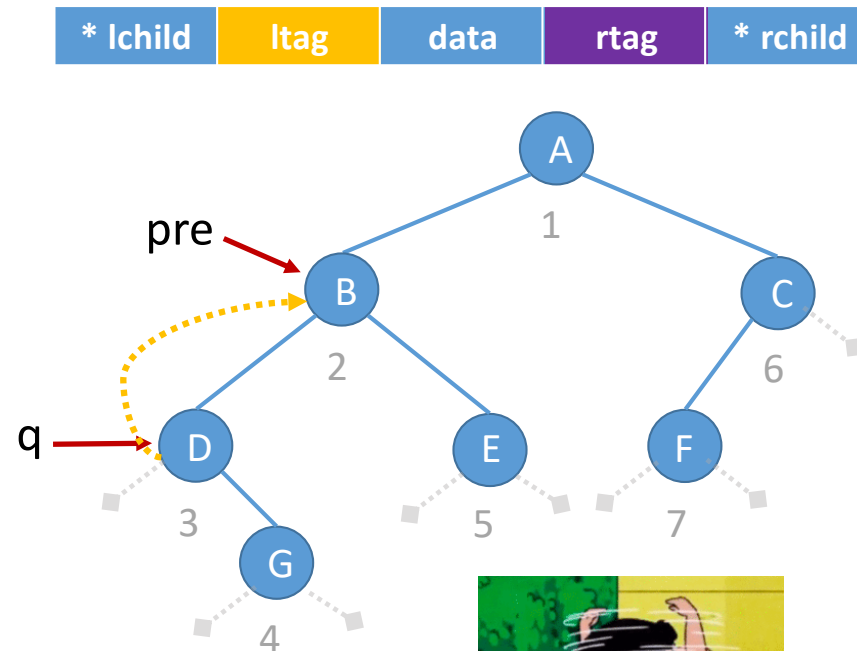
思考：处理遍历的最后一个结点时，为什么没有判断 rchild 是否为 NULL？

答：中序遍历的最后一个结点右孩子指针必为空。

```
//中序线索化二叉树T
void CreateInThread(ThreadTree T){
    ThreadTree pre = NULL;
    if(T != NULL){
        InThread(T, pre);    //非空二叉树，线索化
        pre->rchild = NULL;  //线索化二叉树
        pre->rtag = 1;        //处理遍历的最后一个结点
    }
}
```

初步建成的树,
ltag、rtag=0

先序线索化



//先序遍历二叉树, 一边遍历一边线索化

```
void PreThread(ThreadTree T){  
    if(T!=NULL){  
        visit(T); //先处理根节点  
        PreThread(T->lchild);  
        PreThread(T->rchild);  
    }  
}  
  
void visit(ThreadNode *q) {  
    if(q->lchild==NULL){ //左子树为空, 建立前驱线索  
        q->lchild=pre;  
        q->ltag=1;  
    }  
    if(pre!=NULL&&pre->rchild==NULL){  
        pre->rchild=q; //建立前驱结点的后继线索  
        pre->rtag=1;  
    }  
    pre=q;  
}
```

//全局变量 pre, 指向当前访问结点的前驱

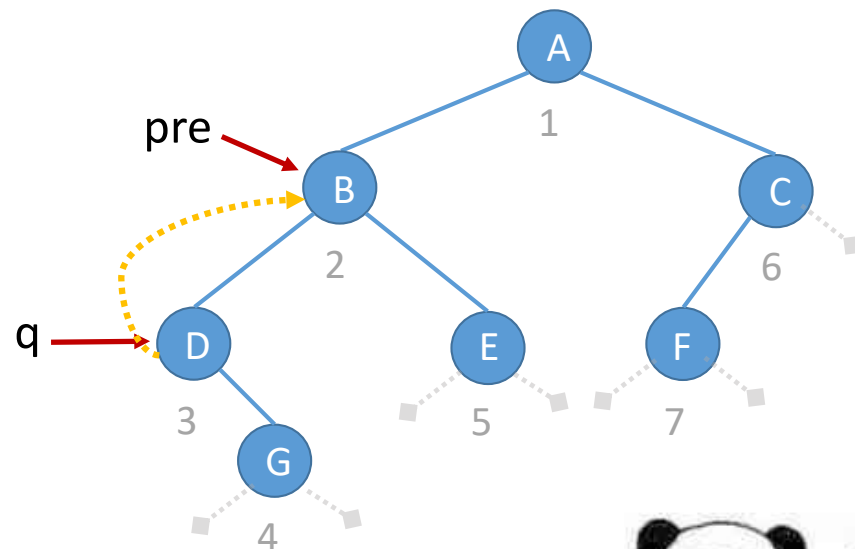
ThreadNode *pre=NULL;

王道考研/CSKAOYAN.COM

初步建成的树,
ltag、rtag=0

先序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------



可以

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){  
    if(T!=NULL){  
        visit(T);           //先处理根节点  
        PreThread(T->lchild);  
        PreThread(T->rchild);  
    }  
}
```

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){  
    if(T!=NULL){  
        visit(T);           //先处理根节点  
        if (T->ltag==0) //lchild不是前驱线索  
            PreThread(T->lchild);  
        PreThread(T->rchild);  
    }  
}
```

先序线索化

初步建成的树，
ltag、rtag=0

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

//全局变量 pre，指向当前访问结点的前驱
ThreadNode *pre=NULL;

//先序线索化二叉树T

```
void CreatePreThread(ThreadTree T){
    pre=NULL;           //pre初始为NULL
    if(T!=NULL){        //非空二叉树才能线索化
        PreThread(T);    //先序线索化二叉树
        if(pre->rchild==NULL)
            pre->rtag=1; //处理遍历的最后一个结点
    }
}
```

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){
    if(T!=NULL){
        visit(T); //先处理根节点
        if(T->ltag==0) //lchild不是前驱线索
            PreThread(T->lchild);
        PreThread(T->rchild);
    }
}

void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL && pre->rchild==NULL){
        pre->rchild=q; //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```

先序线索化 (王道教材Style)

```
//先序线索化
void PreThread(ThreadTree p, ThreadTree &pre){
    if(p!=NULL){
        if(p->lchild==NULL){           //左子树为空, 建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){
            pre->rchild=p;             //建立前驱结点的后继线索
            pre->rtag=1;
        }
        pre=p;
        if(p->ltag==0)
            PreThread(p->lchild, pre); //递归, 先序遍历左子树
        PreThread(p->rchild, pre);     //递归, 先序遍历右子树
    }
}
```



```
//先序线索化二叉树T
void CreatePreThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){
        PreThread(T, pre);           //非空二叉树, 线索化
        if(pre->rchild==NULL)        //线索化二叉树
            pre->rchild=T;            //处理遍历的最后一个结点
            pre->rtag=1;
    }
}
```

初步建成的树，
ltag、rtag=0

后序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

//全局变量 pre，指向当前访问结点的前驱
ThreadNode *pre=NULL;

//后序线索化二叉树T

```
void CreatePostThread(ThreadTree T){
    pre=NULL;           //pre初始为NULL
    if(T!=NULL){        //非空二叉树才能线索化
        PostThread(T);   //后序线索化二叉树
        if (pre->rchild==NULL)
            pre->rtag=1;  //处理遍历的最后一个结点
    }
}
```

//后遍历二叉树，一边遍历一边线索化

```
void PostThread(ThreadTree T){
    if(T!=NULL){
        PostThread(T->lchild); //后序遍历左子树
        PostThread(T->rchild); //后序遍历右子树
        visit(T);              //访问根节点
    }
}

void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q; //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```

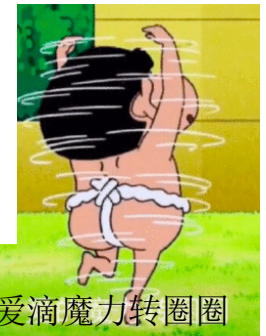

后序线索化（王道教材Style）

//后序线索化

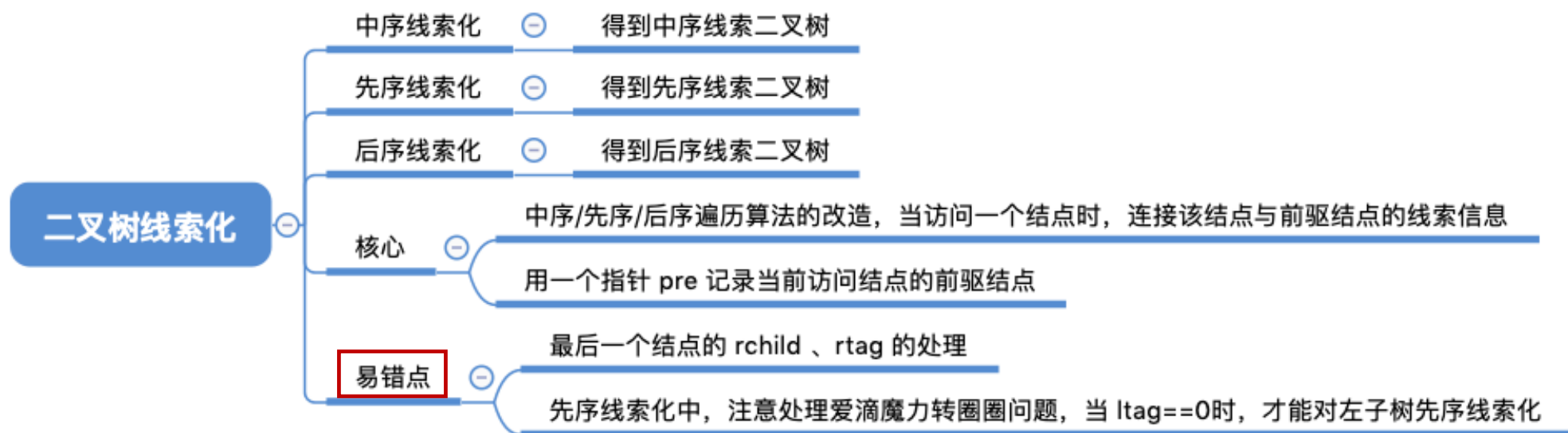
```
void PostThread(ThreadTree p, ThreadTree &pre){
    if(p!=NULL){
        PostThread(p->lchild, pre); //递归，线索化左子树
        PostThread(p->rchild, pre); //递归，线索化右子树
        if(p->lchild==NULL){ //左子树为空，建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
    } //if(p!=NULL)
}
```

//后序线索化二叉树T

```
void CreatePostThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){ //非空二叉树，线索化
        PostThread(T, pre); //线索化二叉树
        if(pre->rchild==NULL) //处理遍历的最后一个结点
            pre->rtag=1;
    }
}
```



知识回顾与重要考点



本节内容

线索二叉树
找前驱/后继

王道考研/CSKAOYAN.COM

1

知识总览

线索二叉树找前驱/后继

中序线索二叉树

先序线索二叉树

后序线索二叉树

王道考研/CSKAOYAN.COM

2

中序线索二叉树找中序后继

中序遍历序列: D G B E A F C

在中序线索二叉树中找到指定结点*p的中序后继 next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$

p 必有右孩子

中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

next = p的右子树中最左下结点

王道考研/CSKAOYAN.COM

3

中序线索二叉树找中序后继

```

//找到以P为根的子树中, 第一个被中序遍历的结点
ThreadNode *Firstnode(ThreadNode *p){
    //循环找到最左下结点(不一定是叶结点)
    while(p->ltag==0) p=p->lchild;
    return p;
}

//在中序线索二叉树中找到结点p的后继结点
ThreadNode *Nextnode(ThreadNode *p){
    //右子树中最左下结点
    if(p->rtag==0) return Firstnode(p->rchild);
    else return p->rchild; //rtag==1直接返回后继线索
}

//对中序线索二叉树进行中序遍历(利用线索实现的非递归算法)
void Inorder(ThreadNode *T){
    for(ThreadNode *p=Firstnode(T);p!=NULL;p=Nextnode(p))
        visit(p);
}
        
```

在中序线索二叉树中找到指定结点*p的中序后继 next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$

p 必有右孩子

中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

next = p的右子树中最左下结点

王道考研/CSKAOYAN.COM

4

中序线索二叉树找中序前驱

中序遍历序列: D G B E A F C

在中序线索二叉树中找到指定结点*p的中序前驱 pre

①若 $p \rightarrow ltag == 1$, 则 $pre = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$

p 必有左孩子

中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

pre = p 的左子树中最右下结点

王道考研/CSKAOYAN.COM

5

中序线索二叉树找中序前驱

```

//找到以P为根的子树中, 最后一个被中序遍历的结点
ThreadNode *Lastnode(ThreadNode *p){
    //循环找到最右下结点(不一定是叶结点)
    while(p->rtag==0) p=p->rchild;
    return p;
}

//在中序线索二叉树中找到结点p的前驱结点
ThreadNode *Prenode(ThreadNode *p){
    //左子树中最右下结点
    if(p->ltag==0) return Lastnode(p->lchild);
    else return p->lchild; //ltag==1直接返回前驱线索
}

//对中序线索二叉树进行逆向中序遍历
void RevInorder(ThreadNode *T){
    for(ThreadNode *p=Lastnode(T);p!=NULL;p=Prenode(p))
        visit(p);
}
        
```

在中序线索二叉树中找到指定结点*p的中序前驱 pre

①若 $p \rightarrow ltag == 1$, 则 $pre = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$

p 必有左孩子

中序遍历——左 根 右

(左 根 右) 根 右

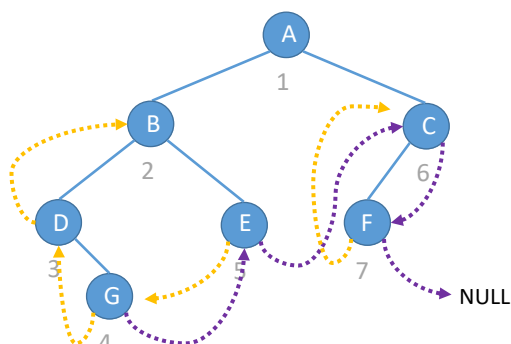
(左 根 (左 根 右)) 根 右

pre = p 的左子树中最右下结点

王道考研/CSKAOYAN.COM

6

先序线索二叉树找先序后继



先序遍历序列: ABDGECF



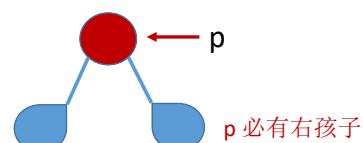
若p有左孩子, 则先序后继为左孩子

若p没有左孩子, 则先序后继为右孩子

在先序线索二叉树中找到指定结点*p 的先序后继 next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$



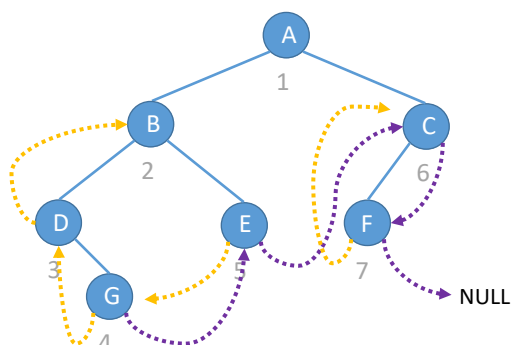
先序遍历——根 左 右
根 (根 左 右) 右

先序遍历——根 右
根 (根 左 右)

王道考研/CSKAOYAN.COM

7

先序线索二叉树找先序前驱



先序遍历序列: ABDGECF

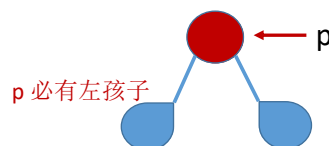


荒唐的答案

在先序线索二叉树中找到指定结点*p 的先序前驱 pre

①若 $p \rightarrow ltag == 1$, 则 $next = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$



先序遍历——根 左 右



先序遍历中, 左右子树中的结点只可能是根的后继, 不可能是前驱

王道考研/CSKAOYAN.COM

8

先序线索二叉树找先序前驱

①如果能找到p的父节点, 且p是左孩子

先序遍历——根 左 右

根 (根 左 右) 右

p的父节点即为其前驱

②如果能找到p的父节点, 且p是右孩子, 其左兄弟为空

先序遍历——根 右

根 (根 左 右)

p的父节点即为其前驱

③如果能找到p的父节点, 且p是右孩子, 其左兄弟非空

根 左 右

p的前驱为左兄弟子树中最后一个被先序遍历的结点

④如果p是根节点, 则p没有先序前驱

王道考研/CSKAOYAN.COM

9

后序线索二叉树找后序前驱

后序遍历序列: GDEBFCA

说着说着老子就要动手了

在后序线索二叉树中找到指定结点*p 的后序前驱 pre

- ①若 $p \rightarrow ltag == 1$, 则 $pre = p \rightarrow lchild$
- ②若 $p \rightarrow ltag == 0$

p 必有左孩子

后序遍历——左 右 根

左 (左 右 根) 根

后序遍历——左 根

(左 右 根) 根

王道考研/CSKAOYAN.COM

10

后序线索二叉树找后序后继

后序遍历序列: G D E B F C A

在后序线索二叉树中找到指定结点*p
的后序后继 next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$

p 必有右孩子

后序遍历——左 右 根

后序遍历中, 左右子树中的结点只可能是根的前驱, 不可能是后继

荒唐的答案

王道考研/CSKAOYAN.COM

11

后序线索二叉树找后序后继

改用三叉链表可以找到父节点

①如果能找到 p 的父节点, 且 p 是右孩子

后序遍历——左 右 根

左 (左 右 根) 根

p 的父节点即为其后继

②如果能找到 p 的父节点, 且 p 是左孩子, 其右兄弟为空

后序遍历——左 根

(左 右 根) 根

p 的父节点即为其后继

③如果能找到 p 的父节点, 且 p 是左孩子, 其右兄弟非空

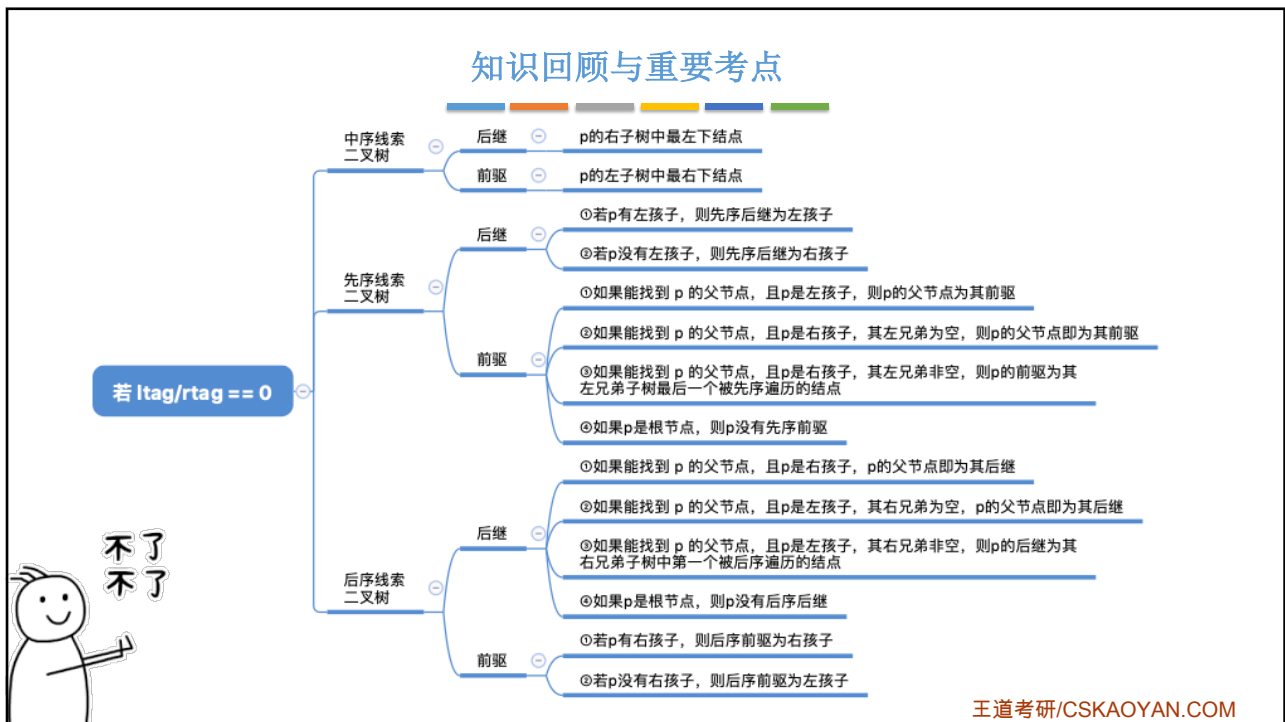
左 右 根

p 的后继为 右兄弟子树中第一个被后序遍历的结点

④如果 p 是根节点, 则 p 没有后序后继

王道考研/CSKAOYAN.COM

12



13



14

本节内容

树
存储结构

王道考研/CSKAOYAN.COM

1

知识总览

树的存储结构

树的逻辑结构回顾

双亲表示法

孩子表示法

孩子兄弟表示法

重要考点: 树、森林与二叉树的转换

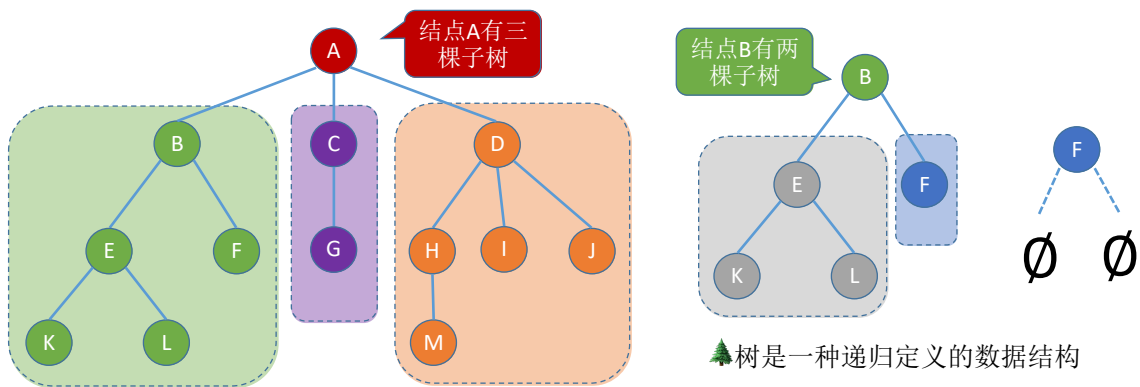
王道考研/CSKAOYAN.COM

2

树的逻辑结构

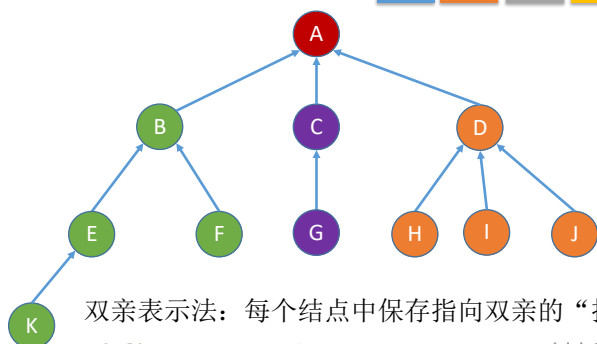
树是 n ($n \geq 0$) 个结点的有限集合, $n = 0$ 时, 称为空树, 这是一种特殊情况。在任意一棵非空树中应满足:

- 1) 有且仅有一个特定的称为**根**的结点。
- 2) 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个**互不相交的有限集合** $T_1, T_2, \dots, T_m$, 其中每个集合本身又是一棵树, 并且称为根结点的**子树**。



3

双亲表示法（顺序存储）



双亲表示法：每个结点中保存指向双亲的“指针”

```
#define MAX_TREE_SIZE 100 // 树中最多结点数
typedef struct {           // 树的结点定义
    ElemType data;        // 数据元素
    int parent;           // 双亲位置域
} PTNode;
typedef struct {           // 树的类型定义
    PTNode nodes[MAX_TREE_SIZE]; // 双亲表示
    int n;                // 结点数
} PTree;
```

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

根节点固定存储在0, -1表示没有双亲

王道考研/CSKAOYAN.COM

4

双亲表示法（顺序存储）

双亲表示法：每个结点中保存指向双亲的“指针”

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11	M	7
12	L	4
13		

新增数据元素，
无需按逻辑上的
次序存储

王道考研/CSKAOYAN.COM

5

双亲表示法（顺序存储）

双亲表示法：每个结点中保存指向双亲的“指针”

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6		-1
7	H	3
8	I	3
9	J	3
10	K	4
11	M	7
12	L	4
13		

删除数据元素
(方案一)

新增数据元素，
无需按逻辑上的
次序存储

王道考研/CSKAOYAN.COM

6

双亲表示法 (顺序存储)

双亲表示法: 每个结点中保存指向双亲的“指针”

```
typedef struct{
    PTreeNode nodes[MAX_TREE_SIZE];
    int n;
}PTree;
```

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	L	4
7	H	3
8	I	3
9	J	3
10	K	4
11	M	7
12		
13		

删除数据元素 (方案二)

如果删除的不是叶子结点呢?

思考人生

王道考研/CSKAOYAN.COM

7

双亲表示法 (顺序存储)

双亲表示法: 每个结点中保存指向双亲的“指针”

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6		-1
7	H	3
8	I	3
9	J	3
10	K	4
11	M	7
12	L	4
13		

优点: 查指定结点的双亲很方便

空数据导致遍历更慢

缺点: 查指定结点的孩子只能从头遍历

王道考研/CSKAOYAN.COM

8

回顾：二叉树的顺序存储

★ 二叉树的顺序存储中，一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 $2i$
- i 的右孩子 $2i+1$
- i 的父节点 $\lfloor i/2 \rfloor$

结点编号不仅反映了存储位置，也隐含了结点之间的逻辑关系

王道考研/CSKAOYAN.COM

9

孩子表示法（顺序+链式存储）

	data	*firstChild
0	A	1
1	B	4
2	C	6
3	D	7
4	E	10
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^

孩子表示法：顺序存储各个节点，每个结点中保存孩子链表头指针

王道考研/CSKAOYAN.COM

10

孩子表示法 (顺序+链式存储)

```
struct CTNode {
    int child; //孩子结点在数组中的位置
    struct CTNode *next; //下一个孩子
};
typedef struct {
    ElemType data;
    struct CTNode *firstChild; //第一个孩子
} CTBox;
typedef struct {
    CTBox nodes[MAX_TREE_SIZE];
    int n, r; //结点数和根的位置
} CTree;
```

	data	*firstChild
0	A	1 → 2 → 3 ^
1	B	4 → 5 ^
2	C	6 ^
3	D	7 → 8 → 9 ^
4	E	10 ^
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^

指向第一个孩子

如何实现增/删/查

思考人生

王道考研/CSKAOYAN.COM

11

孩子兄弟表示法 (链式存储)

```
//二叉树的结点 (链式存储)
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;

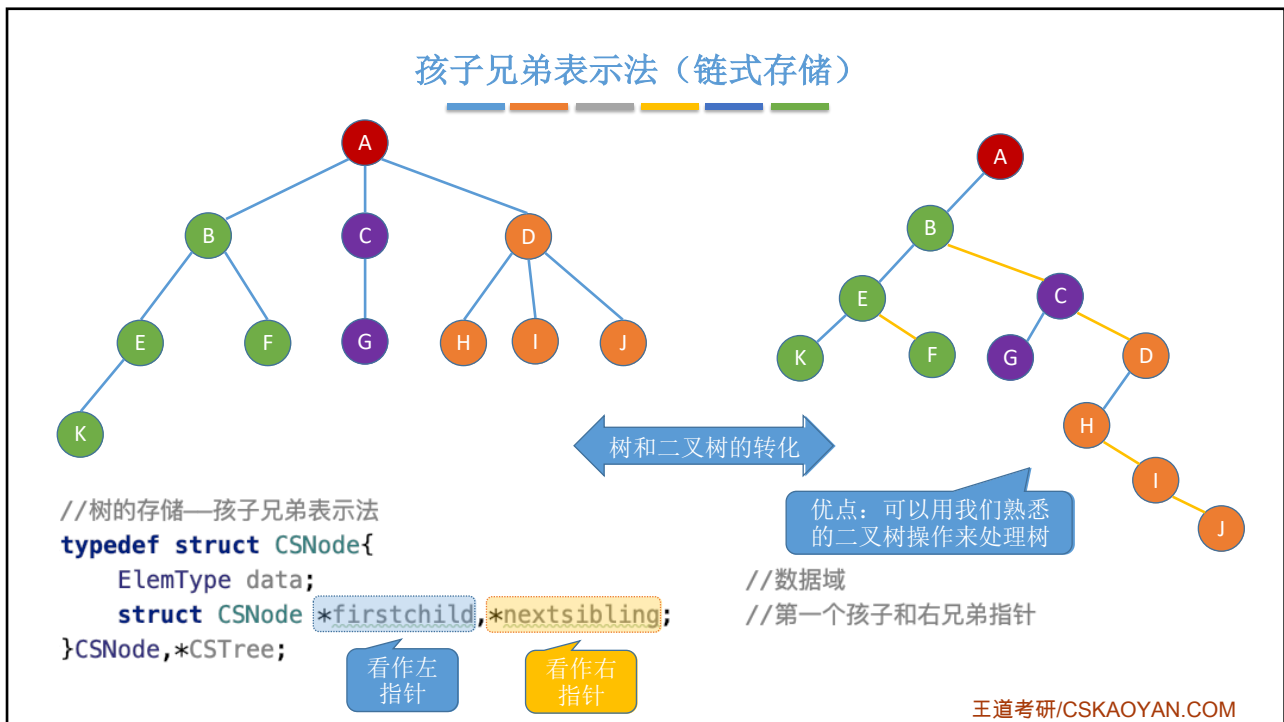
//树的存储—孩子兄弟表示法
typedef struct CSNode{
    ElemType data;
    struct CSNode *firstchild,*nextsibling;
}CSNode,*CSTree;
```

二叉链表

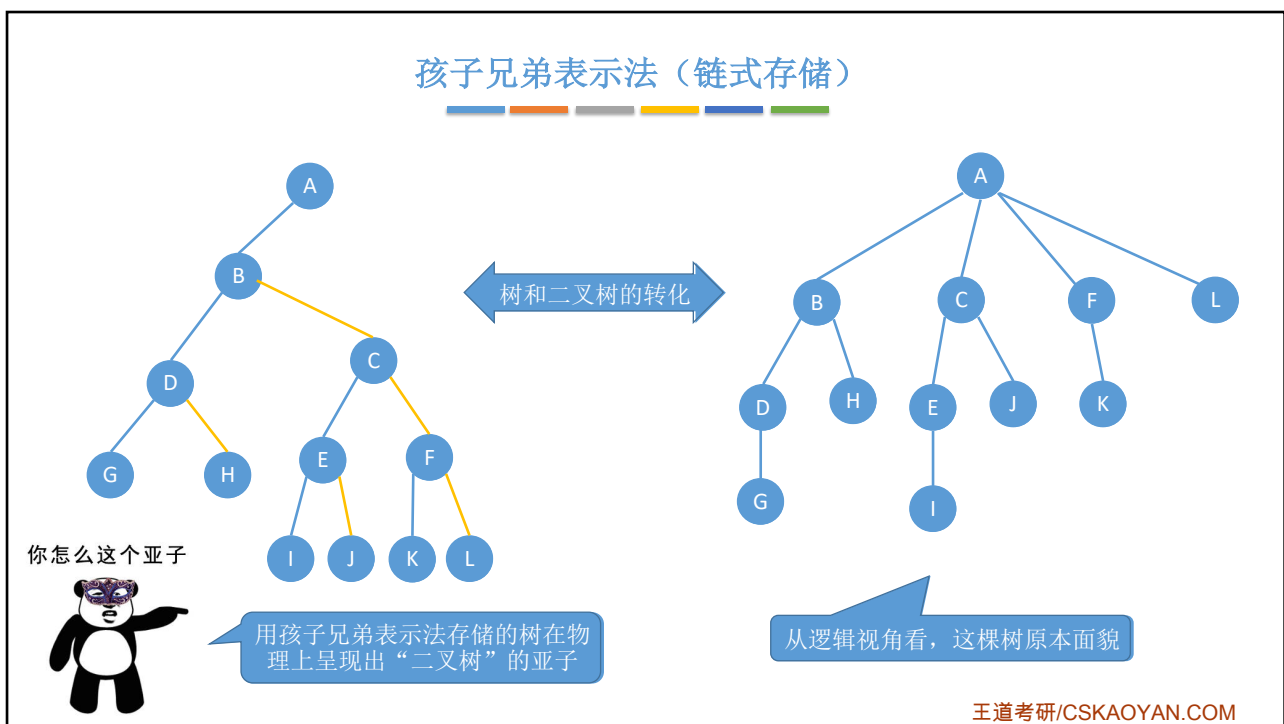
//数据域
//第一个孩子和右兄弟指针

王道考研/CSKAOYAN.COM

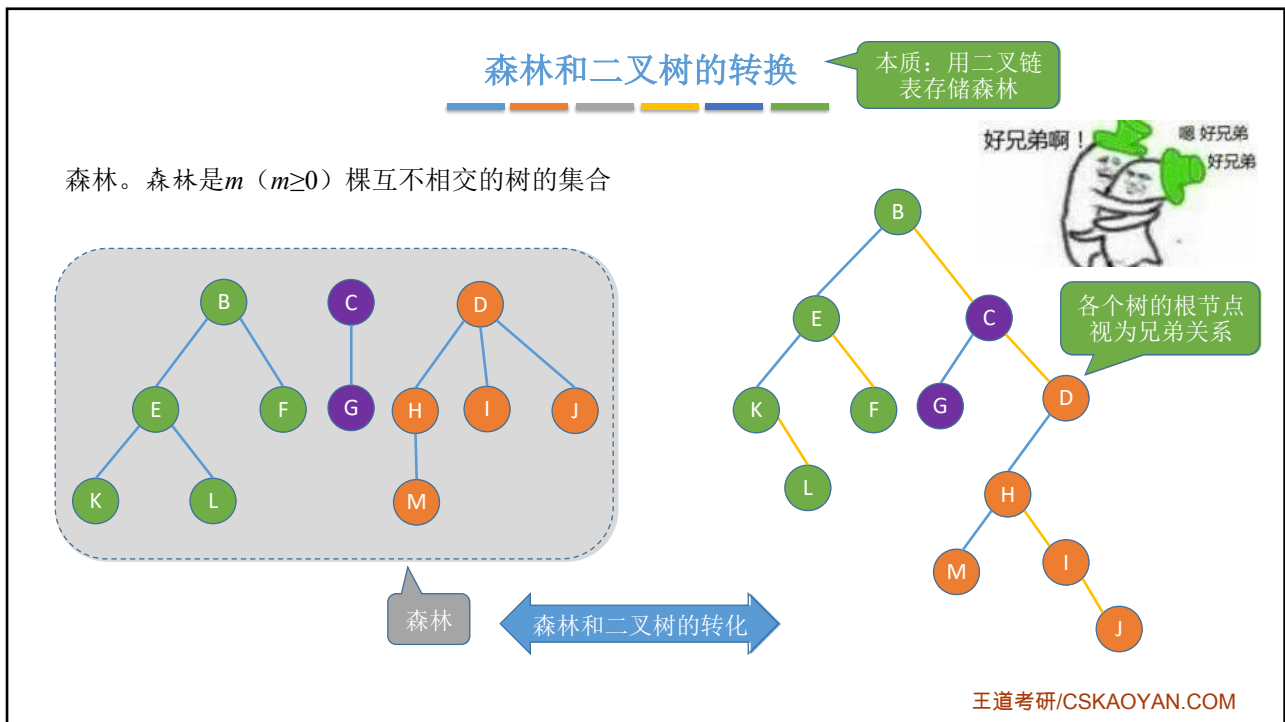
12



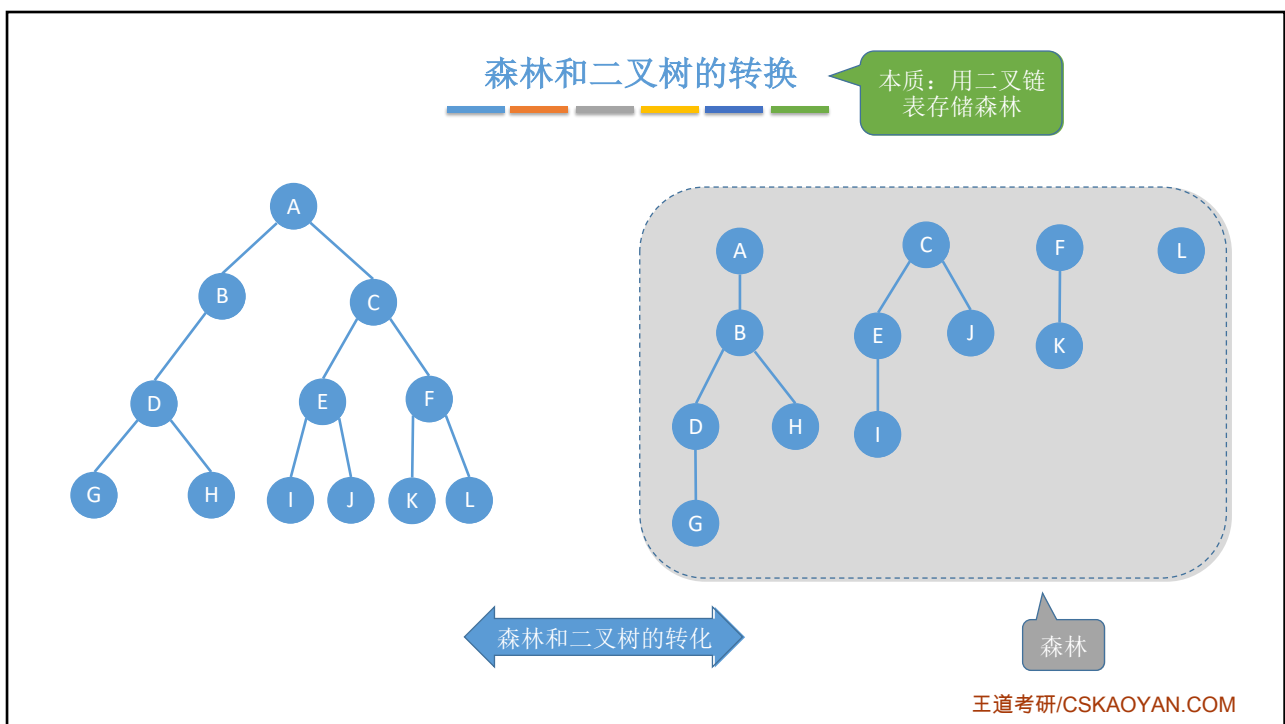
13



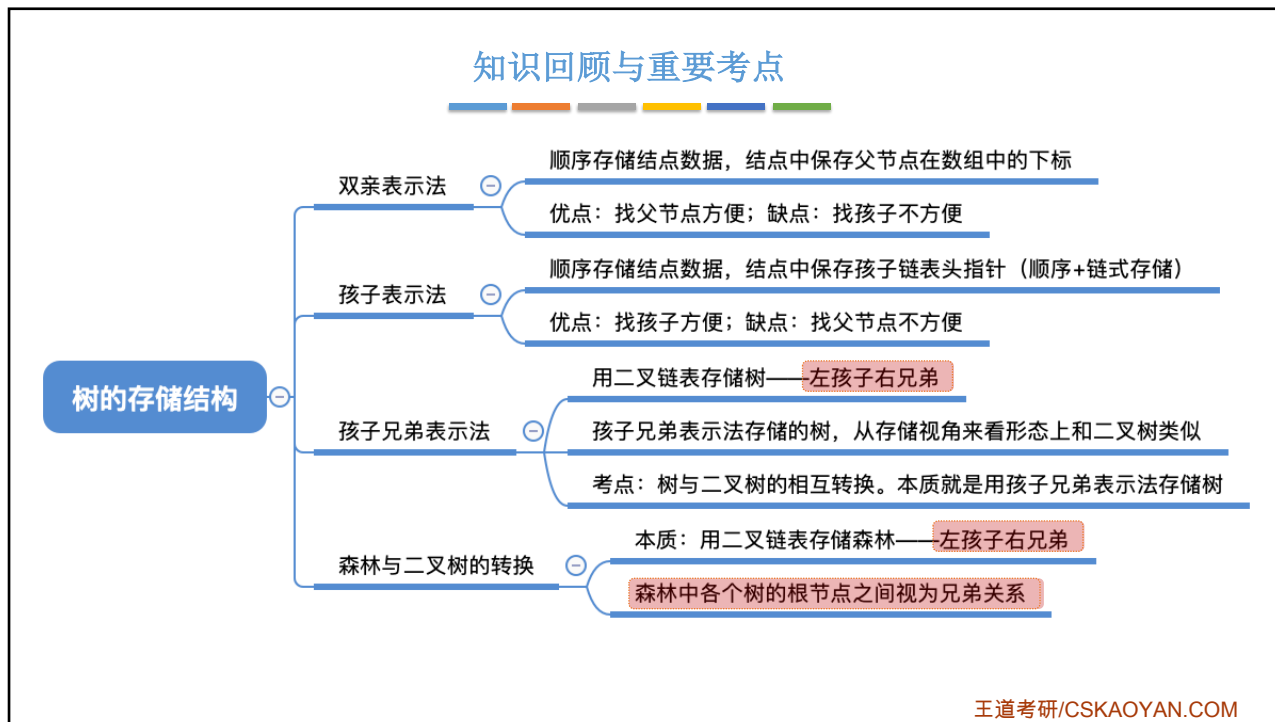
14



15



16



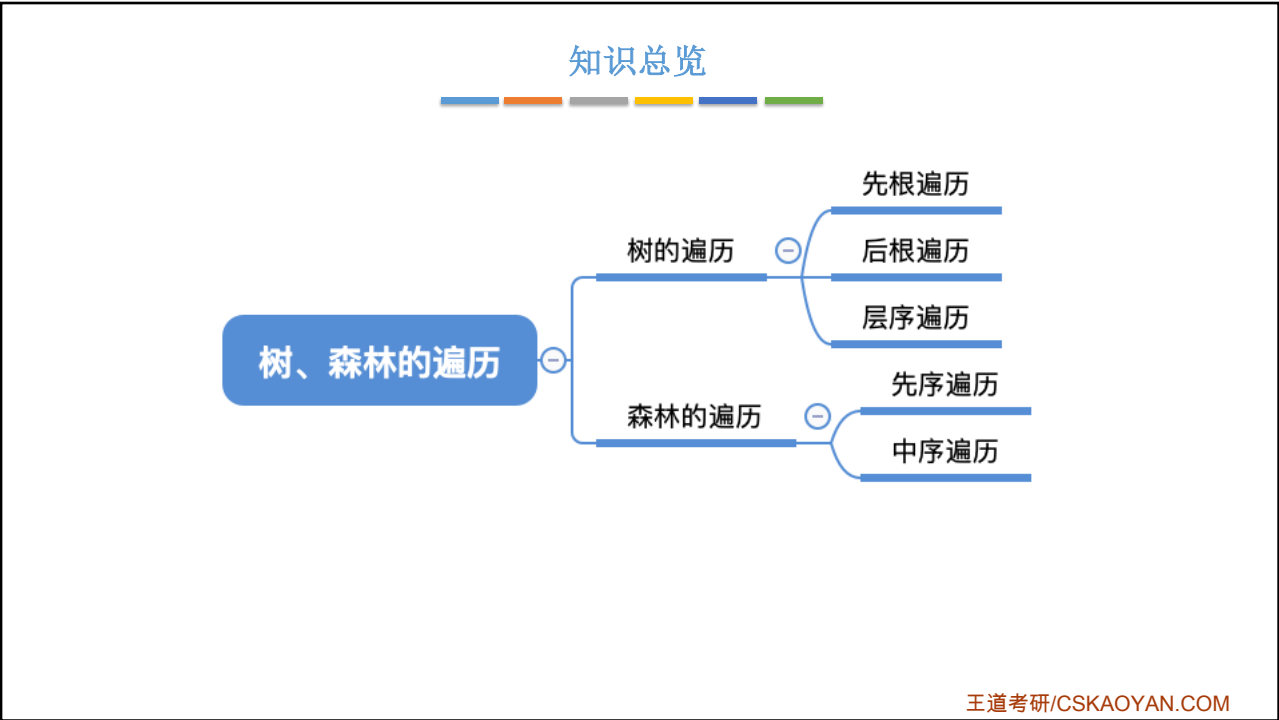
17

本节内容

树、森林
的遍历

王道考研/CSKAOYAN.COM

1

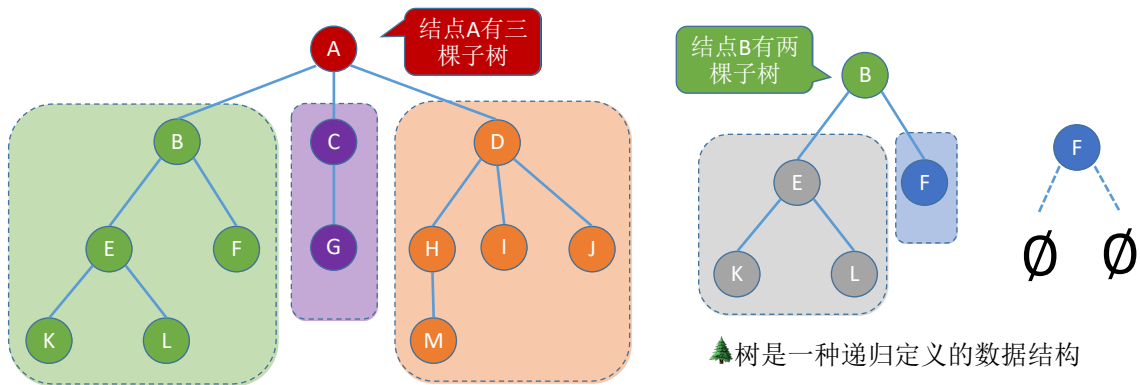


2

树的逻辑结构

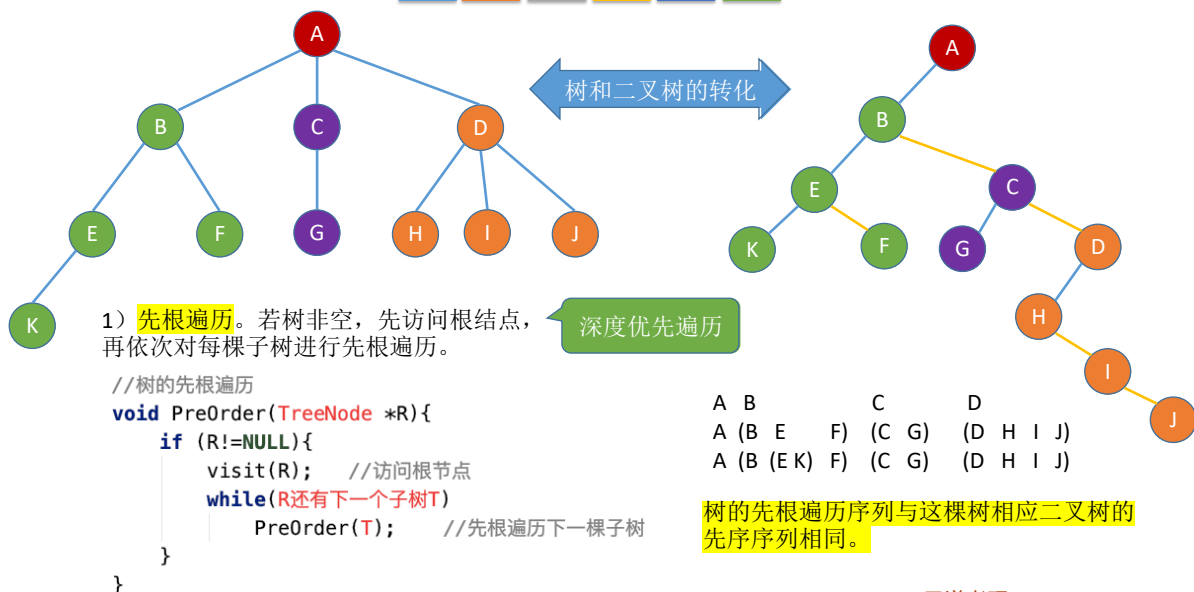
树是 n ($n \geq 0$) 个结点的有限集合, $n = 0$ 时, 称为空树, 这是一种特殊情况。在任意一棵非空树中应满足:

- 1) 有且仅有一个特定的称为根的结点。
- 2) 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个互不相交的有限集合 $T_1, T_2, \dots, T_m$, 其中每个集合本身又是一棵树, 并且称为根结点的子树。



3

树的先根遍历



4

树的后根遍历

树和二叉树的转化

2) **后根遍历**。若树非空, 先依次对每棵子树进行后根遍历, 最后再访问根结点。

深度优先遍历

```
//树的后根遍历
void PostOrder(TreeNode *R){
    if (R!=NULL){
        while(R还有下一个子树T)
            PostOrder(T); //后根遍历下一棵子树
        visit(R); //访问根节点
    }
}
```

B C D A
(E F B) (G C) (H I J D) A
((K E) F B) (G C) (H I J D) A

树的后根遍历序列与这棵树相应二叉树的中序序列相同。

王道考研/CSKAOYAN.COM

5

树的层次遍历

广度优先遍历

3) **层次遍历** (用队列实现)

- ①若树非空, 则根节点入队
- ②若队列非空, 队头元素出队并访问, 同时将该元素的孩子依次入队
- ③重复②直到队列为空

← 出队

A
B
C
D
E
F
G
H
I
J
K

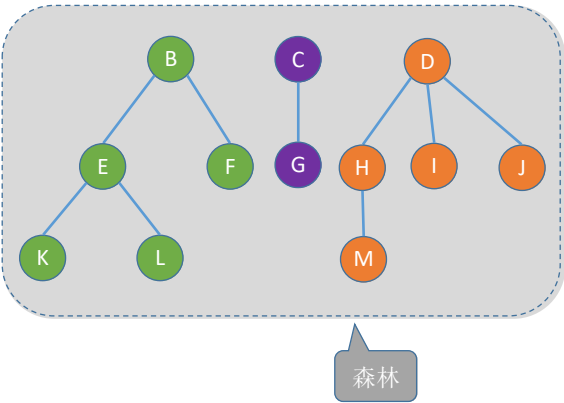
← 入队

王道考研/CSKAOYAN.COM

6

森林的先序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



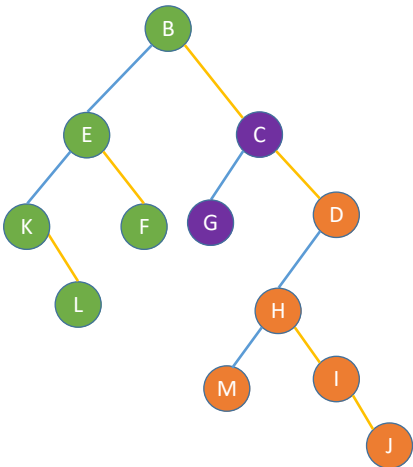
- 1) 先序遍历森林。
若森林为非空，则按如下规则进行遍历：
访问森林中第一棵树的根结点。
先序遍历第一棵树中根结点的子树森林。
先序遍历除去第一棵树之后剩余的树构成的森林。

B		C		D							
(B	E	F)	(C	G)	(D	H	I	J)			
(B	(E	K	L)	F)	(C	G)	(D	(H	M)	I	J)

效果等同于依次对各个树进行先根遍历

森林的先序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



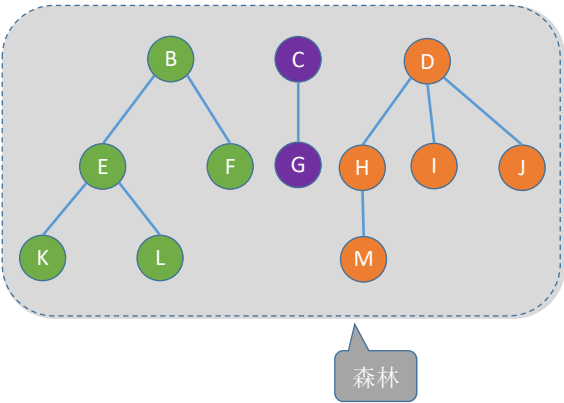
- 1) 先序遍历森林。
若森林为非空，则按如下规则进行遍历：
访问森林中第一棵树的根结点。
先序遍历第一棵树中根结点的子树森林。
先序遍历除去第一棵树之后剩余的树构成的森林。

B		C		D							
(B	E	F)	(C	G)	(D	H	I	J)			
(B	(E	K	L)	F)	(C	G)	(D	(H	M)	I	J)

效果等同于依次对二叉树的先序遍历

森林的中序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



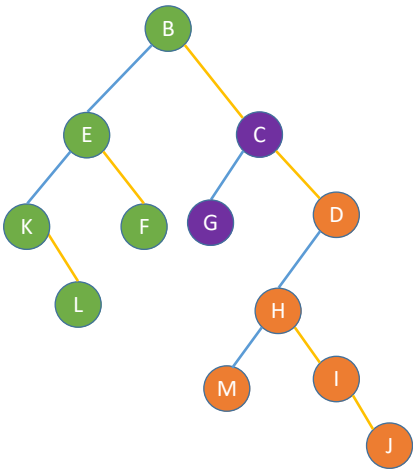
2) 中序遍历森林。
若森林为非空，则按如下规则进行遍历：
中序遍历森林中第一棵树的根结点的子树森林。
访问第一棵树的根结点。
中序遍历除去第一棵树之后剩余的树构成的森林。

```
      B      C      D
(   E F B) (G C) (  H I J D)
((K L E) F B) (G C) ((M H) I J D)
```

效果等同于依次对各个树进行后根遍历

森林的中序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



2) 中序遍历森林。
若森林为非空，则按如下规则进行遍历：
中序遍历森林中第一棵树的根结点的子树森林。
访问第一棵树的根结点。
中序遍历除去第一棵树之后剩余的树构成的森林。

```
      B      C      D
(   E F B) (G C) (  H I J D)
((K L E) F B) (G C) ((M H) I J D)
```

效果等同于依次对二叉树的中序遍历

知识回顾与重要考点

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

王道考研/CSKAOYAN.COM

本节内容

二叉排序树
(BST)

王道考研/CSKAOYAN.COM

1

知识总览

二叉排序树

二叉排序树的定义

查找操作

插入操作

删除操作

查找效率分析

王道考研/CSKAOYAN.COM

2

二叉排序树的定义

二叉排序树可用于元素的有序组织、搜索

二叉排序树, 又称二叉查找树 (BST, Binary Search Tree)
一棵二叉树或者是空二叉树, 或者是具有如下性质的二叉树:
左子树上所有结点的关键字均小于根结点的关键字;
右子树上所有结点的关键字均大于根结点的关键字。
左子树和右子树又各是一棵二叉排序树。

左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历, 可以得到一个递增的有序序列

王道考研/CSKAOYAN.COM

3

二叉排序树的查找

左子树结点值 < 根结点值 < 右子树结点值

若树非空, 目标值与根结点的值比较:
若相等, 则查找成功;
若小于根结点, 则在左子树上查找, 否则在右子树上查找。

查找成功, 返回结点指针; 查找失败返回NULL

例1: 查找关键字为30的结点

```
//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T, int key){
    while(T!=NULL && key!=T->key){
        if(key<T->key) T=T->lchild; //若树空或等于根结点值, 则结束循环
        else T=T->rchild; //小于, 则在左子树上查找
    }
    return T; //大于, 则在右子树上查找
}
```

```
//二叉排序树结点
typedef struct BSTNode{
    int key;
    struct BSTNode *lchild, *rchild;
}BSTNode, *BSTree;
```

王道考研/CSKAOYAN.COM

4

二叉排序树的查找

左子树结点值 < 根结点值 < 右子树结点值

若树非空, 目标值与根结点的值比较:
若相等, 则查找成功;
若小于根结点, 则在左子树上查找, 否则在右子树上查找。

查找成功, 返回结点指针; 查找失败返回NULL

例1: 查找关键字为30的结点

```
//二叉排序树结点
typedef struct BSTNode{
    int key;
    struct BSTNode *lchild,*rchild;
}BSTNode,*BSTree;

//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T,int key){
    while(T!=NULL&&key!=T->key){ //若树空或等于根结点值, 则结束循环
        if(key<T->key) T=T->lchild; //小于, 则在左子树上查找
        else T=T->rchild; //大于, 则在右子树上查找
    }
    return T;
}
```

王道考研/CSKAOYAN.COM

5

二叉排序树的查找

左子树结点值 < 根结点值 < 右子树结点值

若树非空, 目标值与根结点的值比较:
若相等, 则查找成功;
若小于根结点, 则在左子树上查找, 否则在右子树上查找。

查找成功, 返回结点指针; 查找失败返回NULL

例2: 查找关键字为12的结点

```
//二叉排序树结点
typedef struct BSTNode{
    int key;
    struct BSTNode *lchild,*rchild;
}BSTNode,*BSTree;

//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T,int key){
    while(T!=NULL&&key!=T->key){ //若树空或等于根结点值, 则结束循环
        if(key<T->key) T=T->lchild; //小于, 则在左子树上查找
        else T=T->rchild; //大于, 则在右子树上查找
    }
    return T;
}
```

王道考研/CSKAOYAN.COM

6

二叉排序树的查找

```
//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T,int key){
    while(T!=NULL&&key!=T->key){ //若树空或等于根结点值, 则结束循环
        if(key<T->key) T=T->lchild; //小于, 则在左子树上查找
        else T=T->rchild; //大于, 则在右子树上查找
    }
    return T;
}

//在二叉排序树中查找值为 key 的结点 (递归实现)
BSTNode *BSTSearch(BSTree T,int key){
    if (T==NULL)
        return NULL; //查找失败
    if (key==T->key)
        return T; //查找成功
    else if (key < T->key)
        return BSTSearch(T->lchild, key); //在左子树中找
    else
        return BSTSearch(T->rchild, key); //在右子树中找
}
```

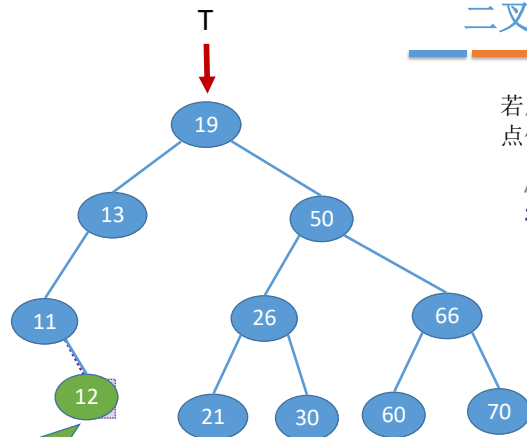
最坏空间复杂度 $O(1)$

最坏空间复杂度 $O(h)$

王道考研/CSKAOYAN.COM

7

二叉排序树的插入



例: 插入关键字为12的结点



嗨嗨, 醒醒,
敲代码了!

练习: 实现非
递归插入

若原二叉排序树为空, 则直接插入结点; 否则, 若关键字 k 小于根结点值, 则插入到左子树, 若关键字 k 大于根结点值, 则插入到右子树

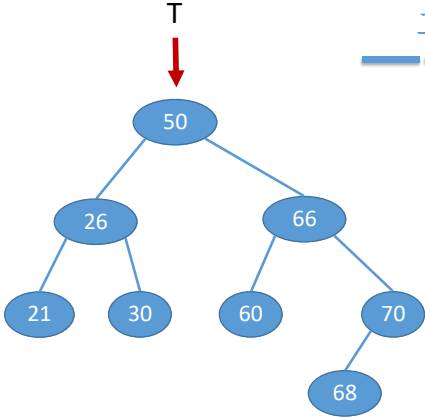
//在二叉排序树插入关键字为 k 的新结点 (递归实现) 最坏空间复杂度 $O(h)$

```
int BST_Insert(BSTree T, int k){
    if(T==NULL){ //原树为空, 新插入的结点为根结点
        T=(BSTree)malloc(sizeof(BSTNode));
        T->key=k;
        T->lchild=T->rchild=NULL;
        return 1; //返回1, 插入成功
    }
    else if(k==T->key) //树中存在相同关键字的结点, 插入失败
        return 0;
    else if(k<T->key) //插入到T的左子树
        return BST_Insert(T->lchild,k);
    else //插入到T的右子树
        return BST_Insert(T->rchild,k);
}
```

王道考研/CSKAOYAN.COM

8

二叉排序树的构造



```

//按照 str[] 中的关键字序列建立二叉排序树
void Creat_BST(BSTree &T,int str[],int n){
    T=NULL;           //初始时T为空树
    int i=0;
    while(i<n){        //依次将每个关键字插入到二叉排序树中
        BST_Insert(T,str[i]);
        i++;
    }
}
    
```

例1: 按照序列str={50, 66, 60, 26, 21, 30, 70, 68}建立BST

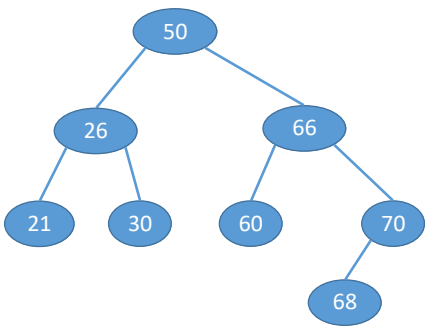
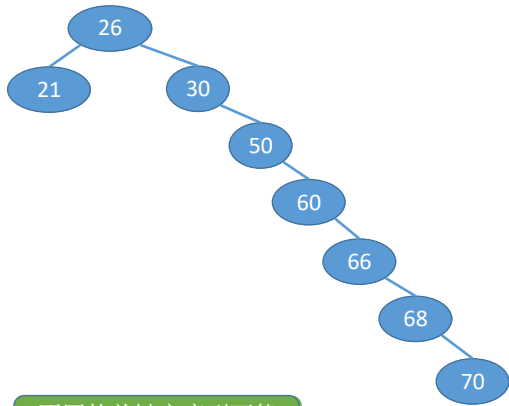
例2: 按照序列str={50, 26, 21, 30, 66, 60, 70, 68}建立BST

不同的关键字序列可能得到同款二叉排序树

王道考研/CSKAOYAN.COM

9

二叉排序树的构造

例1: 按照序列str={50, 66, 60, 26, 21, 30, 70, 68}建立BST

例2: 按照序列str={50, 26, 21, 30, 66, 60, 70, 68}建立BST

例3: 按照序列str={26, 21, 30, 50, 60, 66, 68, 70}建立BST

不同的关键字序列可能得到同款二叉排序树

也可能得到不同款二叉排序树

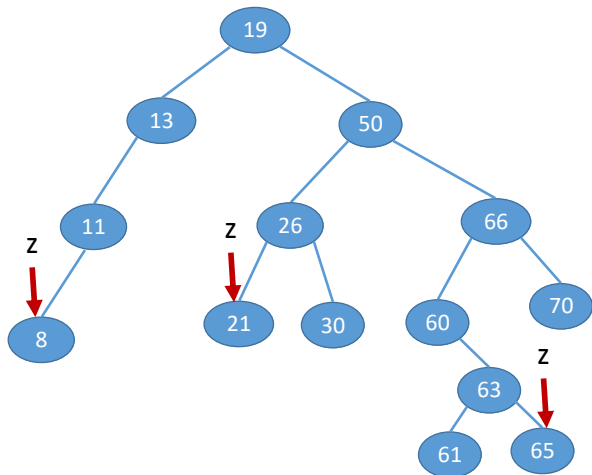
王道考研/CSKAOYAN.COM

10

二叉排序树的删除

先搜索找到目标结点:

① 若被删除结点 z 是叶结点, 则直接删除, 不会破坏二叉排序树的性质。



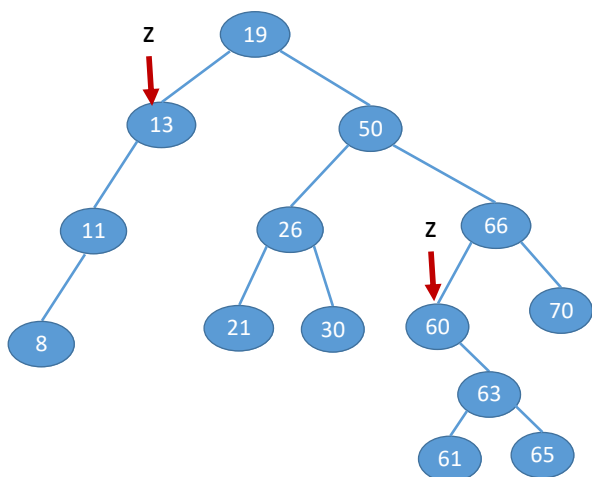
左子树结点值 < 根结点值 < 右子树结点值

王道考研/CSKAOYAN.COM

11

二叉排序树的删除

② 若结点 z 只有一棵左子树或右子树, 则让 z 的子树成为 z 父结点的子树, 替代 z 的位置。



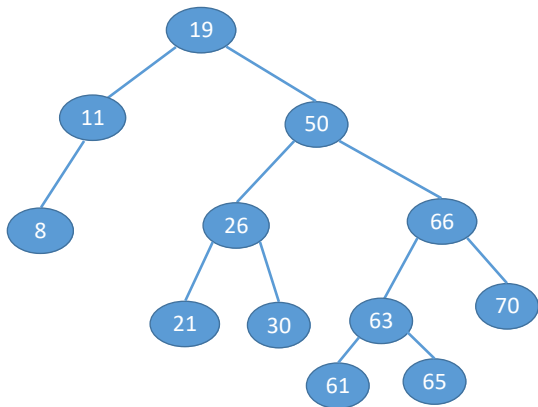
左子树结点值 < 根结点值 < 右子树结点值

王道考研/CSKAOYAN.COM

12

二叉排序树的删除

② 若结点 z 只有一棵左子树或右子树, 则让 z 的子树成为 z 父结点的子树, 替代 z 的位置。



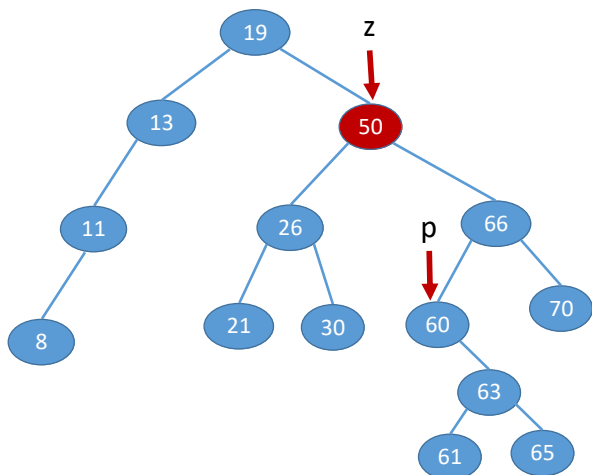
左子树结点值 < 根结点值 < 右子树结点值

王道考研/CSKAOYAN.COM

13

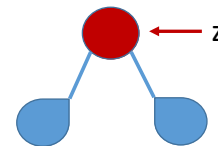
二叉排序树的删除

③ 若结点 z 有左、右两棵子树, 则令 z 的直接后继(或直接前驱)替代 z , 然后从二叉排序树中删去这个直接后继(或直接前驱), 这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历, 可以得到一个递增的有序序列



中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

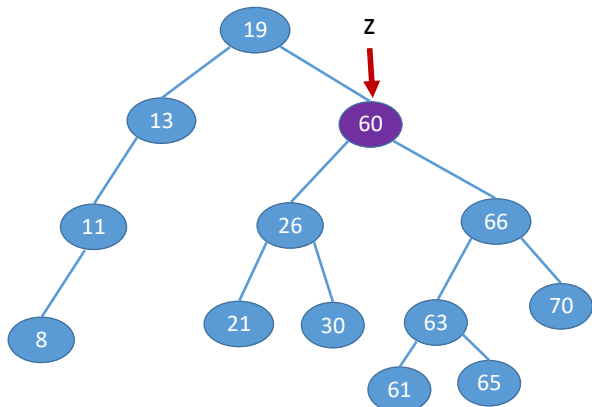
z 的后继: z 的右子树中最左下结点(该结点一定没有左子树)

王道考研/CSKAOYAN.COM

14

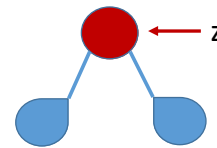
二叉排序树的删除

③ 若结点 z 有左、右两棵子树, 则令 z 的直接后继(或直接前驱)替代 z , 然后从二叉排序树中删去这个直接后继(或直接前驱), 这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历, 可以得到一个递增的有序序列



中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

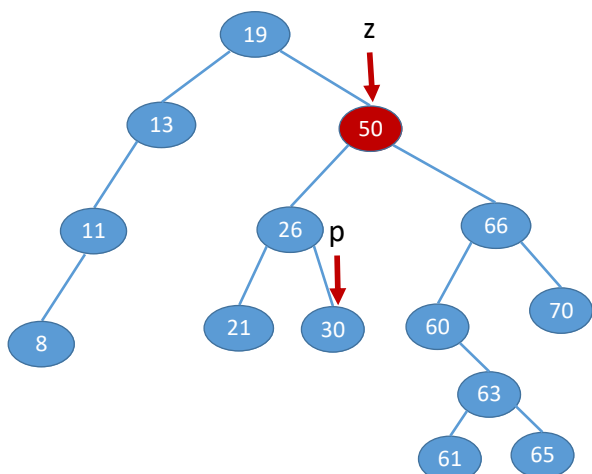
z 的后继: z 的右子树中最左下结点(该结点一定没有左子树)

王道考研/CSKAOYAN.COM

15

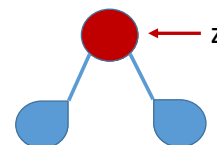
二叉排序树的删除

③ 若结点 z 有左、右两棵子树, 则令 z 的直接后继(或直接前驱)替代 z , 然后从二叉排序树中删去这个直接后继(或直接前驱), 这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历, 可以得到一个递增的有序序列



中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

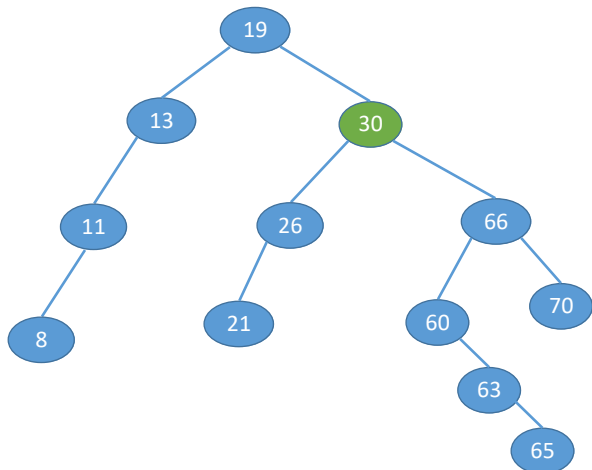
z 的前驱: z 的左子树中最右下结点(该结点一定没有右子树)

王道考研/CSKAOYAN.COM

16

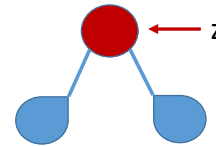
二叉排序树的删除

③ 若结点 z 有左、右两棵子树, 则令 z 的直接后继(或直接前驱)替代 z , 然后从二叉排序树中删去这个直接后继(或直接前驱), 这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历, 可以得到一个递增的有序序列



中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

z 的前驱: z 的左子树中最右下结点(该结点一定没有右子树)

王道考研/CSKAOYAN.COM

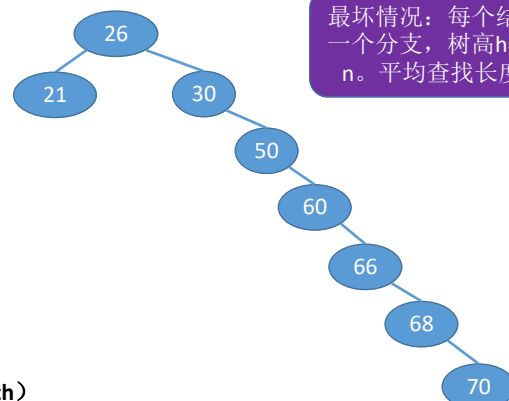
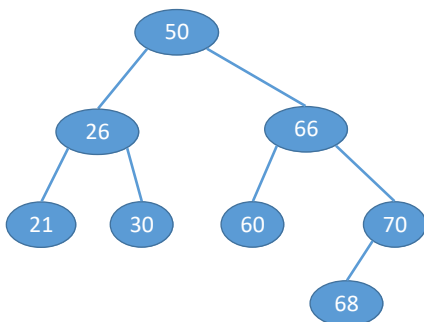
17

查找效率分析

若树高 h , 找到最下层的一个结点需要对比 h 次

最好情况: n 个结点的二叉树最小高度为 $\lfloor \log_2 n \rfloor + 1$ 。
平均查找长度 = $O(\log_2 n)$

查找长度——在查找运算中, 需要对比关键字的次数称为查找长度, 反映了查找操作时间复杂度



最坏情况: 每个结点只有一个分支, 树高 h =结点数 n 。平均查找长度= $O(n)$

查找成功的平均查找长度 ASL (Average Search Length)

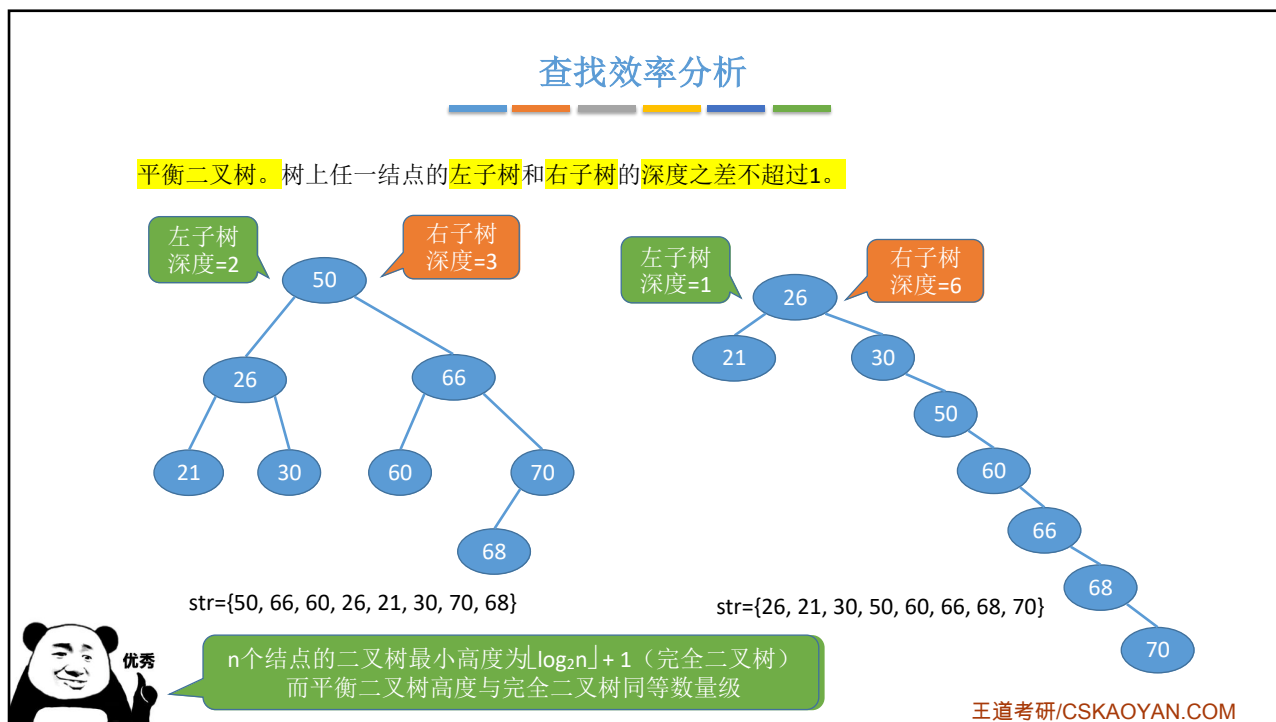
$$ASL = (1*1 + 2*2 + 3*4 + 4*1)/8 = 2.625$$

$$ASL = (1*1 + 2*2 + 3*1 + 4*1 + 5*1 + 6*1 + 7*1)/8 = 3.75$$

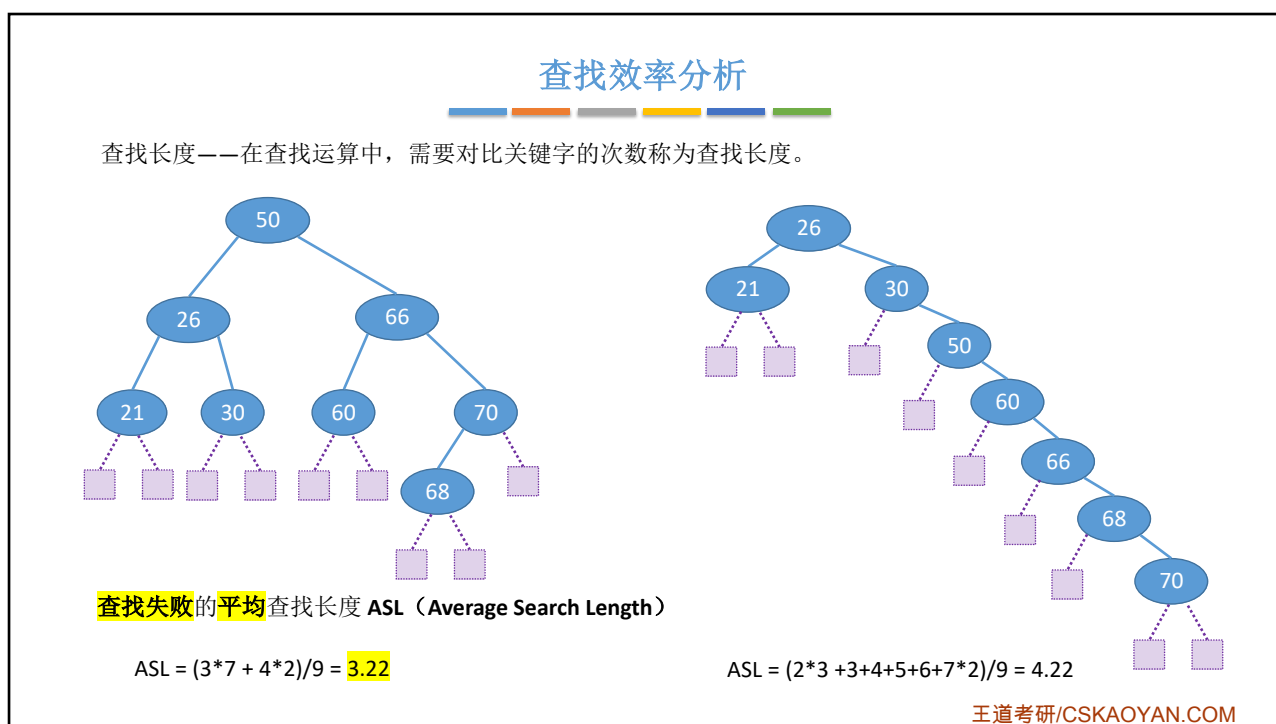


王道考研/CSKAOYAN.COM

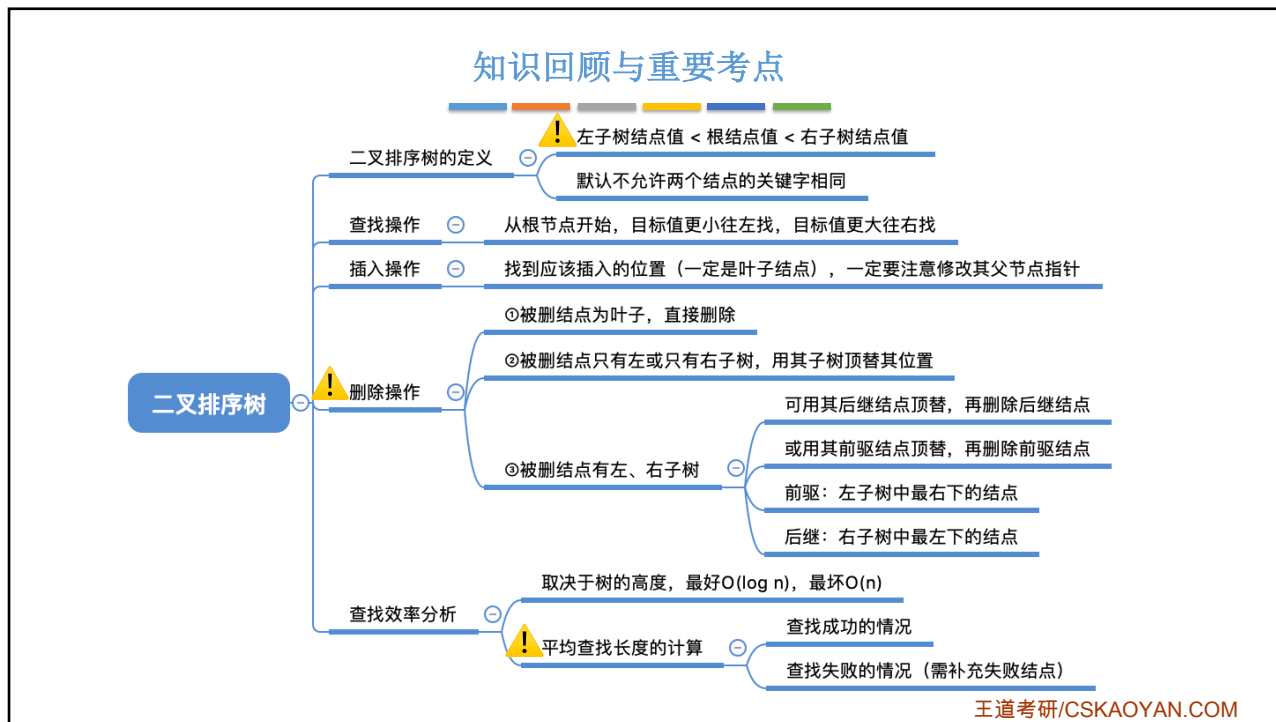
18



19



20



本节内容

平衡二叉树
(AVL)

王道考研/CSKAOYAN.COM

1

知识总览

平衡二叉树

定义

插入操作

插入新结点后如何调整“不平衡”问题

查找效率分析

王道考研/CSKAOYAN.COM

2

平衡二叉树的定义

G. M. Adelson-Velsky 和 E. M. Landis

平衡二叉树 (Balanced Binary Tree), 简称平衡树 (AVL树) —— 树上任一结点的左子树和右子树的高度之差不超过1。
 结点的平衡因子 = 左子树高 - 右子树高。

平衡二叉树结点的平衡因子的值只可能是-1、0或1。

只要有任一结点的平衡因子绝对值大于1, 就不是平衡二叉树

```
//平衡二叉树结点
typedef struct AVLNode{
    int key;           //数据域
    int balance;       //平衡因子
    struct AVLNode *lchild,*rchild;
}AVLNode,*AVLTree;
```

王道考研/CSKAOYAN.COM

3

平衡二叉树的插入

在二叉排序树中插入新结点后, 如何保持平衡?

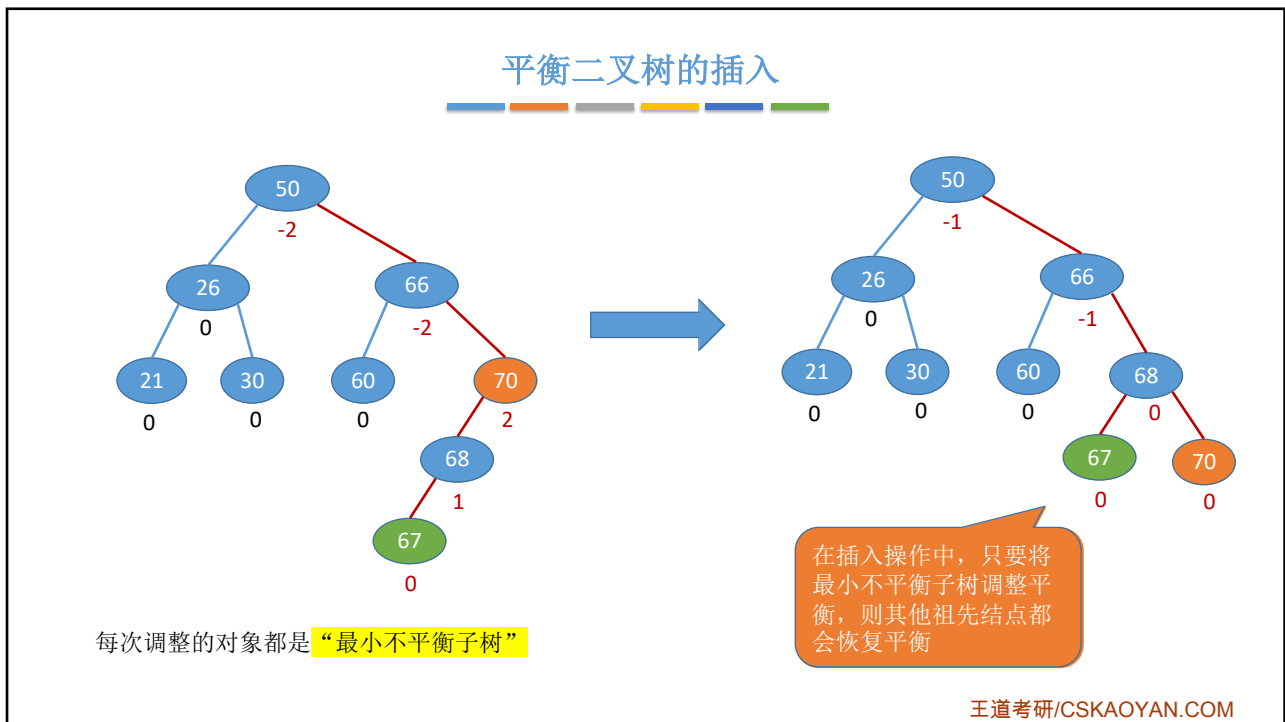
查找路径上的所有结点都有可能受到影响

从插入点往回找到第一个不平衡结点, 调整以该结点为根的子树

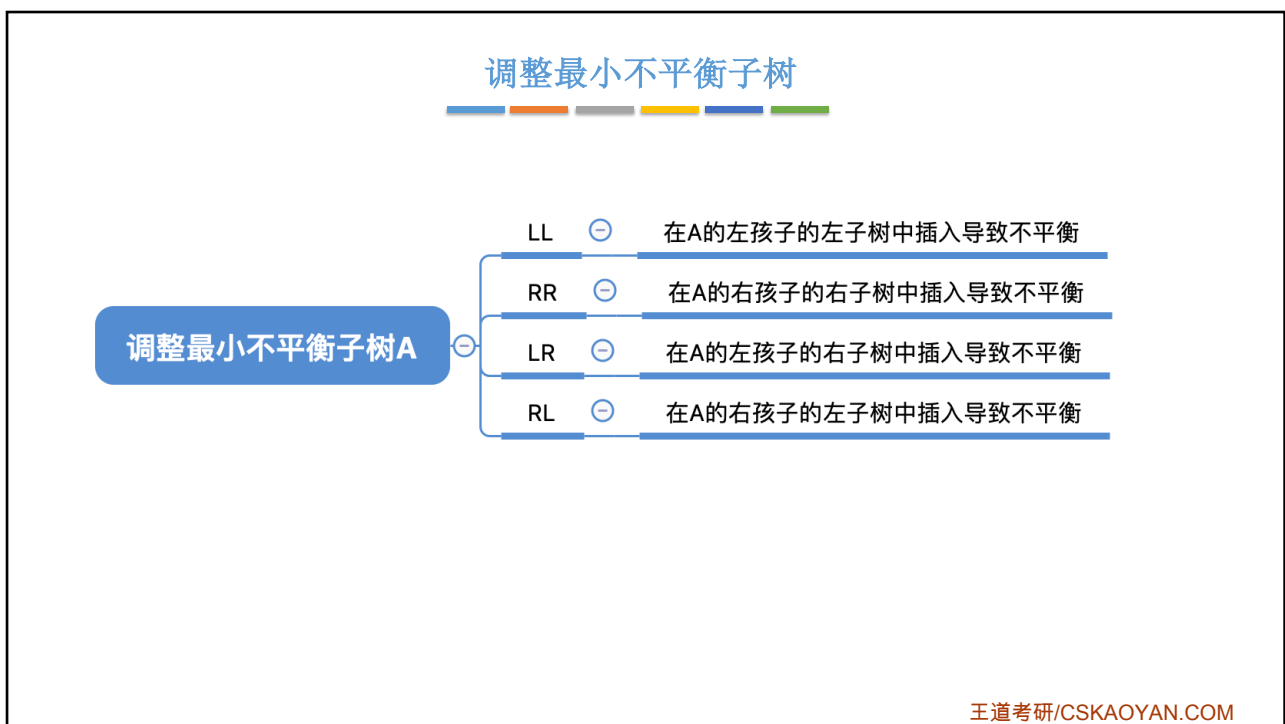
每次调整的对象都是“最小不平衡子树”

王道考研/CSKAOYAN.COM

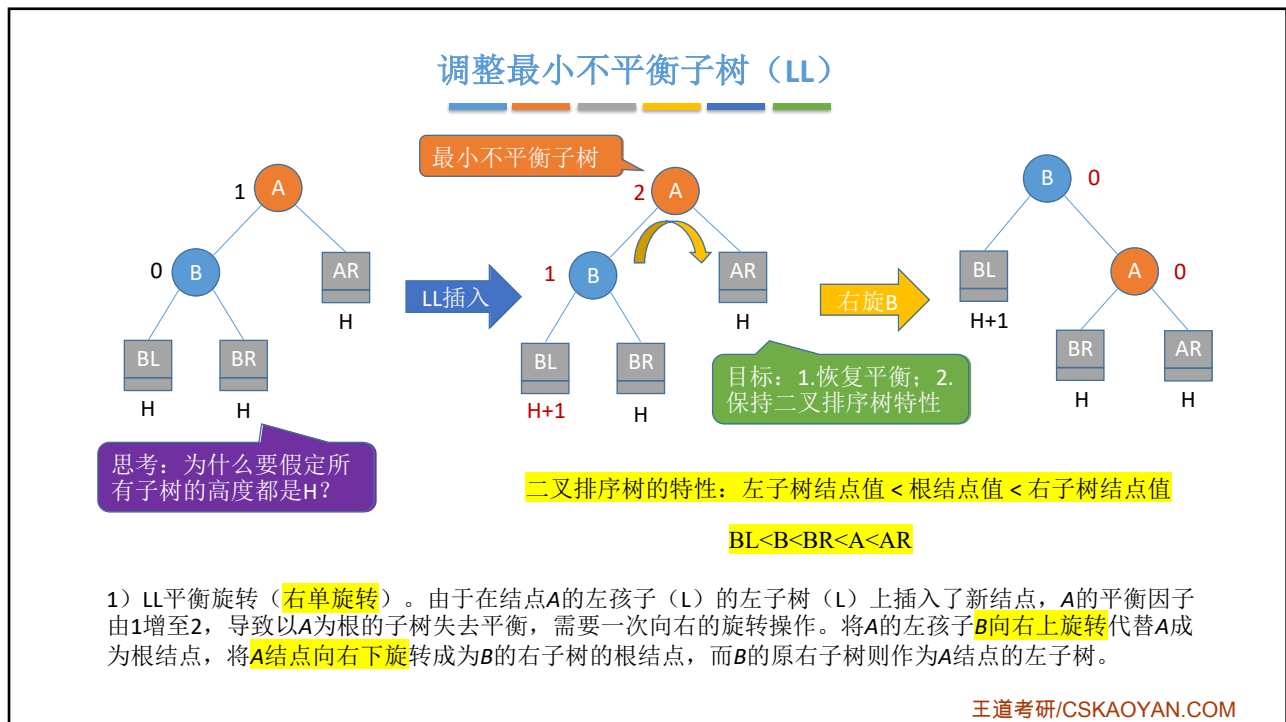
4



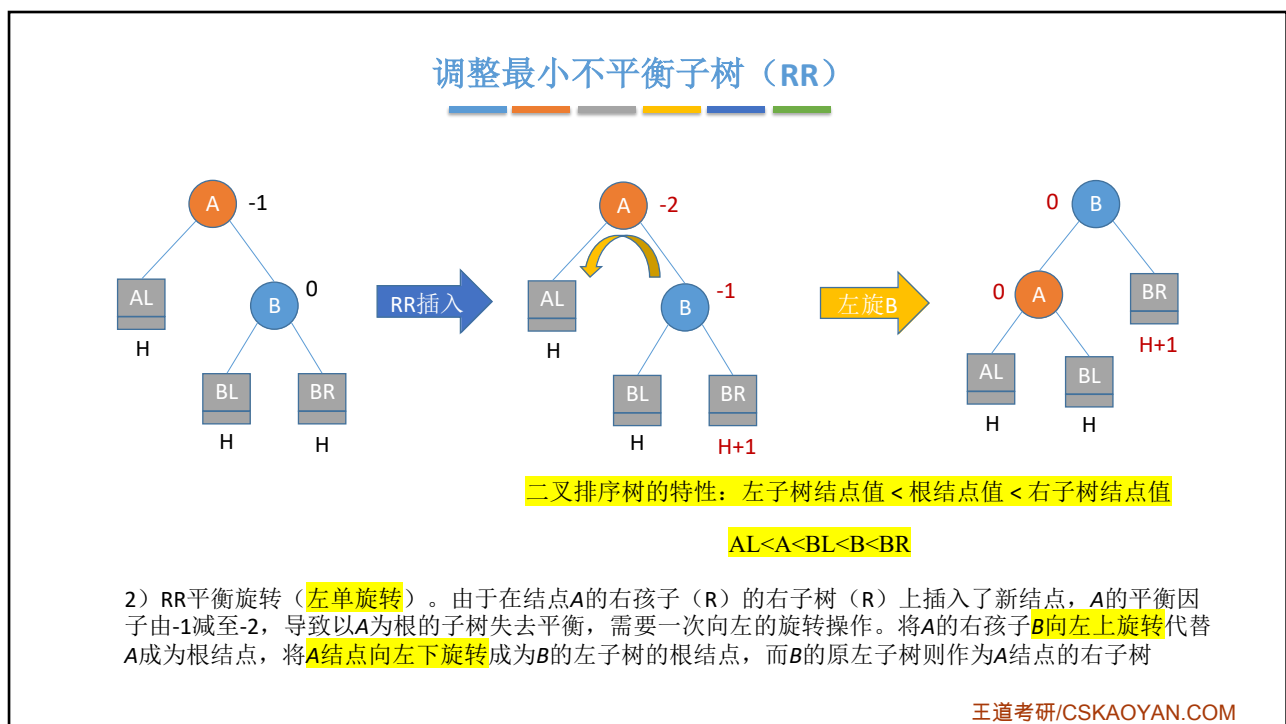
5



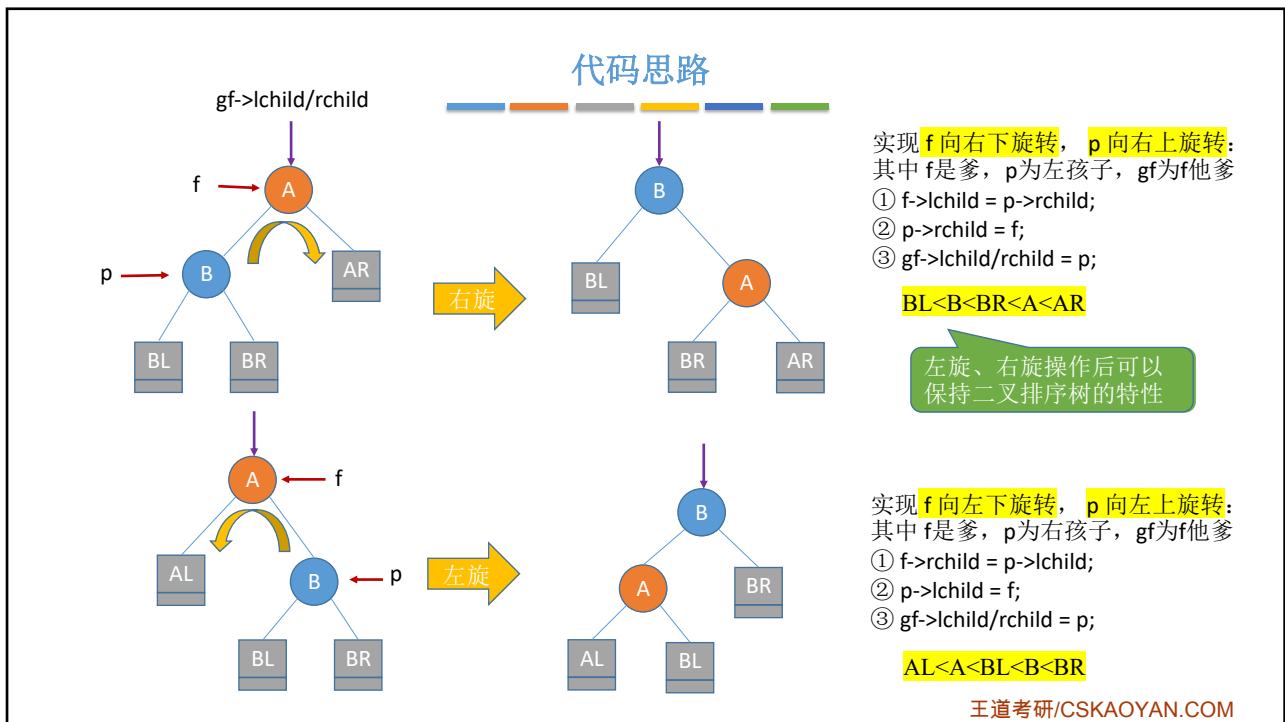
6



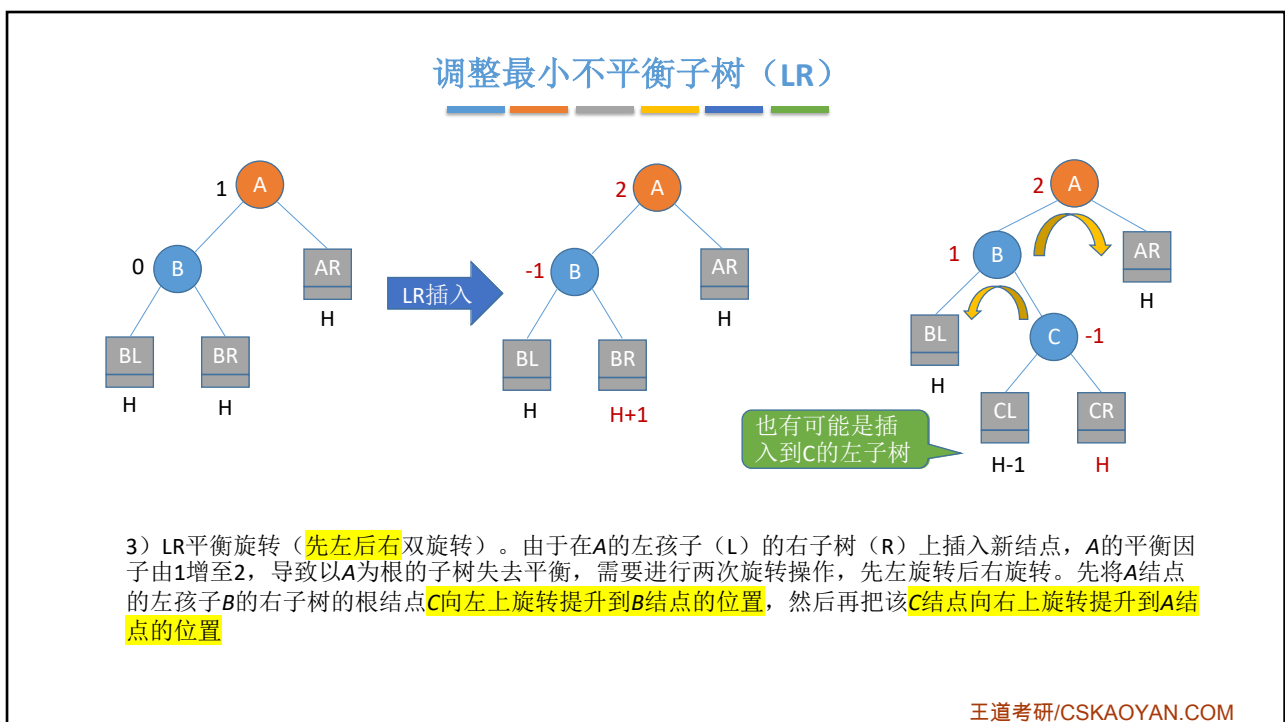
7



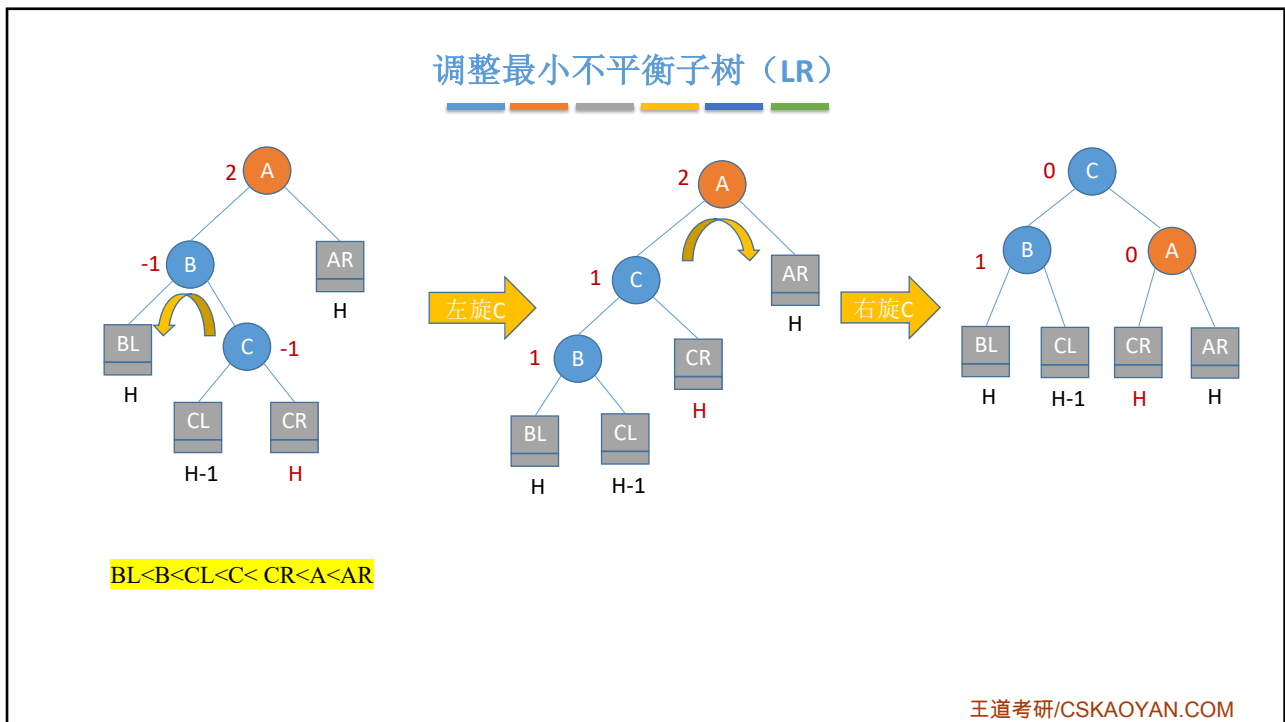
8



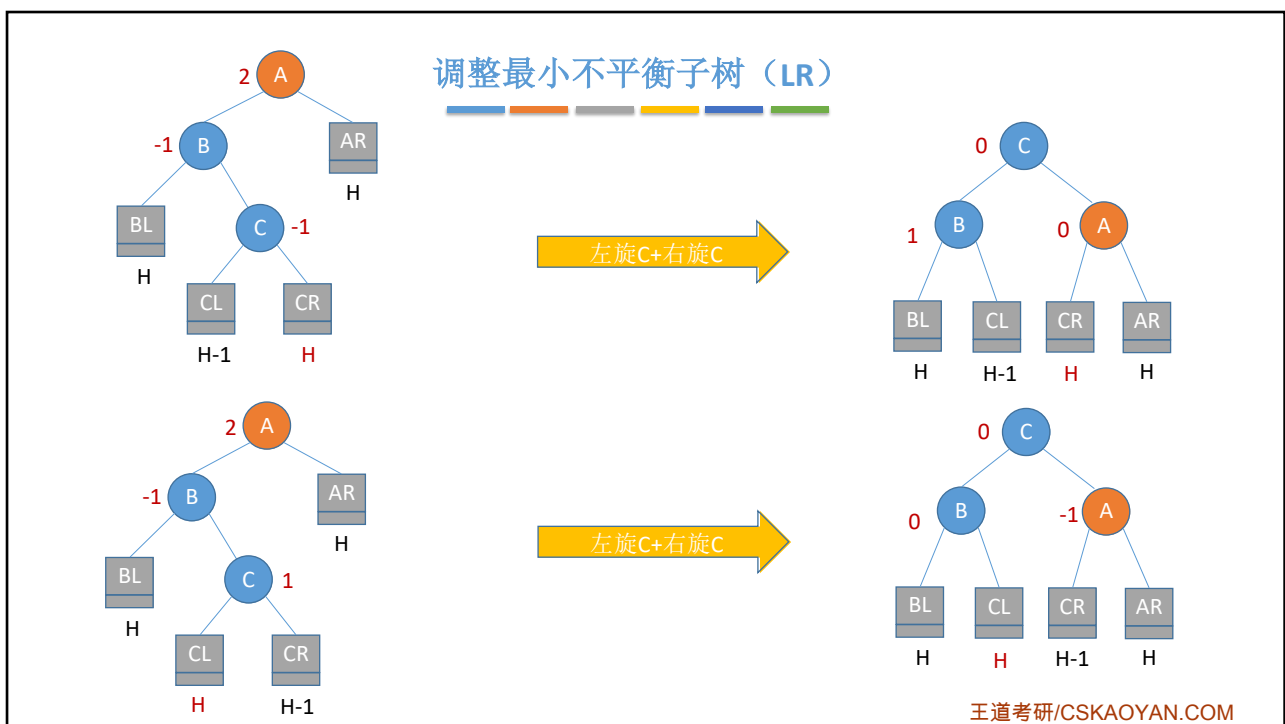
9



10



11



12

调整最小不平衡子树 (RL)

$AL < A < CL < C < CR < B < BR$

4) RL平衡旋转(先右后左双旋转)。由于在A的右孩子(R)的左子树(L)上插入新结点, A的平衡因子由-1减至-2, 导致以A为根的子树失去平衡, 需要进行两次旋转操作, 先右旋转后左旋转。先将A结点的右孩子B的左子树的根结点C向右上旋转提升到B结点的位置, 然后再把该C结点向左上旋转提升到A结点的位置

王道考研/CSKAOYAN.COM

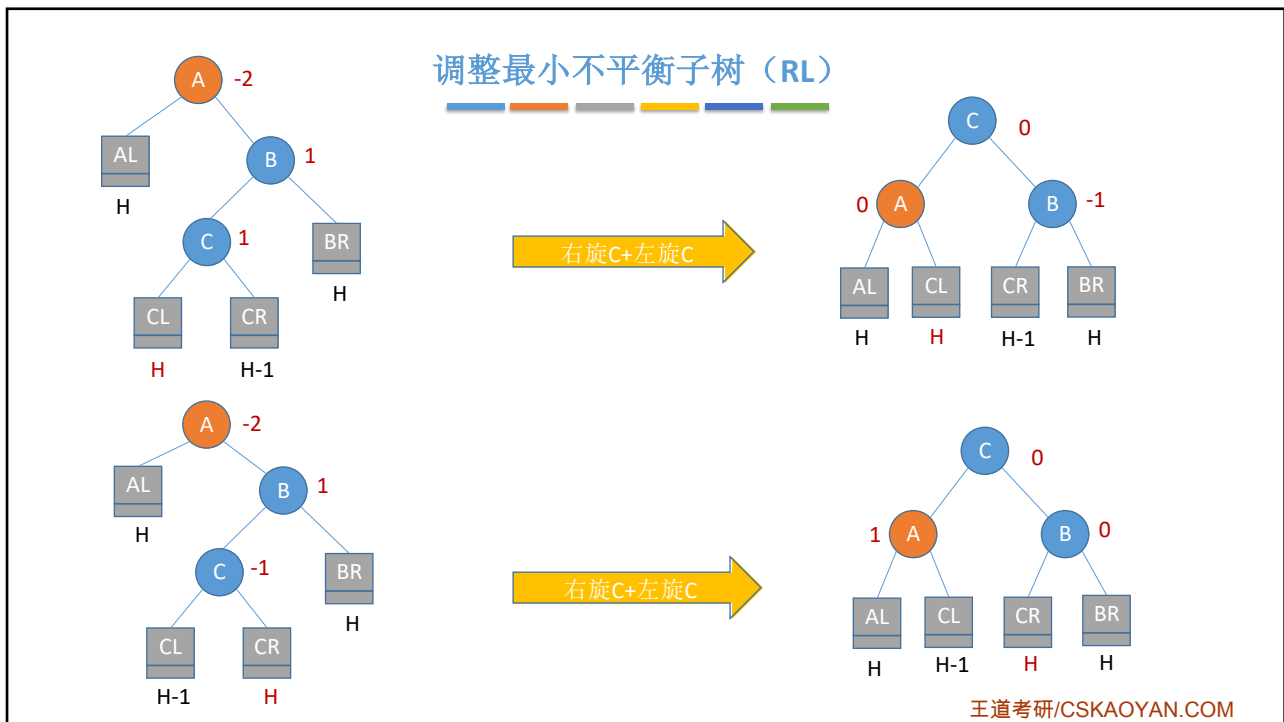
13

调整最小不平衡子树 (RL)

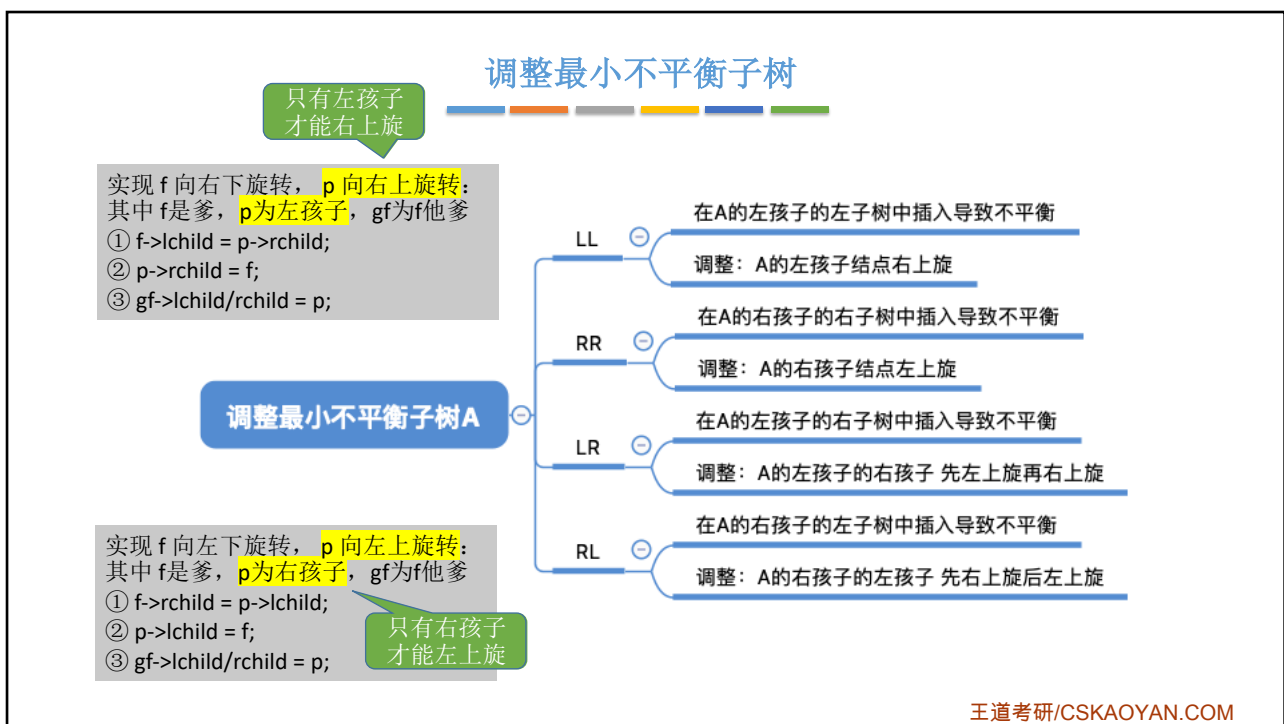
$AL < A < CL < C < CR < B < BR$

王道考研/CSKAOYAN.COM

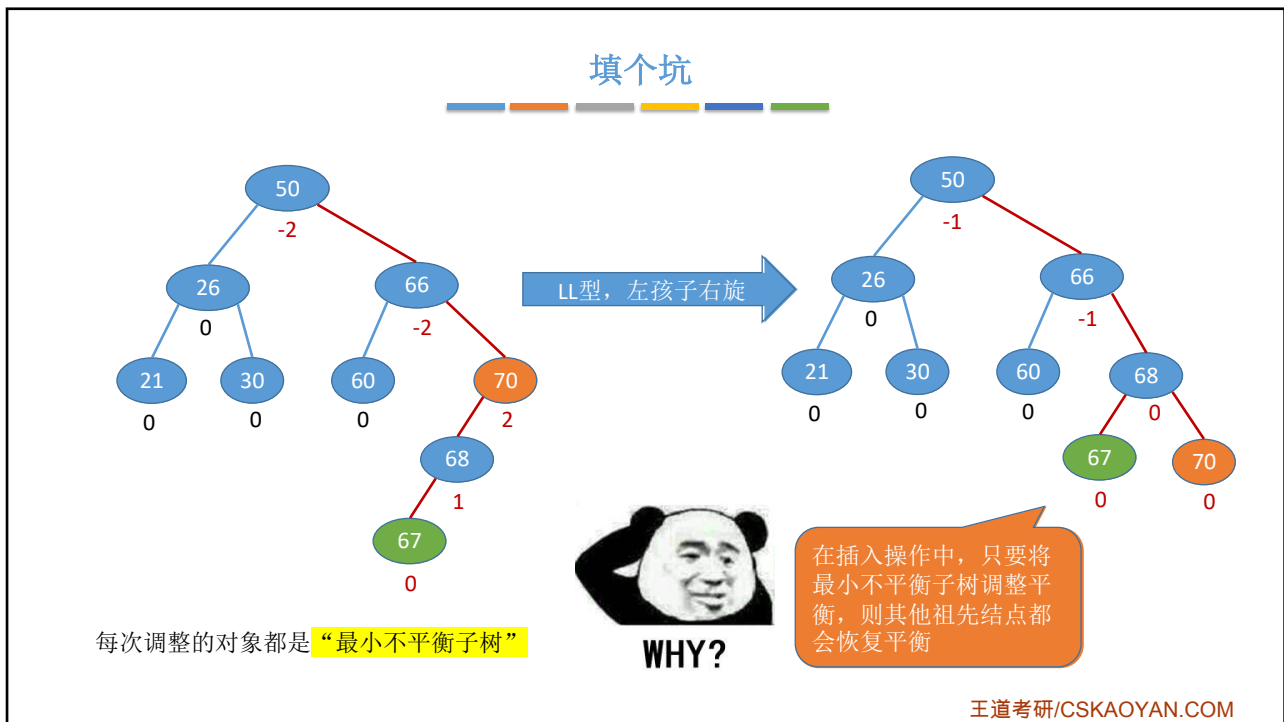
14



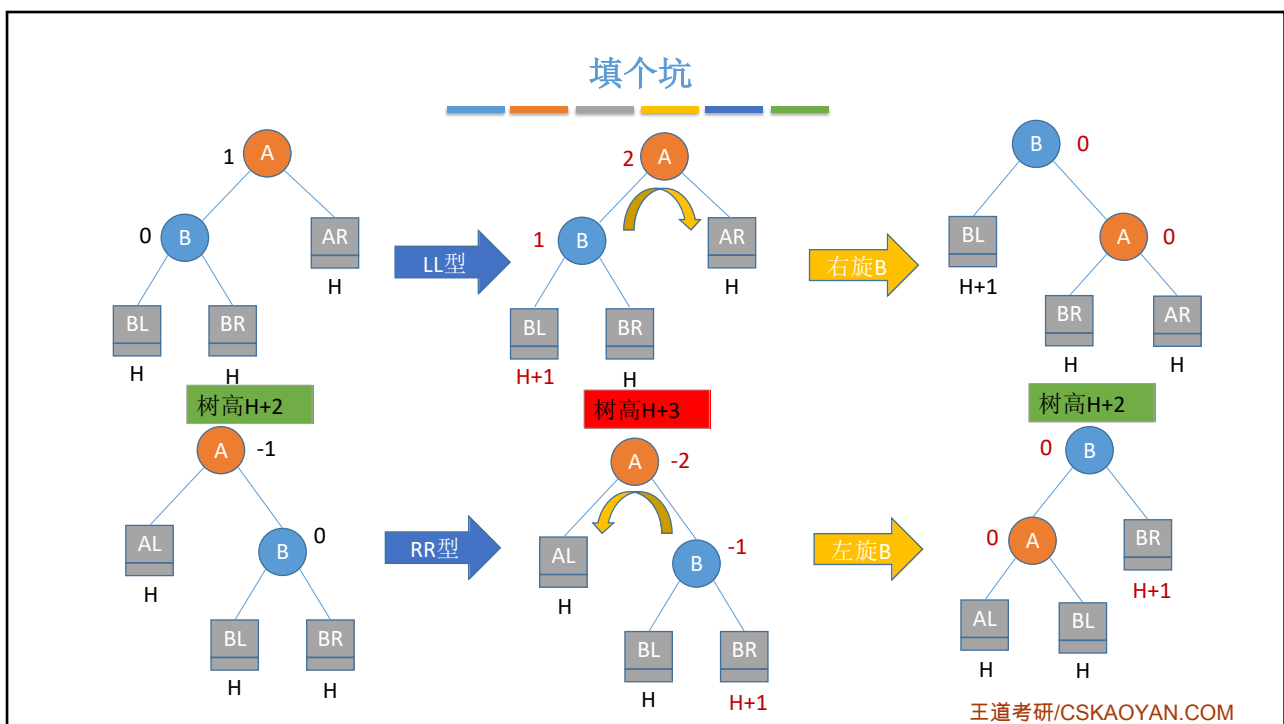
15



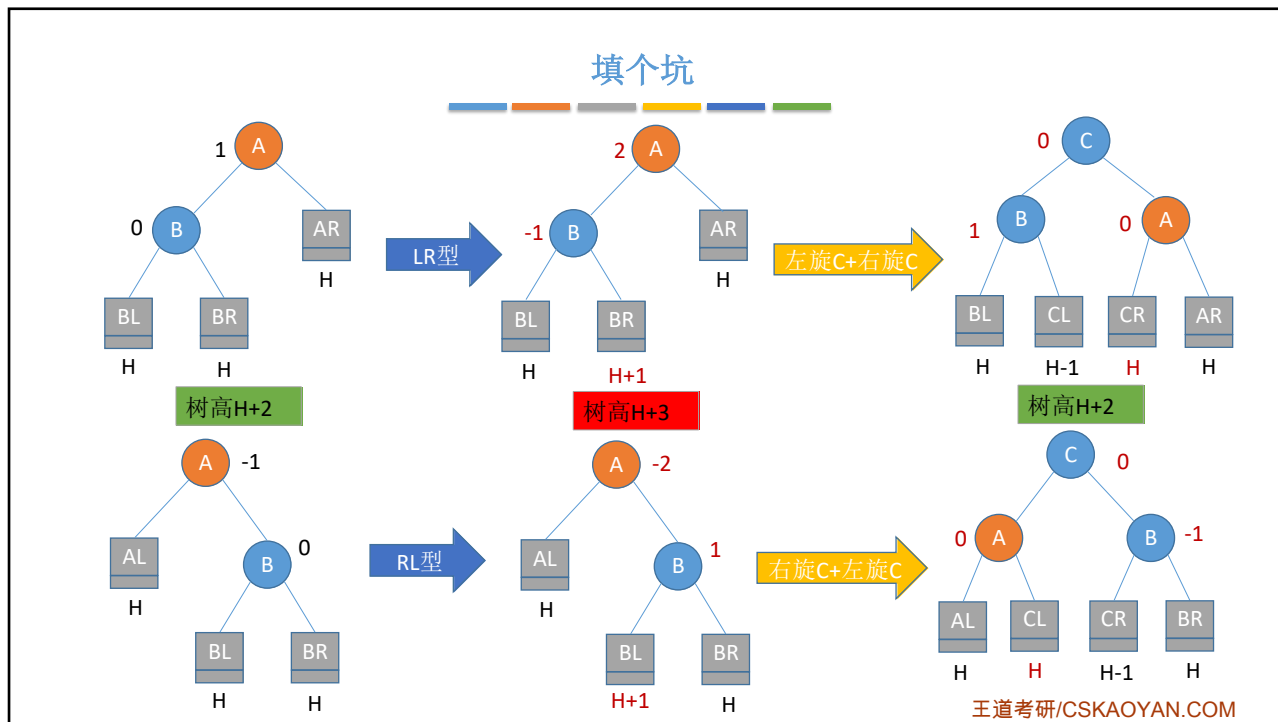
16



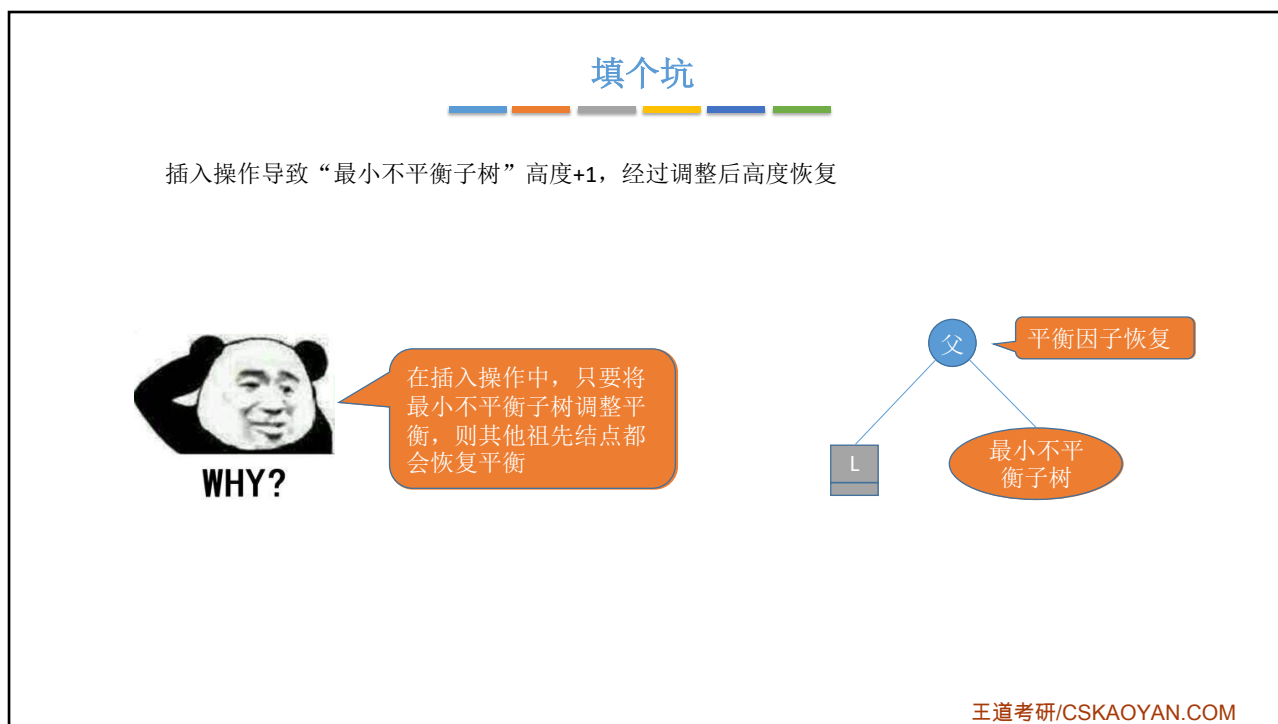
17



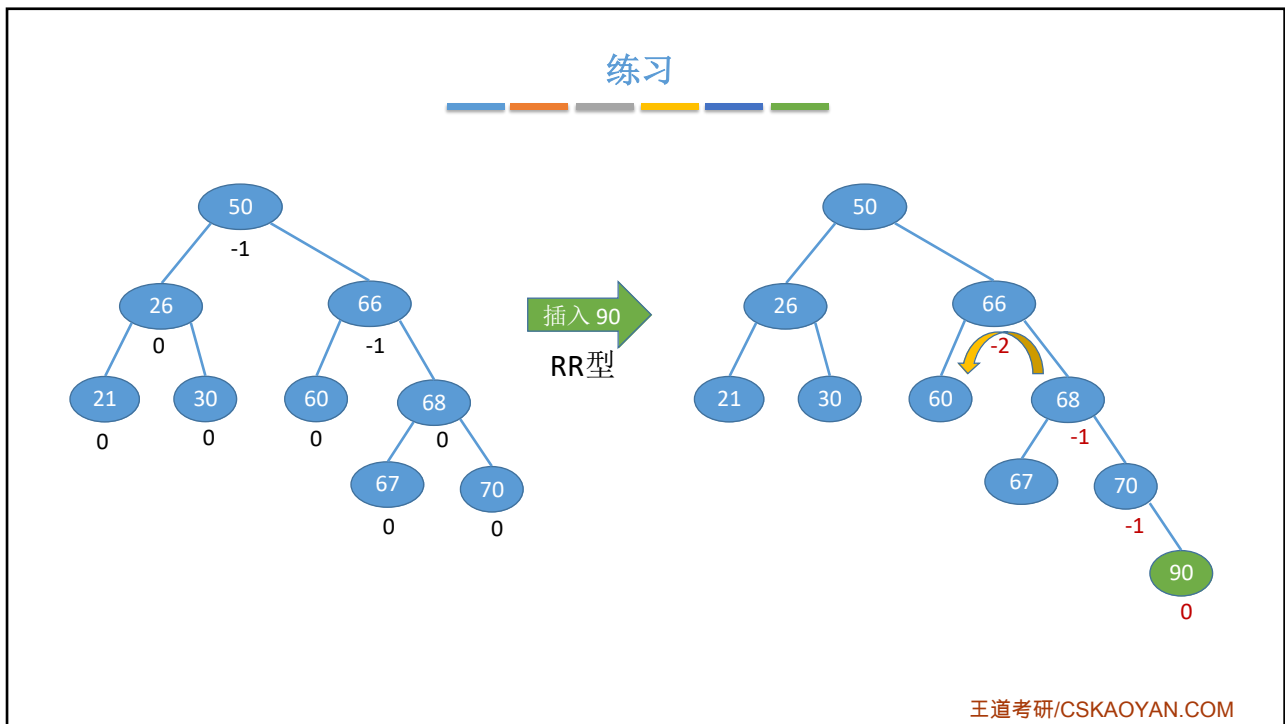
18



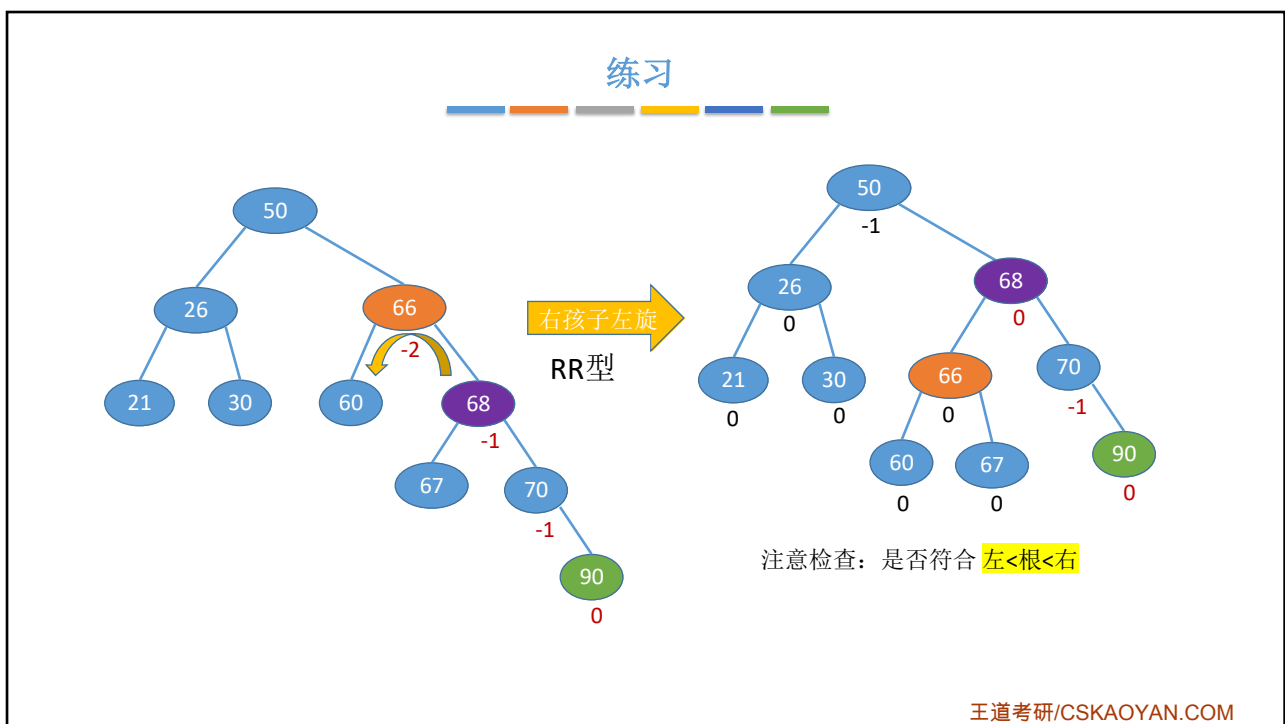
19



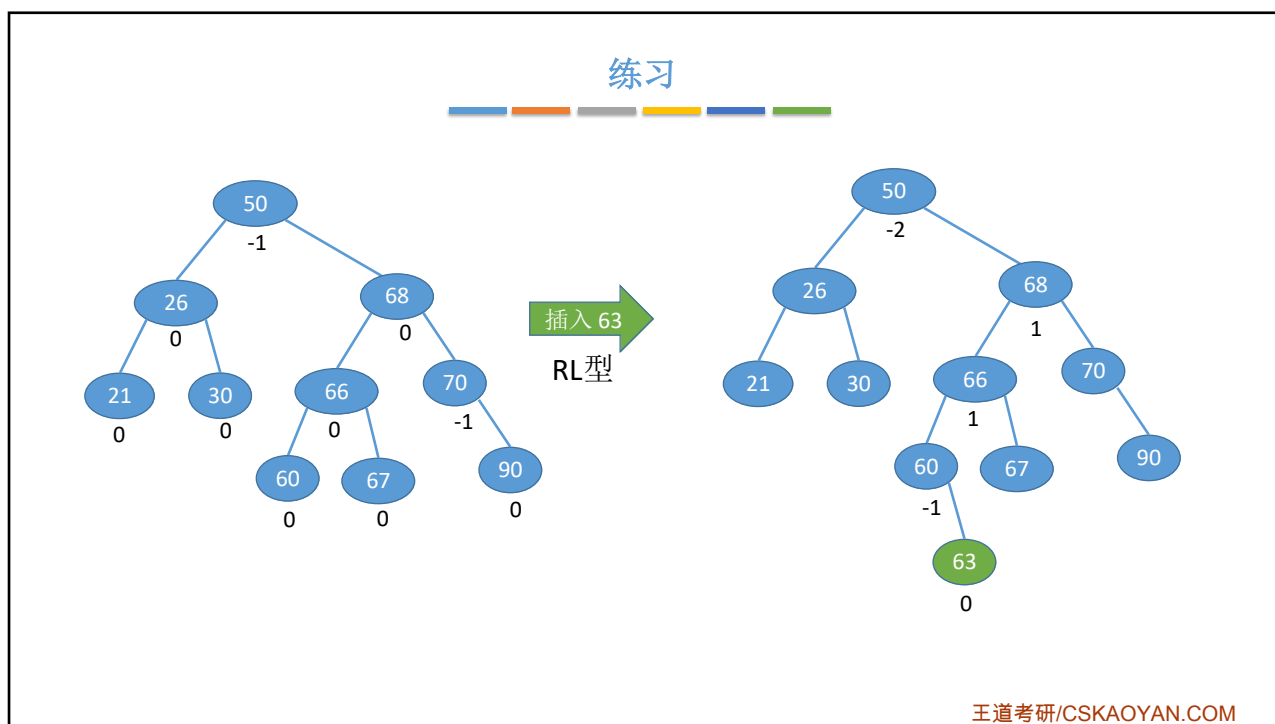
20



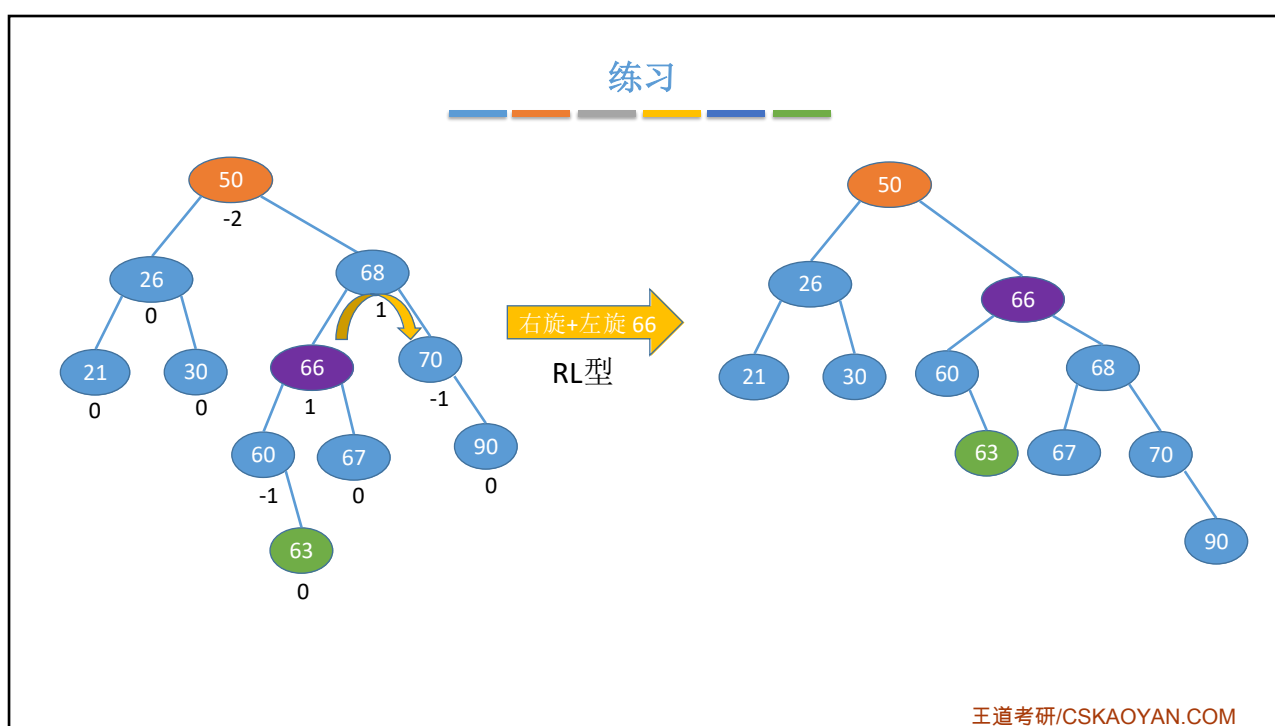
21



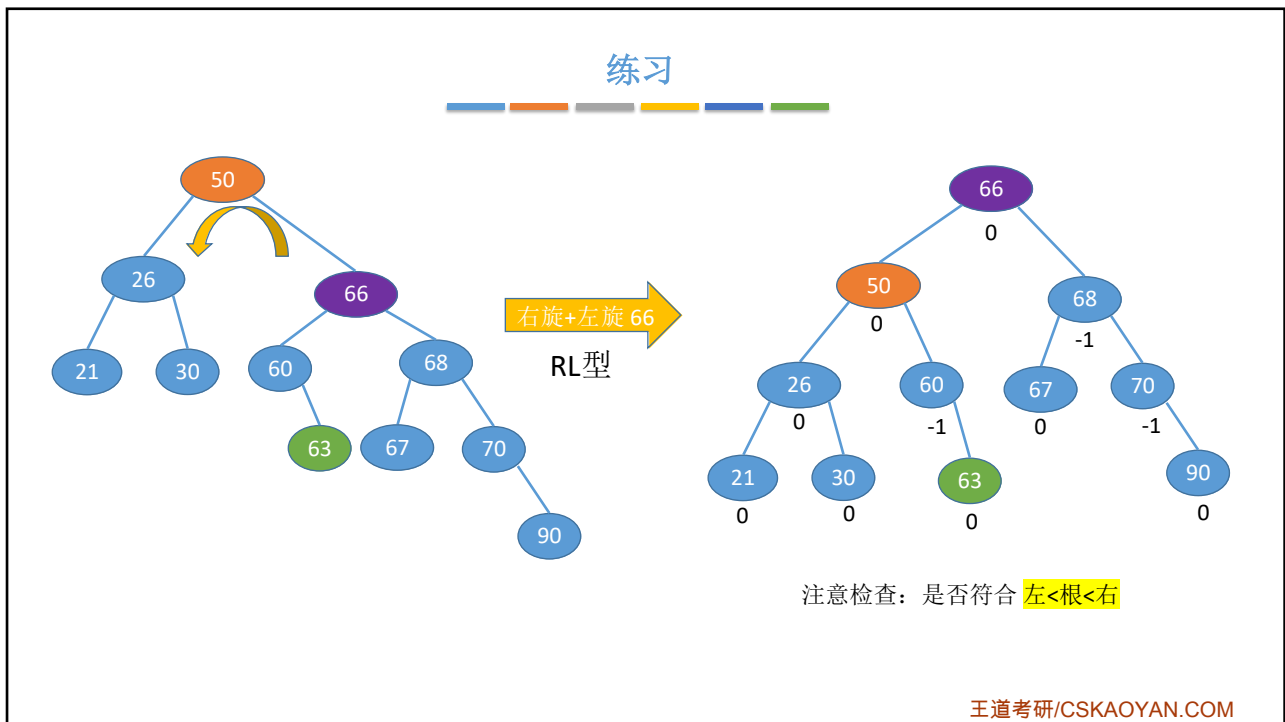
22



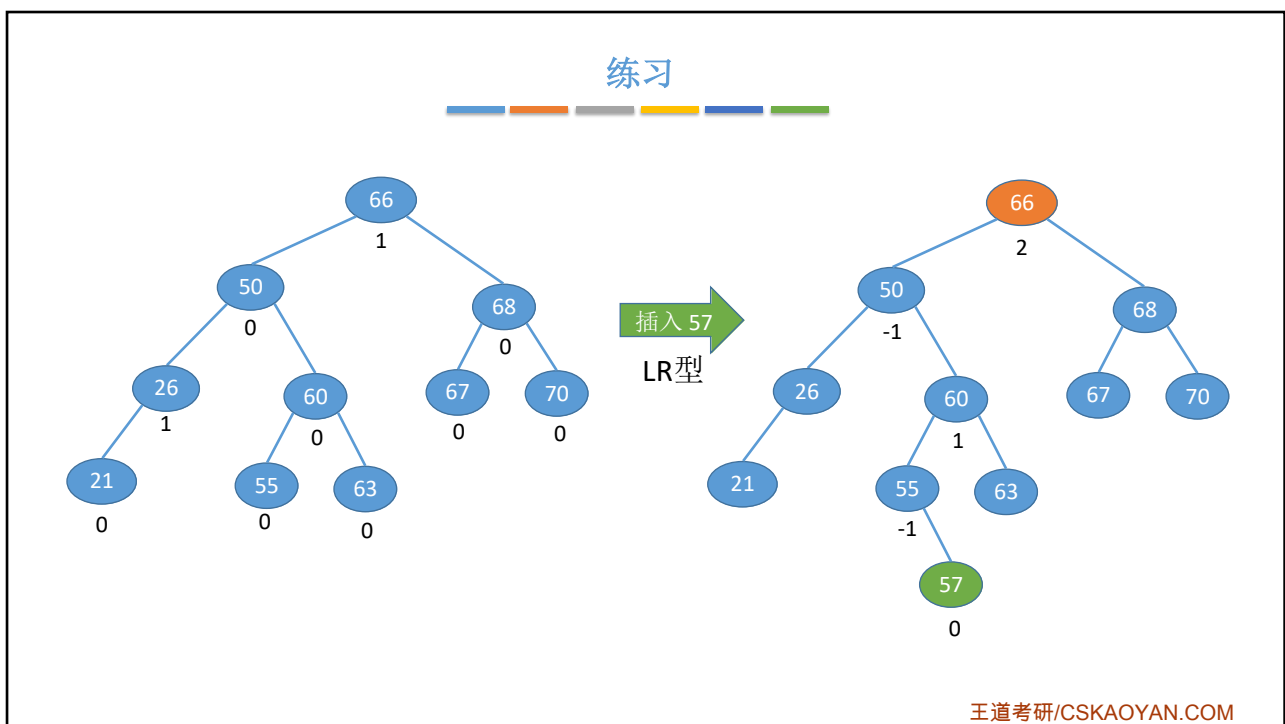
23



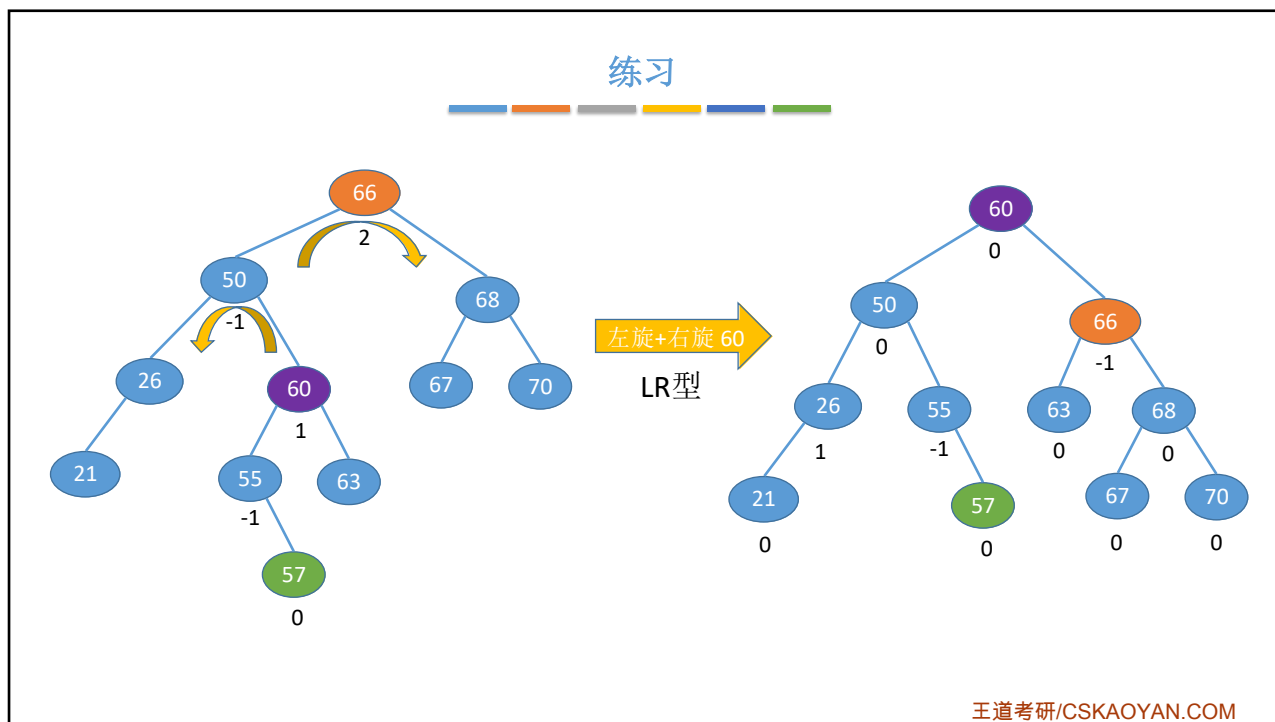
24



25



26



27

查找效率分析

若树高为 h , 则最坏情况下, 查找一个关键字最多需要对比 h 次, 即查找操作的时间复杂度不可能超过 $O(h)$

平衡二叉树——树上任一结点的左子树和右子树的高度之差不超过1。

假设以 n_h 表示深度为 h 的平衡树中含有的最少结点数。

则有 $n_0 = 0, n_1 = 1, n_2 = 2$, 并且有 $n_h = n_{h-1} + n_{h-2} + 1$

可以证明含有 n 个结点的平衡二叉树的最大深度为 $O(\log_2 n)$, 平衡二叉树的平均查找长度为 $O(\log_2 n)$

王道考研/CSKAOYAN.COM

28

查找效率分析

《An algorithm for the organization of information》——G.M. Adelson-Velsky 和 E.M. Landis, 1962



Figure 1

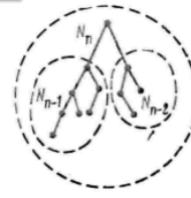


Figure 2

The recording algorithm is such that at each moment, the reference board is an admissible tree.
 Lemma 1. Let the number of cells of the admissible tree be equal to N . Then the maximum length of the branch is not greater than $(3/2) \log_2 (N + 1)$.

Proof. Let us denote by N_n the minimum number of cells in the admissible tree when the given maximum length of the branch is n . Then it can be easily proven (see Figure 2) that $N_n = N_{n-1} + N_{n-2} + 1$.

When we solve this equation in finite remainders, we get

$$N_n = \left(1 + \frac{2}{\sqrt{5}}\right) \left(\frac{1 + \sqrt{5}}{2}\right)^n + \left(1 - \frac{2}{\sqrt{5}}\right) \left(\frac{1 - \sqrt{5}}{2}\right)^n - 1.$$

Whence

$$n < \log_{\frac{1+\sqrt{5}}{2}} (N + 1) < \frac{3}{2} \log_2 (N + 1),$$

王道考研/CSKAOYAN.COM

29

知识回顾与重要考点

平衡二叉树

定义 ⊖ 树上任一结点的左子树和右子树的高度之差不超过1
 结点的平衡因子=左子树高-右子树高

插入操作 ⊖ 和二叉排序树一样, 找合适的位置插入
 新插入的结点可能导致其祖先们平衡因子改变, 导致失衡

调整“不平衡” ⊖ 找到最小不平衡子树进行调整, 记最小不平衡子树的根为A
 LL ⊖ 在A的左孩子的左子树插入导致A不平衡, 将A的左孩子右旋
 RR ⊖ 在A的右孩子的右子树插入导致A不平衡, 将A的右孩子左旋
 LR ⊖ 在A的左孩子的右子树插入导致A不平衡, 将A的左孩子的右孩子 先左旋再右旋
 RL ⊖ 在A的右孩子的左子树插入导致A不平衡, 将A的右孩子的左孩子 先右旋再左旋

查找效率分析 ⊖ 考点: 高为h的平衡二叉树最少有几个结点——递推求解
 平衡二叉树最大深度为 $O(\log n)$, 平均查找长度/查找的时间复杂度为 $O(\log n)$



实现 f 向右下旋转, p 向右上旋转:
 其中 f 是爹, p 为左孩子, gf 为 f 他爹
 ① $f \rightarrow lchild = p \rightarrow rchild$;
 ② $p \rightarrow rchild = f$;
 ③ $gf \rightarrow lchild/rchild = p$;



实现 f 向左下旋转, p 向左上旋转:
 ① $f \rightarrow rchild = p \rightarrow lchild$;
 ② $p \rightarrow lchild = f$;
 ③ $gf \rightarrow lchild/rchild = p$;

王道考研/CSKAOYAN.COM

30

本节内容

哈夫曼树

王道考研/CSKAOYAN.COM

1

知识总览

哈夫曼树

带权路径长度

哈夫曼树的定义

哈夫曼树的构造

哈夫曼编码

王道考研/CSKAOYAN.COM

2

带权路径长度

结点的**权**: 有某种现实含义的数值(如: 表示结点的重要性等)

结点的带权路径长度: 从树的根到该结点的路径长度(经过的边数)与该结点上权值的乘积

树的带权路径长度: 树中所有**叶结点**的带权路径长度之和(WPL, Weighted Path Length)

$$WPL = \sum_{i=1}^n w_i l_i$$

王道考研/CSKAOYAN.COM

3

哈夫曼树的定义

都是哈夫曼树

WPL=2*1+2*3+2*4+2*5=26

WPL=1*5+2*4+3*1+3*3=25

WPL=1*5+2*4+3*1+3*3=25

WPL=1*1+2*3+3*5+3*4=34

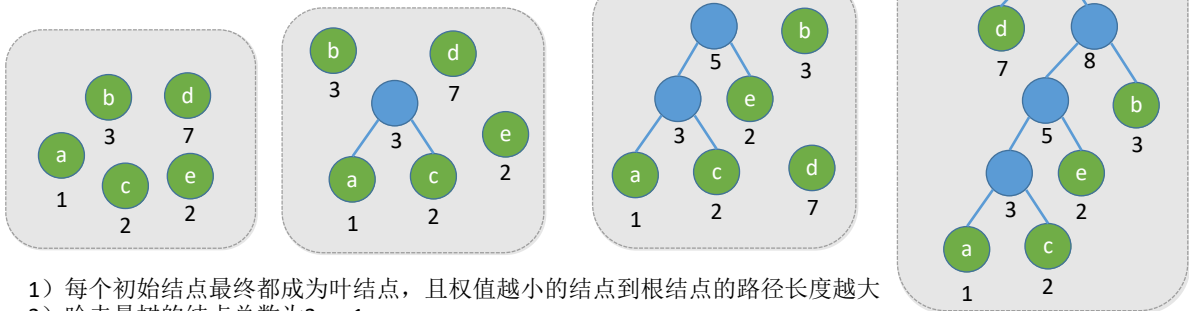
在含有n个带权叶结点的二叉树中, 其中**带权路径长度(WPL)最小的二叉树**称为**哈夫曼树**, 也称**最优二叉树**

4

哈夫曼树的构造

给定 n 个权值分别为 $w_1, w_2, \dots, w_n$ 的结点, 构造哈夫曼树的算法描述如下:

- 1) 将这 n 个结点分别作为 n 棵仅含一个结点的二叉树, 构成森林 F 。
- 2) 构造一个新结点, 从 F 中选取两棵根结点权值最小的树作为新结点的左、右子树, 并且将新结点的权值置为左、右子树上根结点的权值之和。
- 3) 从 F 中删除刚才选出的两棵树, 同时将新得到的树加入 F 中。
- 4) 重复步骤2) 和3), 直至 F 中只剩下一棵树为止。



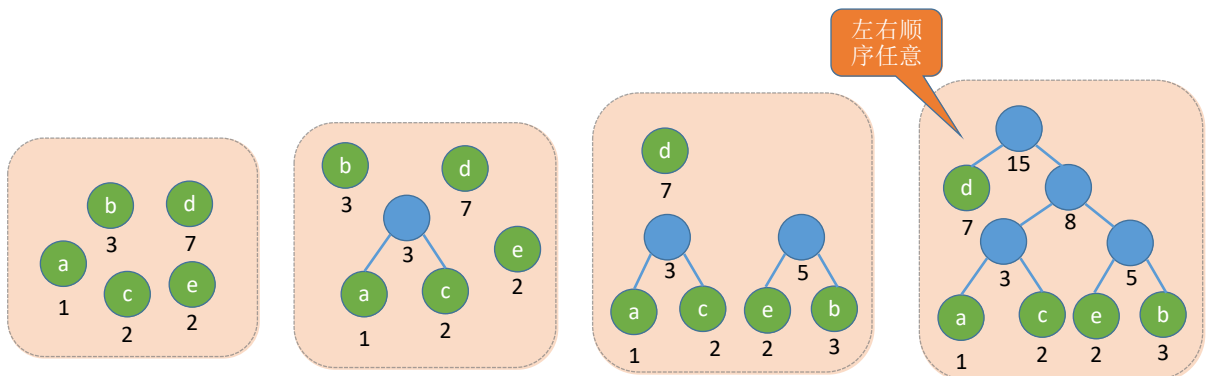
- 1) 每个初始结点最终都成为叶结点, 且权值越小的结点到根结点的路径长度越大
- 2) 哈夫曼树的结点总数为 $2n - 1$
- 3) 哈夫曼树中不存在度为1的结点。
- 4) 哈夫曼树并不唯一, 但WPL必然相同且为最优

$$WPL_{\min} = 1*7 + 2*3 + 3*2 + 4*1 + 4*2 = 31$$

王道考研/CSKAOYAN.COM

5

哈夫曼树的构造

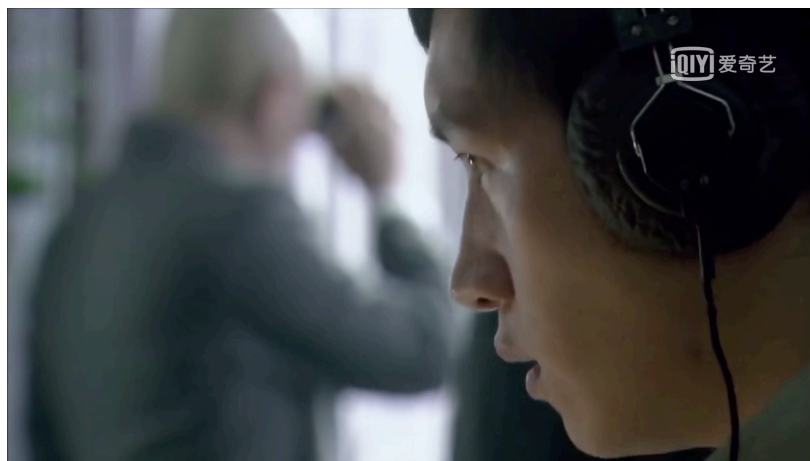


$$WPL = 1*7 + 3*(1+2+2+3) = 31$$

王道考研/CSKAOYAN.COM

6

哈夫曼编码



电报——点、划 两个信号(二进制0/1)

王道考研/CSKAOYAN.COM

7

哈夫曼编码

固定长度编码——每个字符用相等长度的二进制位表示

A—0100 0001
B—0100 0010
C—0100 0011
D—0100 0100

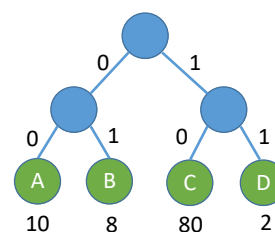
ASCII编码

A—00
B—01
C—10
D—11

每个字符用长度为2的二进制表示

假设, 100题中有80题选C, 10题选A, 8题选B, 2题选D

所有答案的二进制长度= $80*2+10*2+8*2+2*2=200$ bit



$$WPL = 80*2 + 10*2 + 8*2 + 2*2 = 200$$

王道考研/CSKAOYAN.COM

8

哈夫曼编码

固定长度编码——每个字符用相等长度的二进制位表示

A—00
B—01
C—10
D—11

每个字符用长度为2的二进制表示

假设, 100题中有80题选C, 10题选A, 8题选B, 2题选D
所有答案的二进制长度=80*2+10*2+8*2+2*2=200 bit

小渣 → 100个选择题 → 老渣

可变长度编码——允许对不同字符用不等长的二进制位表示

C—0
A—10
B—111
D—110

WPL = 80*2+10*2+8*2+2*2=200

WPL = 80*1+10*2+2*3+8*3=130

王道考研/CSKAOYAN.COM

9

哈夫曼编码

可变长度编码——允许对不同字符用不等长的二进制位表示

若没有一个编码是另一个编码的前缀, 则称这样的编码为前缀编码

前缀码解码无歧义

C—0
A—10
B—111
D—110

WPL = 80*1+10*2+2*3+8*3=130

CAAABD: 0101010111110

小渣 → CAAABD → 老渣

好的收到, CBBB

非前缀码解码有歧义

C—0
A—1
B—111
D—110

CAAABD: 0111111110

王道考研/CSKAOYAN.COM

10

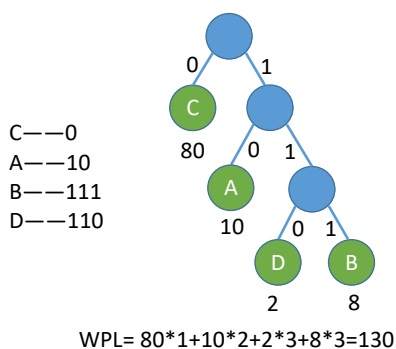
哈夫曼编码

固定长度编码——每个字符用相等长度的二进制位表示

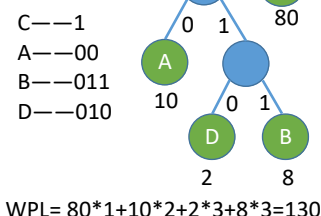
可变长度编码——允许对不同字符用不等长的二进制位表示

若没有一个编码是另一个编码的前缀, 则称这样的编码为前缀编码

有哈夫曼树得到哈夫曼编码——字符集中的每个字符作为一个叶子结点, 各个字符出现的频度作为结点的权值, 根据之前介绍的方法构造哈夫曼树



哈夫曼树不唯一, 因此哈夫曼编码不唯一



哈夫曼编码可用于数据压缩



王道考研/CSKAOYAN.COM

11

英文字母频次

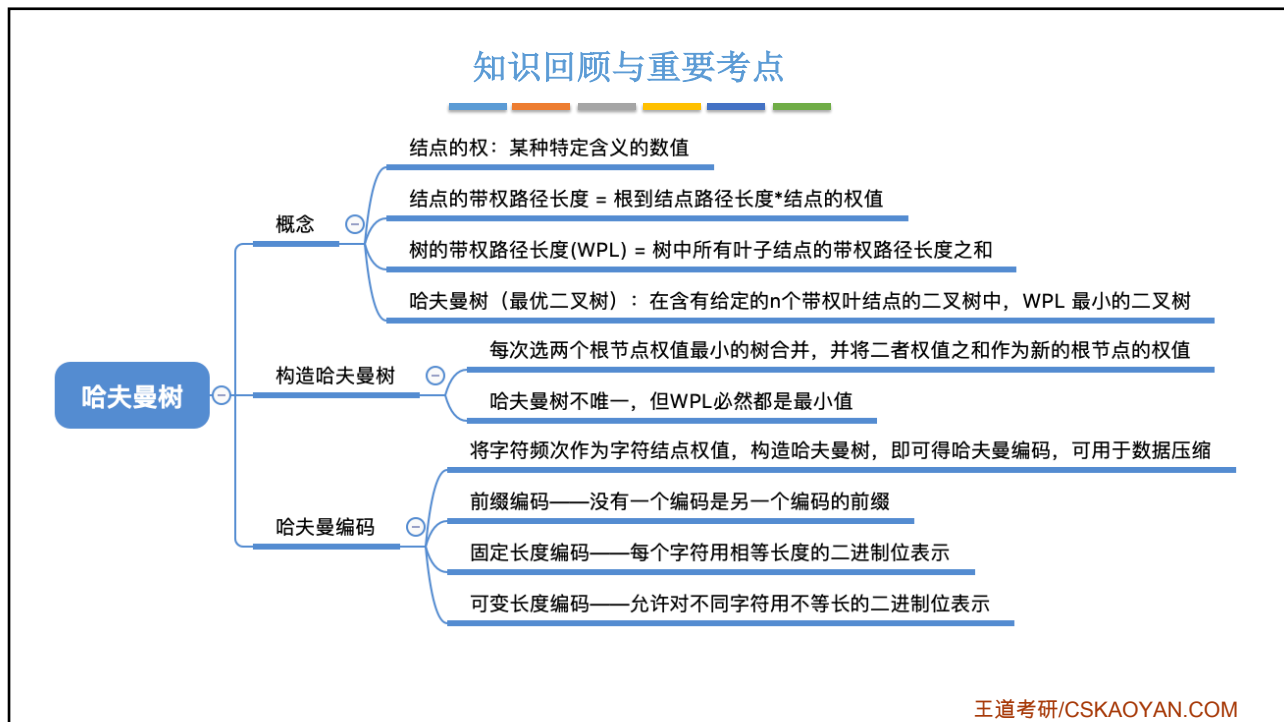
英文字母使用频率表:(%)

A 8.19	B 1.47	C 3.83	D 3.91	E 12.25	F 2.26	G 1.71
H 4.57	I 7.10	J 0.14	K 0.41	L 3.77	M 3.34	N 7.06
O 7.26	P 2.89	Q 0.09	R 6.85	S 6.36	T 9.41	
U 2.58	V 1.09	W 1.59	X 0.21	Y 1.58	Z 0.08	

试试设计哈夫曼编码, 并计算数据压缩率

王道考研/CSKAOYAN.COM

12



13

结局

可变长度编码——允许对不同字符用不等长的二进制位表示

若没有一个编码是另一个编码的前缀, 则称这样的编码为**前缀编码**

哈夫曼编码

```

graph TD
    Root(( )) ---|0| C((C  
80))
    Root ---|1| Node1(( ))
    Node1 ---|0| A((A  
10))
    Node1 ---|1| Node2(( ))
    Node2 ---|0| D((D  
2))
    Node2 ---|1| B((B  
8))
            
```

小渣 → 100道选择题 → 老渣

WPL= 80*1+10*2+2*3+8*3=130 bit

王道考研/CSKAOYAN.COM

14